

尚硅谷大数据技术之高频面试题

(作者：尚硅谷研究院)

版本：V9.1.3

目录

第 1 章 核心技术	11
1.1 Linux&Shell	11
1.1.1 Linux 常用高级命令	11
1.1.2 Shell 常用工具及写过的脚本	11
1.1.3 Shell 中单引号和双引号区别	12
1.2 Hadoop	13
1.2.1 Hadoop 常用端口号	13
1.2.2 HDFS 读流程和写流程	13
1.2.3 HDFS 小文件处理	14
1.2.4 HDFS 的 NameNode 内存	15
1.2.5 Shuffle 及优化	15
1.2.6 Yarn 工作机制	16
1.2.7 Yarn 调度器	17
1.2.8 HDFS 块大小	18
1.2.9 Hadoop 脑裂原因及解决办法?	18
1.3 Zookeeper	19
1.3.1 常用命令	19
1.3.2 选举机制	19
1.3.3 Zookeeper 符合法则中哪两个?	20
1.3.4 Zookeeper 脑裂	20
1.3.5 Zookeeper 用来干嘛了	20
1.4 Flume	21
1.4.1 Flume 组成, Put 事务, Take 事务	21
1.4.2 Flume 拦截器	22
1.4.3 Flume Channel 选择器	23
1.4.4 Flume 监控器	23
1.4.5 Flume 采集数据会丢失吗?	23
1.4.6 Flume 如何提高吞吐量	24

1.5 Kafka	24
1.5.1 Kafka 架构	24
1.5.2 Kafka 生产端分区分配策略	26
1.5.3 Kafka 丢不丢数据	27
1.5.4 Kafka 的 ISR 副本同步队列	27
1.5.5 Kafka 数据重复	28
1.5.6 Kafka 如何保证数据有序 or 怎么解决乱序	29
1.5.7 Kafka 分区 Leader 选举规则	30
1.5.8 Kafka 中 AR 的顺序	30
1.5.9 Kafka 日志保存时间	31
1.5.10 Kafka 过期数据清理	32
1.5.11 Kafka 为什么能高效读写数据	32
1.5.12 自动创建主题	33
1.5.13 副本数设定	34
1.5.14 Kafka 分区数	34
1.5.15 Kafka 增加分区	34
1.5.16 Kafka 中多少个 Topic	35
1.5.17 Kafka 消费者是拉取数据还是推送数据	35
1.5.18 Kafka 消费端分区分配策略	35
1.5.19 消费者再平衡的条件	36
1.5.20 指定 Offset 消费	37
1.5.21 指定时间消费	37
1.5.22 Kafka 监控	38
1.5.23 Kafka 数据积压	38
1.5.24 如何提升吞吐量	39
1.5.25 Kafka 中数据量计算	40
1.5.26 Kafka 如何压测?	40
1.5.27 磁盘选择	41
1.5.28 内存选择	42
1.5.29 CPU 选择	43
1.5.30 网络选择	43
1.5.31 Kafka 挂掉	44
1.5.32 Kafka 的机器数量	44
1.5.33 服役新节点退役旧节点	44

1.5.34 Kafka 单条日志传输大小	44
1.5.35 Kafka 参数优化	45
1.6 Hive	46
1.6.1 Hive 的架构	46
1.6.2 HQL 转换为 MR 流程	46
1.6.3 Hive 和数据库比较	47
1.6.4 内部表和外部表	48
1.6.5 系统函数	48
1.6.6 自定义 UDF、UDTF 函数	49
1.6.7 窗口函数	49
1.6.8 Hive 优化	51
1.6.9 Hive 解决数据倾斜方法	58
1.6.10 Hive 的数据中含有字段的分隔符怎么处理?	63
1.6.11 MySQL 元数据备份	63
1.6.12 如何创建二级分区表?	64
1.6.13 Union 与 Union all 区别	64
1.7 Datax	64
1.7.1 DataX 与 Sqoop 区别	64
1.7.2 速度控制	65
1.7.3 内存调整	65
1.7.4 空值处理	66
1.7.5 配置文件生成脚本	66
1.7.6 DataX 一天导入多少数据	67
1.7.7 Datax 如何实现增量同步	67
1.8 Maxwell	67
1.8.1 Maxwell 与 Canal、FlinkCDC 的对比	67
1.8.2 Maxwell 好处	68
1.8.3 Maxwell 底层原理	68
1.8.4 全量同步速度如何	68
1.8.5 Maxwell 数据重复问题	68
1.9 DolphinScheduler 调度器	68
1.9.1 每天集群运行多少指标?	68
1.9.2 任务挂了怎么办?	68
1.9.3 DS 挂了怎么办?	69

1.10 Spark Core & SQL	69
1.10.1 Spark 运行模式	69
1.10.2 Spark 常用端口号	69
1.10.3 RDD 五大属性	70
1.10.4 RDD 弹性体现在哪里	70
1.10.5 Spark 的转换算子（8 个）	70
1.10.6 Spark 的行动算子（5 个）	71
1.10.7 map 和 mapPartitions 区别	71
1.10.8 Repartition 和 Coalesce 区别	72
1.10.9 reduceByKey 与 groupByKey 的区别	72
1.10.10 Spark 中的血缘	72
1.10.11 Spark 任务的划分	72
1.10.12 SparkSQL 中 RDD、DataFrame、DataSet 三者的转换及区别	73
1.10.13 Hive on Spark 和 Spark on Hive 区别	73
1.10.14 Spark 内核源码（重点）	74
1.10.15 Spark 统一内存模型	76
1.10.16 Spark 为什么比 MR 快？	76
1.10.17 Spark Shuffle 和 Hadoop Shuffle 区别？	77
1.10.18 Spark 提交作业参数（重点）	77
1.10.19 Spark 任务使用什么进行提交，JavaEE 界面还是脚本	78
1.10.20 请列举会引起 Shuffle 过程的 Spark 算子，并简述功能。	78
1.10.21 Spark 操作数据库时，如何减少 Spark 运行中的数据库连接数？	78
1.10.22 Spark 数据倾斜	78
1.12 Flink	79
1.12.1 Flink 基础架构组成？	79
1.12.2 Flink 和 Spark Streaming 的区别？	79
1.12.3 Flink 提交作业流程及核心概念	80
1.12.4 Flink 的部署模式及区别？	81
1.12.5 Flink 任务的并行度优先级设置？资源一般如何配置？	81
1.12.6 Flink 的三种时间语义	82
1.12.7 你对 Watermark 的认识	82
1.12.8 Watermark 多并行度下的传递、生成原理	82
1.12.9 Flink 怎么处理乱序和迟到数据？	83
1.12.10 说说 Flink 中的窗口（分类、生命周期、触发、划分）	84

1.12.11 Flink 的 keyby 怎么实现的分区？分区、分组的区别是什么？	85
1.12.12 Flink 的 Interval Join 的实现原理？Join 不上的怎么办？	85
1.12.13 介绍一下 Flink 的状态编程、状态机制？	85
1.12.14 Flink 如何实现端到端一致性？.....	86
1.12.15 Flink 分布式快照的原理是什么	86
1.12.16 Checkpoint 的参数怎么设置的？	87
1.12.17 Flink 内存模型（重点）	88
1.12.18 Flink 常见的维表 Join 方案	88
1.12.19 FlinkCDC 锁表问题	88
1.13 HBase	89
1.13.1 HBase 存储结构	89
1.13.2 HBase 的写流程	90
1.13.3 HBase 的读流程	91
1.13.6 HBase 的合并	92
1.13.7 RowKey 设计原则	92
1.13.8 RowKey 如何设计	92
1.13.9 HBase 二级索引原理	93
1.14 Clickhouse	93
1.14.1 Clickhouse 的优势	93
1.14.2 Clickhouse 的引擎	93
1.14.3 Flink 写入 Clickhouse 怎么保证一致性？	94
1.14.4 Clickhouse 存储多少数据？几张表？	94
1.14.5 Clickhouse 使用本地表还是分布式表	94
1.14.6 Clickhouse 的物化视图	94
1.14.8 Clickhouse 的优化	95
1.14.9 Clickhouse 的新特性 Projection	95
1.14.10 Clickhouse 的索引、底层存储	96
1.15 可视化报表工具	97
1.16 Doris	97
1.17 Sqoop	97
1.18 Azkaban	99
1.19 JavaSE	99
1.19.1 什么是多线程&多线程的优点	99
1.19.2 如何创建多线程	99

1.19.3 如何创建线程池	99
1.19.4 ThreadPoolExecutor 构造函数参数解析	100
1.19.5 列举线程安全的 Map 集合	100
1.19.6 StringBuffer 和 StringBuilder 的区别	100
1.19.7 ArrayList 和 LinkedList 的区别	100
1.19.8 HashMap 和 Hashtable 的区别	100
1.19.9 HashMap 的底层原理	101
1.19.10 HashMap 里面放 100 条数据，初始化应该是多少	102
1.20 MySQL	103
1.20.1 MyISAM 与 InnoDB 的区别	103
1.20.2 MySQL 四种索引	103
1.20.3 MySQL 的事务	103
1.20.4 MySQL 事务隔离级别	104
1.20.5 MyISAM 与 InnoDB 对比	104
1.20.6 B 树和 B+树对比	105
1.21 Redis	105
1.21.1 Redis 缓存穿透、缓存雪崩、缓存击穿	105
1.21.2 Redis 哨兵模式	106
1.21.3 Redis 数据类型	106
1.21.4 热数据通过什么样的方式导入 Redis	106
1.21.5 Redis 的存储模式 RDB，AOF	107
1.21.6 Redis 存储的是 k-v 类型，为什么还会有 Hash?	107
1.22 JVM	107
第 2 章 离线数仓项目	107
2.1 提高自信	107
2.2 为什么做这个项目	108
2.3 数仓概念	108
2.4 项目架构	109
2.5 框架版本选型	109
2.6 服务器选型	111
2.7 集群规模	111
2.8 人员配置参考	114
2.8.1 整体架构	114
2.8.2 你的职级等级及晋升规则	114

2.8.3 人员配置参考	115
2.9 从 0-1 搭建项目，你需要做什么？	116
2.10 数仓建模准备	117
2.11 数仓建模	119
2.12 数仓每层做了哪些事	121
2.13 数据量	123
2.14 项目中遇到哪些问题？（****）	124
2.15 离线--业务	125
2.15.1 SKU 和 SPU	125
2.15.2 订单表跟订单详情表区别？	125
2.15.3 上卷和下钻	125
2.15.4 TOB 和 TOC 解释	126
2.15.5 流转 G 复活指标	126
2.15.6 活动的话，数据量会增加多少？怎么解决？	127
2.15.7 哪个商品卖的好？	127
2.15.8 数据仓库每天跑多少张表，大概什么时候运行，运行多久？	127
2.15.9 哪张表数据量最大	128
2.15.10 哪张表最费时间，有没有优化	129
2.15.11 并发峰值多少？大概哪个时间点？	130
2.15.12 分析过最难的指标	130
2.15.13 数仓中使用的哪种文件存储格式	130
2.15.14 数仓当中数据多久删除一次	130
2.15.15 Mysql 业务库中某张表发生变化，数仓中表需要做什么改变	130
2.15.16 50 多张表关联，如何进行性能调优	130
2.15.17 拉链表的退链如何实现	130
2.15.18 离线数仓如何补数	131
2.15.19 产品经理给新指标，该如何开发	131
2.15.20 新出指标，原有建模无法实现，如何操作	131
2.15.21 和哪些部门沟通，以及沟通什么内容	131
2.15.22 你们的需求和指标都是谁给的	131
2.15.23 业务场景：时间跨度比较大，数据模型的数据怎么更新的，例如：借款， 使用一年，再还款，这个数据时间跨度大，在处理的时候怎么处理	131
2.15.24 map 任务从 0%到 1%很慢，但是从 1%到 100%很快，期间出了什么问题？	132

2.15.25 数据倾斜场景除了 group by 和 join 外, 还有哪些场景	132
2.15.26 没有意义的用户是怎么判断的	132
2.15.27 你的公司方向是电子商务, 自营的还是提货平台? 你们会有自己的商品吗?	132
2.15.28 ods 事实表中订单状态会发生变化, 你们是通过什么方式去监测数据变化的	133
2.15.29 当天订单没有闭环结束的数据量?	133
2.15.30 你们维度数据要做 ETL 吗? 除了用户信息脱敏? 没有做其他 ETL 吗? ..	133
2.15.31 怎么做加密, 加密数据要用怎么办, 我讲的 md5, 他问我 md5 怎么做恢复	133
2.15.32 表生命周期管理怎么做的?	133
2.15.33 当 ADS 计算完, 如何判断指标是正确的	134
2.15.34 ADS 层指标计算错误, 如何解决	134
2.15.35 任务跑起来之后, 整个集群资源占用比例	134
2.15.36 真实项目流程	135
2.16 埋点	135
第 3 章 实时数仓项目	137
3.1 为什么做这个项目	137
3.2 项目架构	137
3.3 框架版本选型	137
3.4 服务器选型	137
3.5 集群规模	137
3.6 项目建模	138
3.7 数据量	140
3.7.1 数据分层数据量	140
3.7.2 实时组件存储数据量	141
3.7.3 实时 QPS 峰值数据量	141
3.8 项目中遇到哪些问题及如何解决?	141
3.8.1 业务数据采集框架选型问题	141
3.8.2 项目中哪里用到状态编程, 状态是如何存储的, 怎么解决大状态问题	141
3.8.3 项目中哪里遇到了反压, 造成的危害, 定位解决 (*重点*)	142
3.8.4 数据倾斜问题如何解决 (****重点****)	143
3.8.5 数据如何保证一致性问题	143
3.8.6 FlinkSQL 性能比较慢如何优化	143

3.8.7 Kafka 分区动态增加, Flink 监控不到新分区数据导致数据丢失	144
3.8.9 Kafka 某个分区没有数据, 导致下游水位线无法抬升, 窗口无法关闭计算	144
3.8.10 Hbase 的 rowkey 设计不合理导致的数据热点问题	144
3.8.11 Redis 和 HBase 的数据不一致问题	144
3.8.12 双流 join 关联不上如何解决	145
3.9 生产经验	145
3.9.1 Flink 任务提交使用那种模式, 为何选用这种模式	145
3.9.2 Flink 任务提交参数, JobManager 和 TaskManager 分别给多少	145
3.9.3 Flink 任务并行度如何设置	146
3.9.4 项目中 Flink 作业 Checkpoint 参数如何设置	146
3.9.5 迟到数据如何解决	146
3.9.6 实时数仓延迟多少	146
3.9.7 项目开发多久, 维护了多久	146
3.9.8 如何处理缓存冷启动问题	146
3.9.9 如何处理动态分流冷启动问题 (主流数据先到, 丢失数据怎么处理)	147
3.9.10 代码升级, 修改代码, 如何上线	147
3.9.11 如果现在做了 5 个 Checkpoint, Flink Job 挂掉之后想恢复到第三次 Checkpoint 保存的状态上, 如何操作	147
3.9.12 需要使用 flink 记录一群人, 从北京出发到上海, 记录出发时间和到达时间, 同时要显示每个人用时多久, 需要实时显示, 如果让你来做, 你怎么设计? ...	147
3.9.13 flink 内部的数据质量和数据的时效怎么把控的	147
3.9.14 实时任务问题 (延迟) 怎么排查	147
3.9.15 维度数据查询并发量	148
3.9.16 Prometheus+Grafana 是自己搭的吗, 监控哪些指标	148
3.9.17 怎样在不停止任务的情况下改 flink 参数	148
3.9.18 hbase 中有表, 里面的 1 月份到 3 月份的数据我不要了, 我需要删除它 (彻底删除), 要怎么做	148
3.9.19 如果 flink 程序的数据倾斜是偶然出现的, 可能白天可能晚上突然出现, 然后几个月都没有出现, 没办法复现, 怎么解决?	149
3.9.20 维度数据改变之后, 如何保证新 join 的维度数据是正确的数据	149
3.10 实时---业务	149
3.10.1 数据采集到 ODS 层	149
3.10.2 ODS 层	150
3.10.3 DWD+DIM 层	150

3.10.4 DWS 层	151
3.10.5 ADS 层	153
第 4 章 用户画像项目	153
4.1 画像系统主要做了哪些事	153
4.2 项目整体架构	154
4.3 讲一下标签计算的调度过程	154
4.4 整个标签的批处理过程	154
4.5 你们的画像平台有哪些功能 ?	155
4.6 是否做过 Web 应用开发, 实现了什么功能	155
4.7 画像平台的上下游	155
4.8 BitMap 原理, 及为什么可以提高性能	155
第 5 章 数据湖项目	156
5.1 数据湖与数据仓库对比	156
5.2 为什么做这个项目? 解决了什么痛点?	156
5.3 项目架构	157
5.4 业务	157
5.5 优化 or 遇到的问题怎么解决	157
第 6 章 测试&上线流程	158
6.1 测试相关	158
6.1.1 公司有多少台测试服务器?	158
6.1.2 测试服务器配置?	158
6.1.3 测试数据哪来的?	158
6.1.4 如何保证写的 SQL 正确性 (重点)	158
6.1.5 测试之后如何上线?	159
6.1.6 A/B 测试了解	159
6.2 项目实际工作流程	161
6.3 项目当前版本号是多少? 多久升级一次版本	163
6.4 项目中实现一个需求大概多长时间	163
6.5 项目开发中每天做什么事	163
第 7 章 数据治理	164
7.1 元数据管理	164
7.2 数据质量监控	165
7.2.1 实现数据质量监控, 你具体怎么做, 详细说?	165
7.2.2 监控原则	166

7.2.3 数据质量实现	167
7.3 权限管理 (Ranger)	167
7.4 用户认证 (Kerberos)	167
7.5 数据治理	168
第 8 章 中台	170
8.1 什么是中台?	171
8.2 各家台	172
8.3 中台具体划分	172
8.4 中台使用场景	173
8.5 中台的痛点	174
第 9 章 算法题 (LeetCode)	174
9.1 时间复杂度、空间复杂度理解	174
9.2 常见算法求解思想	174
9.3 基本算法	175
9.3.1 冒泡排序	175
9.3.2 快速排序	175
9.3.3 归并排序	176
9.3.4 遍历二叉树	177
9.3.5 二分查找	177
9.4 小青蛙跳台阶	178
9.5 最长回文子串	178
9.6 数字字符转化成 IP	178
第 10 章 场景题	179
10.1 手写 Flink 的 UV	179
10.2 Flink 的分组 TopN	179
10.3 Spark 的分组 TopN	179
10.4 如何快速从 40 亿条数据中快速判断, 数据 123 是否存在	179
10.5 给你 100G 数据, 1G 内存, 如何排序?	179
10.6 公平调度器容器集中在同一个服务器上?	179
10.7 匹马赛跑, 1 个赛道, 每次 5 匹进行比赛, 无法对每次比赛计时, 但知道每次比赛结果的先后顺序, 最少赛多少次可以找出前三名?	180
10.8 给定一个点、一条线、一个三角形、一个有向无环图, 请用 java 面向对象的思想进行建模	180
10.9 现场出了一道 sql 题, 让说出 sql 的优化, 优化后效率提升了多少	180

第 11 章 HQL 场景题	180
第 12 章 面试说明	180
12.1 面试过程最关键的是什么?	180
12.2 面试时该怎么说?	180
12.3 面试技巧	181
12.3.1 六个常见问题	181
12.3.2 两个注意事项	182
12.3.3 自我介绍	182

第 1 章 核心技术

1.1 Linux&Shell

1.1.1 Linux 常用高级命令

序号	命令	命令解释
1	top	实时显示系统中各个进程的资源占用状况（CPU、内存和执行时间）
2	jmap -heap 进程号	查看某个进程内存
3	free -m	查看系统内存使用情况
4	ps -ef	查看进程
5	netstat -tunlp grep 端口号	查看端口占用情况
6	du -sh 路径*	查看路径下的磁盘使用情况 例如：\$ du -sh /opt/
7	df -h	查看磁盘存储情况

1.1.2 Shell 常用工具及写过的脚本

1) awk、sed、cut、sort

2) 用 Shell 写过哪些脚本

（1）集群启动，分发脚本

```
#!/bin/bash
```

```
case $1 in
```

```
"start")
  for i in hadoop102 hadoop103 hadoop104
  do
    ssh $i "绝对路径"
  done
;;
"stop")
;;
esac
```

(2) 数仓层级内部的导入: ods->dwd->dws ->ads

```
①#!/bin/bash
②定义变量 APP=gmall
③获取时间
  传入 按照传入时间
  不传 T+1
④sql="
  先按照当前天 写 sql => 遇到时间 $do_date 遇到表 {$APP}.
  自定义函数 UDF UDTF {$APP}.
"
⑤执行 sql
```

1.1.3 Shell 中单引号和双引号区别

1) 在/home/atguigu/bin 创建一个 test.sh 文件

```
[atguigu@hadoop102 bin]$ vim test.sh
```

在文件中添加如下内容

```
#!/bin/bash
do_date=$1

echo '$do_date'
echo "$do_date"
echo "'$do_date'"
echo '"$do_date"'
echo `date`
```

2) 查看执行结果

```
[atguigu@hadoop102 bin]$ test.sh 2022-02-10
$do_date
2022-02-10
'2022-02-10'
"$do_date"
2022 年 05 月 02 日 星期四 21:02:08 CST
```

3) 总结:

- (1) 单引号不取变量值
- (2) 双引号取变量值
- (3) 反引号`, 执行引号中命令
- (4) 双引号内部嵌套单引号, 取出变量值

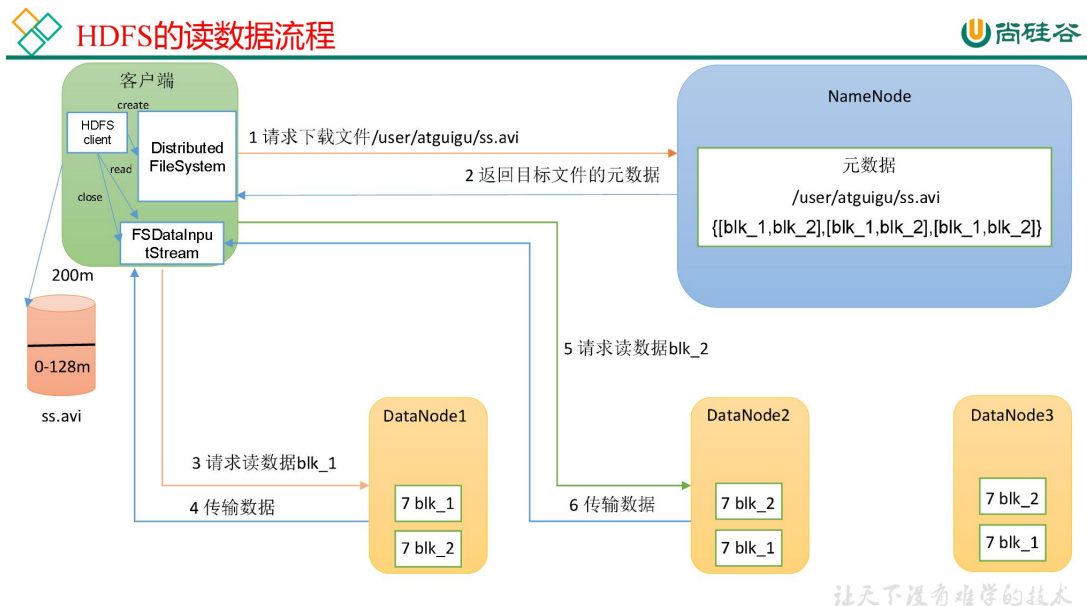
(5) 单引号内部嵌套双引号，不取出变量值

1.2 Hadoop

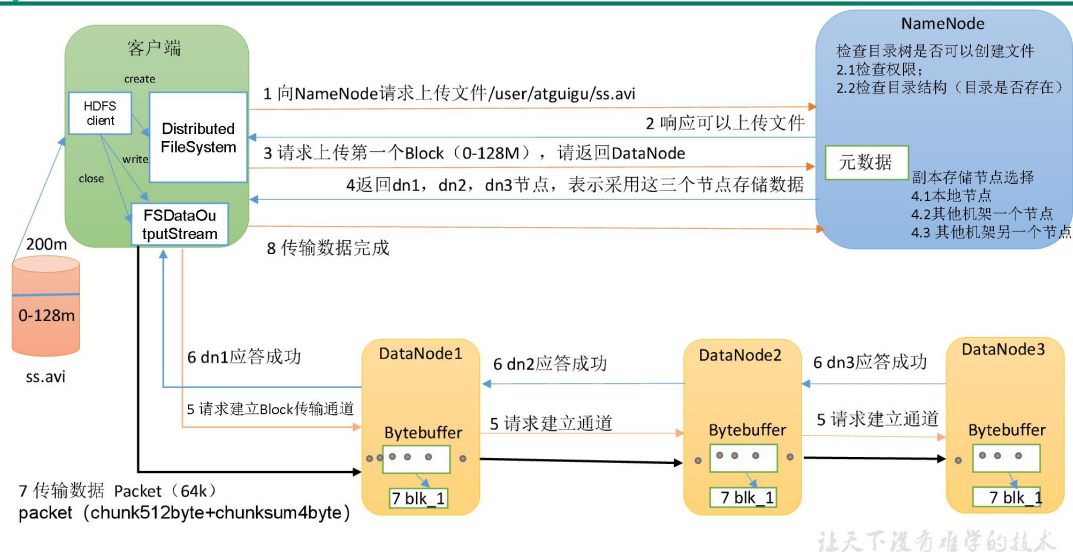
1.2.1 Hadoop 常用端口号

	hadoop2.x	hadoop3.x
访问 HDFS 端口	50070	9870
访问 MR 执行情况端口	8088	8088
历史服务器	19888	19888
客户端访问集群端口	9000	8020

1.2.2 HDFS 读流程和写流程



HDFS的写数据流程



注意：HDFS 写入流程时候，某台 dataNode 挂掉如何运行？

当 DataNode 突然挂掉了，客户端接收不到这个 DataNode 发送的 ack 确认，客户端会通知 NameNode，NameNode 检查并确认该块的副本与规定的不符，NameNode 会通知闲置的 DataNode 去复制副本，并将挂掉的 DataNode 作下线处理。等挂掉的 DataNode 节点恢复后，删除该节点中曾经拷贝的不完整副本数据。

1.2.3 HDFS 小文件处理

1) 会有什么影响

(1) 存储层面

1 个文件块，占用 namenode 多大内存 150 字节

128G 能存储多少文件块？ $128\text{ g} * 1024\text{ m} * 1024\text{ kb} * 1024\text{ byte} / 150\text{ 字节} = 9.1\text{ 亿文件块}$

(2) 计算层面

每个小文件都会起到一个 MapTask，1 个 MapTask 默认内存 1G。浪费资源。

2) 怎么解决

(1) 采用 har 归档方式，将小文件归档

(2) 采用 `CombineTextInputFormat`

(3) 自己写一个 MR 程序将产生的小文件合并成一个大文件。如果是 Hive 或者 Spark 有 merge 功能自动帮助我们合并。

(4) 有小文件场景开启 JVM 重用；如果没有小文件，不要开启 JVM 重用，因为会一直占用使用到的 Task 卡槽，直到任务完成才释放。

JVM 重用可以使得 JVM 实例在同一个 job 中重新使用 N 次，N 的值可以在 Hadoop 的 mapred-site.xml 文件中进行配置。通常在 10-20 之间。

```
<property>
  <name>mapreduce.job.jvm.numtasks</name>
  <value>10</value>
  <description>How many tasks to run per jvm,if set to -1 ,there is no limit</description>
</property>
```

1.2.4 HDFS 的 NameNode 内存

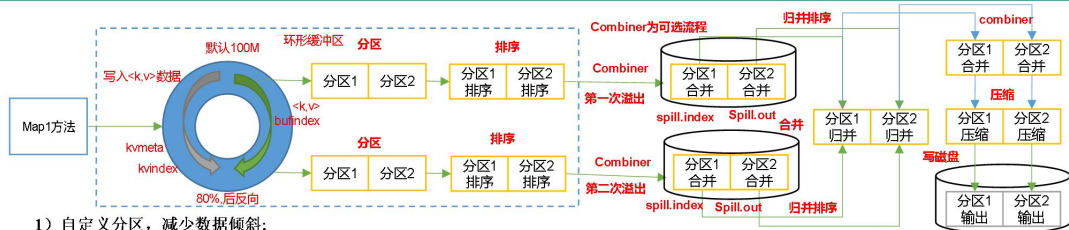
1) Hadoop2.x 系列，配置 NameNode 默认 2000m

2) Hadoop3.x 系列，配置 NameNode 内存是动态分配的

NameNode 内存最小值 1G，每增加 100 万个文件 block，增加 1G 内存。

1.2.5 Shuffle 及优化

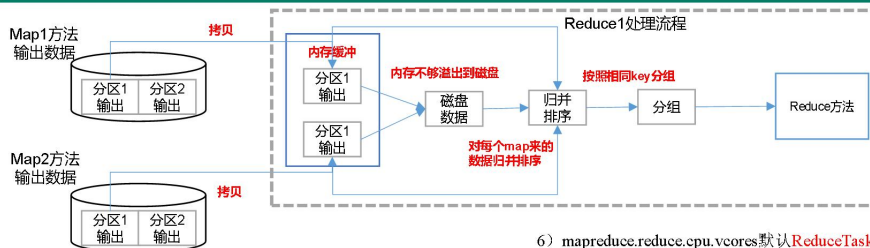
MapReduce优化 (上)



- 1) 自定义分区，减少数据倾斜；
定义类，实现 Partitioner 接口，重写 getPartition 方法
- 2) 减少溢写的次数
mapreduce.task.io.sort.mb
Shuffle 的环形缓冲区大小，默认 100m，可以提高到 200m
mapreduce.map.sort.spill.percent
环形缓冲区溢出的阈值，默认 80%，可以提高到 90%
- 3) 增加每次 Merge 合并次数
mapreduce.task.io.sort.factor 默认 10，可以提高到 20
- 4) 在不影响业务结果的前提下可以提前采用 Combiner
job.setCombinerClass(xxxReducer.class);
- 5) 为了减少磁盘 IO，可以采用 Snappy 或者 LZ4 压缩
conf.setBoolean("mapreduce.map.output.compress", true);
conf.setClass("mapreduce.map.output.compress.codec",
SnappyCodec.class, CompressionCodec.class);

- 6) mapreduce.map.memory.mb 默认 MapTask 内存上限 1024MB。
可以根据 128m 数据对应 1G 内存原则提高该内存。
- 7) mapreduce.map.java.opts: 控制 MapTask 堆内存大小。（如果内存不够，报：java.lang.OutOfMemoryError）
- 8) mapreduce.map.cpu.vcores 默认 MapTask 的 CPU 核数 1。计算密集型任务可以增加 CPU 核数
- 9) 异常重试
mapreduce.map.maxattempts 每个 Map Task 最大重试次数，一旦重试次数超过该值，则认为 Map Task 运行失败，默认值：4。根据机器性能适当提高。

MapReduce优化 (下)

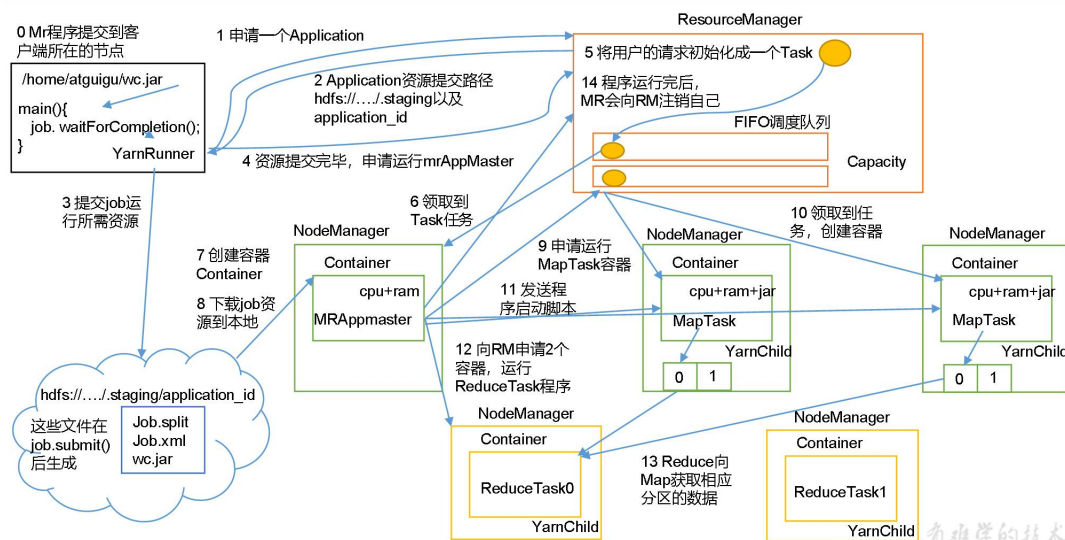


- 1) `mapreduce.reduce.shuffle.parallelcopies` 每个Reduce去Map中拉取数据的并行数, 默认值是5。可以提高到10。
- 2) `mapreduce.reduce.shuffle.input.buffer.percent` Buffer大小占Reduce可用内存的比例, 默认值0.7。可以提高到0.8
- 3) `mapreduce.reduce.shuffle.merge.percent` Buffer中的数据达到多少比例开始写入磁盘, 默认值0.66。可以提高到0.75
- 4) `mapreduce.reduce.memory.mb` 默认ReduceTask内存上限1024MB, 根据128m数据对应1G内存原则, 适当提高内存到4-6G
- 5) `mapreduce.reduce.java.opts`: 控制ReduceTask堆内存大小。(如果内存不够, 报: `java.lang.OutOfMemoryError`)

- 6) `mapreduce.reduce.cpu.vcores` 默认ReduceTask的CPU核数1个。可以提高到2-4个
- 7) `mapreduce.reduce.maxattempts` 每个Reduce Task最大重试次数, 一旦重试次数超过该值, 则认为Map Task运行失败, 默认值: 4。
- 8) `mapreduce.job.reduce.slowstart.completedmaps` 当MapTask完成的比例达到该值后才会为ReduceTask申请资源。默认是0.05。
- 9) `mapreduce.task.timeout` 如果一个Task在一定时间内没有任何进入, 即不会读取新的数据, 也没有输出数据, 则认为该Task处于Block状态, 可能是卡住了, 也许永远会卡住, 为了防止因为用户程序永远Block住不退出, 则强制设置了一个该超时时间 (单位毫秒), 默认是600000 (10分钟)。如果你的程序对每条输入数据的处理时间过长, 建议将该参数调大。
- 10) 如果可以不用Reduce, 尽可能不用

1.2.6 Yarn 工作机制

YARN工作机制



YARN生产环境核心参数

1) Resource Manager相关

yarn.resourcemanager.scheduler.class

配置调度器，默认容量

yarn.resourcemanager.scheduler.client.thread-count

ResourceManager处理调度器请求的线程数量，默认50

2) NodeManager相关

yarn.nodemanager.resource.detect-hardware-capabilities 是否让yarn自己检测硬件进行配置，默认false

yarn.nodemanager.resource.count-logical-processors-as-cores 是否将虚拟核数当作CPU核数，默认false

yarn.nodemanager.resource.pcores-vcores-multiplier 虚拟核数和物理核数乘数，例如：4核8线程，该参数就应设为2，默认1.0

yarn.nodemanager.resource.memory-mb NodeManager使用内存，默认8G

yarn.nodemanager.resource.system-reserved-memory-mb NodeManager为系统保留多少内存
以上二个参数配置一个即可

yarn.nodemanager.resource.cpu-vcores NodeManager使用CPU核数，默认8个

yarn.nodemanager.pmem-check-enabled 是否开启物理内存检查限制container，默认打开

yarn.nodemanager.vmem-check-enabled 是否开启虚拟内存检查限制container，默认打开

yarn.nodemanager.vmem-pmem-ratio 虚拟内存物理内存比例，默认2.1

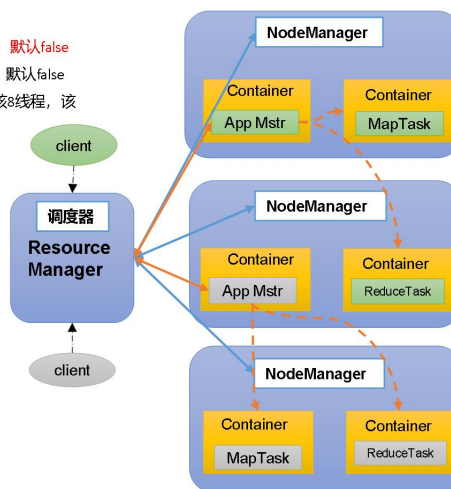
3) Container相关

yarn.scheduler.minimum-allocation-mb 容器最小内存，默认1G

yarn.scheduler.maximum-allocation-mb 容器最大内存，默认8G

yarn.scheduler.minimum-allocation-vcores 容器最小CPU核数，默认1个

yarn.scheduler.maximum-allocation-vcores 容器最大CPU核数，默认4个



1.2.7 Yarn 调度器

1) Hadoop 调度器重要分为三类

FIFO、Capacity Scheduler（容量调度器）和 Fair Sceduler（公平调度器）。

Apache 默认的资源调度器是容量调度器。

CDH 默认的资源调度器是公平调度器。

2) 区别

FIFO 调度器：支持单队列、先进先出 生产环境不会用。

容量调度器：支持多队列。队列资源分配，优先选择资源占用率最低的队列分配资源；作业资源分配，按照作业的优先级和提交时间顺序分配资源；容器资源分配，本地原则（同一节点/同一机架/不同节点不同机架）。

公平调度器：支持多队列，保证每个任务公平享有队列资源。资源不够时可以按照缺额分配。

3) 在生产环境下怎么选择？

大厂：如果对并发度要求比较高，选择公平，要求服务器性能必须 OK。

中小公司，集群服务器资源不太充裕选择容量。

4) 在生产环境怎么创建队列？

（1）调度器默认就 1 个 default 队列，不能满足生产要求。

（2）按照部门：业务部门 1、业务部门 2。

(3) 按照业务模块：登录注册、购物车、下单。

5) 创建多队列的好处？

(1) 因为担心员工不小心，写递归死循环代码，把所有资源全部耗尽。

(2) 实现任务的**降级**使用，特殊时期保证重要的任务队列资源充足。

业务部门 1（重要）=》业务部门 2（比较重要）=》下单（一般）=》购物车（一般）=》
登录注册（次要）

1.2.8 HDFS 块大小

1) 块大小

1.x	64m
2.x 3.x	128m
本地	32m
企业	128m 256m 512m

2) 块大小决定因素

磁盘读写速度

普通的机械硬盘 100m/s => 128m

固态硬盘普通的 300m/s => 256m

内存镜像 500-600m/s => 512m

1.2.9 Hadoop 脑裂原因及解决办法？

1) 出现脑裂的原因

Leader 出现故障，系统开始改朝换代，当 Follower 完成全部工作并且成为 Leader 后，原 Leader 又复活了（它的故障可能是暂时断开或系统暂时变慢，不能及时响应，但其 NameNode 进程还在），并且由于某种原因它对应的 ZKFC 并没有把它设置为 Standby，所以原 Leader 还认为自己是 Leader，客户端向它发出的请求仍会响应，于是脑裂就发生了。

2) Hadoop 通常不会出现脑裂。

如果出现脑裂，意味着多个 Namenode 数据不一致，此时只能选择保留其中一个的数据。
例如：现在有三台 Namenode，分别为 nn1、nn2、nn3，出现脑裂，想要保留 nn1 的数据，步骤为：

(1) 关闭 nn2 和 nn3

(2) 在 nn2 和 nn3 节点重新执行数据同步命令：hdfs namenode -bootstrapStandby

(3) 重新启动 nn2 和 nn3

1.3 Zookeeper

1.3.1 常用命令

ls、get、create、delete、deleteall

1.3.2 选举机制

半数机制（过半机制）： $2n + 1$ ，安装奇数台。

10 台服务器：3 台。

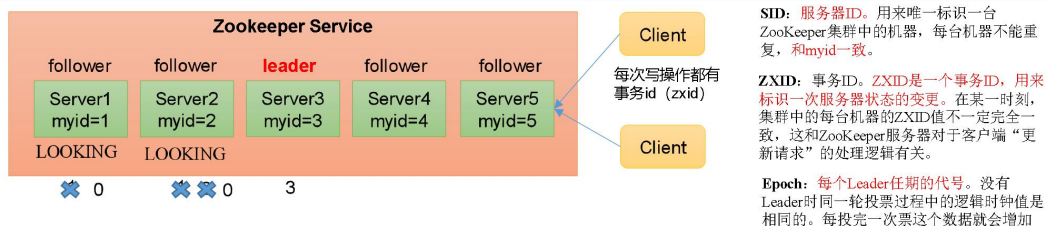
20 台服务器：5 台。

100 台服务器：11 台。

台数多，好处：提高可靠性；坏处：影响通信延时。



Zookeeper选举机制——第一次启动



(1) 服务器1启动，发起一次选举。服务器1投自己一票。此时服务器1票数一票，不够半数以上（3票），选举无法完成，服务器1状态保持为 LOOKING；

(2) 服务器2启动，再发起一次选举。服务器1和2分别投自己一票并交换选票信息；此时服务器1发现服务器2的myid比自己目前投票推荐的（服务器1）大，更改选票为推荐服务器2。此时服务器1票数0票，服务器2票数2票，没有半数以上结果，选举无法完成，服务器1、2状态保持LOOKING

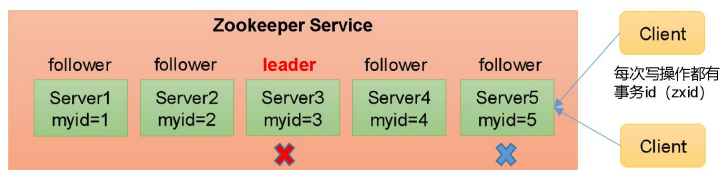
(3) 服务器3启动，发起一次选举。此时服务器1和2都会更改选票为服务器3。此次投票结果：服务器1为0票，服务器2为0票，服务器3为3票。此时服务器3的票数已经超过半数，服务器3当选Leader。服务器1、2更改状态为FOLLOWING，服务器3更改状态为LEADING；

(4) 服务器4启动，发起一次选举。此时服务器1、2、3已经不是LOOKING状态，不会更改选票信息。交换选票信息结果：服务器3为3票，服务器4为1票。此时服务器4服从多数，更改选票信息为服务器3，并更改状态为FOLLOWING；

(5) 服务器5启动，同4一样当小弟。

让天下没有难学的技术

Zookeeper选举机制——非第一次启动



SID: 服务器ID。用来唯一标识一台 ZooKeeper 集群中的机器，每台机器不能重复，和 myid 一致。

ZXID: 事务ID。ZXID 是一个事务ID，用来标识一次服务器状态的变更。在某一时刻，集群中的每台机器的 ZXID 值不一定完全一致，这和 ZooKeeper 服务器对于客户端“更新请求”的处理逻辑有关。

Epoch: 每个 Leader 任期的代号。没有 Leader 时同一轮投票过程中的逻辑时钟值是相同的。每投完一次票这个数据就会增加

(1) 当 ZooKeeper 集群中的一台服务器出现以下两种情况之一时，就会开始进入 Leader 选举：

- 服务器初始化启动。
- 服务器运行期间无法和 Leader 保持连接。

(2) 而当一台机器进入 Leader 选举流程时，当前集群也可能会处于以下两种状态：

- 集群中本来就已经存在一个 Leader。
对于第一种已经存在 Leader 的情况，机器试图去选举 Leader 时，会被告知当前服务器的 Leader 信息，对于该机器来说，仅仅需要和 Leader 机器建立连接，并进行状态同步即可。
- 集群中确实不存在 Leader。

假设 ZooKeeper 由 5 台服务器组成，SID 分别为 1、2、3、4、5，ZXID 分别为 8、8、8、7、7，并且此时 SID 为 3 的服务器是 Leader。某一时刻，3 和 5 服务器出现故障，因此开始进行 Leader 选举。

	(EPOCH, ZXID, SID)	(EPOCH, ZXID, SID)	(EPOCH, ZXID, SID)
SID 为 1、2、4 的机器投票情况：	(1, 8, 1)	(1, 8, 2)	(1, 7, 4)

选举 Leader 规则： ① EPOCH 大的直接胜出 ② EPOCH 相同，事务 id 大的胜出 ③ 事务 id 相同，服务器 id 大的胜出

1.3.3 Zookeeper 符合法则中哪两个？

CAP 理论

CAP 理论告诉我们，一个分布式系统不可能同时满足以下三种

- 一致性 (C:Consistency)
- 可用性 (A:Available)
- 分区容错性 (P:Partition Tolerance)

这三个基本需求，最多只能同时满足其中的两项，因为 P 是必须的，因此往往选择就在 CP 或者 AP 中。

1) 一致性 (C:Consistency)

在分布式环境中，一致性是指数据在多个副本之间是否能够保持数据一致的特性。在一致性的需求下，当一个系统在数据一致的状态下执行更新操作后，应该保证系统的数据仍然处于一致的状态。

2) 可用性 (A:Available)

可用性是指系统提供的服务必须一直处于可用的状态，对于用户的每一个操作请求总是能够在有限的时间内返回结果。

3) 分区容错性 (P:Partition Tolerance)

分布式系统在遇到任何网络分区故障的时候，仍然需要能够保证对外提供满足一致性和可用性的服务，除非是整个网络环境都发生了故障。

ZooKeeper 保证的是 CP

(1) ZooKeeper 不能保证每次服务请求的可用性。(注：在极端环境下，ZooKeeper 可能会丢弃一些请求，消费者程序需要重新请求才能获得结果)。所以说，ZooKeeper 不能保证服务可用性。

(2) 进行 Leader 选举时集群都是不可用。

让天下没有难学的技术

1.3.4 Zookeeper 脑裂

Zookeeper 采用过半选举机制，防止了脑裂。

1.3.5 Zookeeper 用来干嘛了

(1) 作为 HA 的协调者：如 HDFS 的 HA、YARN 的 HA。

(2) 被组件依赖：如 Kafka、HBase、CK。

1.4 Flume

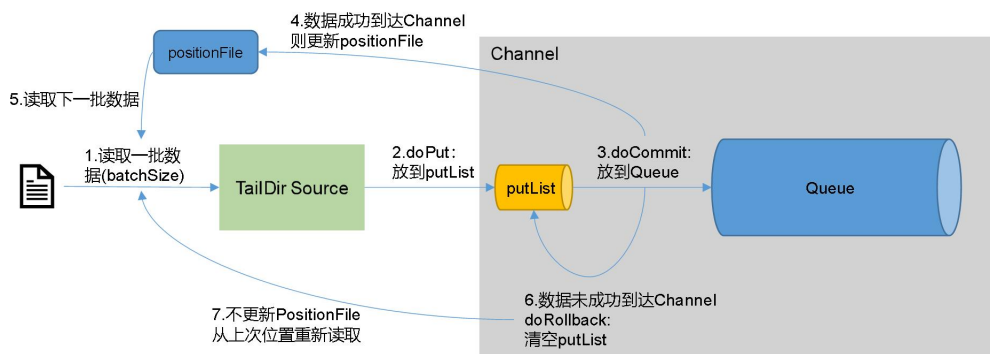
1.4.1 Flume 组成，Put 事务，Take 事务

1) Taildir Source

- (1) 断点续传、多目录
- (2) taildir 底层原理



Flume TailDirSource



让天下没有难学的技术

(3) Taildir 挂了怎么办？

不会丢数：断点续传

重复数据：有可能

(4) 存在的问题及解决方案

①问题：

新文件判断条件 = iNode 值 + 绝对路径（包含文件名）

日志框架凌晨修改了文件名称=》导致会再次重读一次昨天产生的数据

②解决：

方案一：建议生成的文件名称为带日期的。同时配置日志生成框架为不更名的；

方案二：修改 TairDirSource 源码，只按照 iNode 值去确定文件

修改源码视频地址：

https://www.bilibili.com/video/BV1wf4y1G7EQ?p=14&vd_source=891aa1a363111d4914eb12ace2e039af

1d4914eb12ace2e039af

2) file channel /memory channel/kafka channel

(1) File Channel

数据存储于磁盘，优势：可靠性高；劣势：传输速度低

默认容量：100 万个 event

注意：FileChannel 可以通过配置 dataDirs 指向多个路径，每个路径对应不同的硬盘，增大 Flume 吞吐量。

(2) Memory Channel

数据存储于内存，优势：传输速度快；劣势：可靠性差

默认容量：100 个 event

(3) Kafka Channel

数据存储于 Kafka，基于磁盘；

优势：可靠性高；

传输速度快 Kafka Channel 大于 Memory Channel + Kafka Sink 原因省去了 Sink 阶段

(4) 生产环境如何选择

- 如果下一级是 Kafka，优先选择 Kafka Channel。
- 如果是金融、对钱要求准确的公司，选择 File Channel。
- 如果就是普通的日志，通常可以选择 Memory Channel。

每天丢几百万数据 pb 级 亿万富翁，掉 1 块钱会捡？

3) HDFS Sink

(1) 时间（半个小时） or 大小 128m 且 设置 Event 个数等于 0，该值默认 10

具体参数：hdfs.rollInterval=1800, hdfs.rollSize=134217728 且 hdfs.rollCount=0

4) 事务

- Source 到 Channel 是 Put 事务
- Channel 到 Sink 是 Take 事务

1.4.2 Flume 拦截器

1) 拦截器注意事项

(1) 时间戳拦截器：主要是解决零点漂移问题

2) 自定义拦截器步骤

(1) 实现 `Interceptor`

(2) 重写四个方法

➤ `initialize` 初始化

➤ `public Event intercept(Event event)` 处理单个 `Event`

➤ `public List<Event> intercept(List<Event> events)` 处理多个 `Event`，在这个方法中调用 `Event intercept(Event event)`

➤ `close` 方法

(3) 静态内部类，实现 `Interceptor.Builder`

3) 拦截器可以不用吗？

时间戳拦截器建议使用。如果不用需要采用延迟 15-20 分钟处理数据的方式，比较麻烦。

1.4.3 Flume Channel 选择器

`Replicating`：默认选择器。功能：将数据发往下一级所有通道。

`Multiplexing`：选择性发往指定通道。

1.4.4 Flume 监控器

1) 监控到异常现象

采用 `Ganglia` 监控器，监控到 `Flume` 尝试提交的次数远远大于最终成功的次数，说明 `Flume` 运行比较差。主要是内存不够导致的。

2) 解决办法？

(1) 自身：默认内存是 20m，考虑增加 `flume` 内存，在 `flume-env.sh` 配置文件中修改 `flume` 内存为 4-6g

(2) 找朋友：增加服务器台数

搞活动 618 => 增加服务器 => 用完在退出

日志服务器配置：8-16g 内存、磁盘 8T

1.4.5 Flume 采集数据会丢失吗？

- 如果是 `kafka channel` 或者 `FileChannel` 不会丢失数据，数据存储可以存储在磁盘中。

- 如果是 MemoryChannel 有可能丢。

1.4.6 Flume 如何提高吞吐量

- 调整 taildir source 的 batchSize 大小可以控制吞吐量，默认大小 100 个 Event。
- 吞吐量的瓶颈一般是网络带宽。

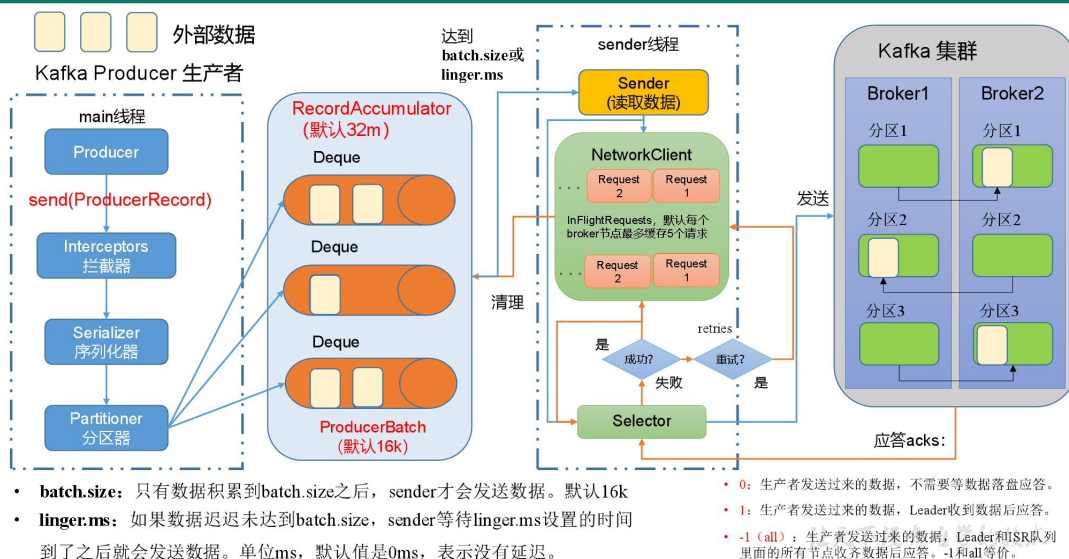
1.5 Kafka

1.5.1 Kafka 架构

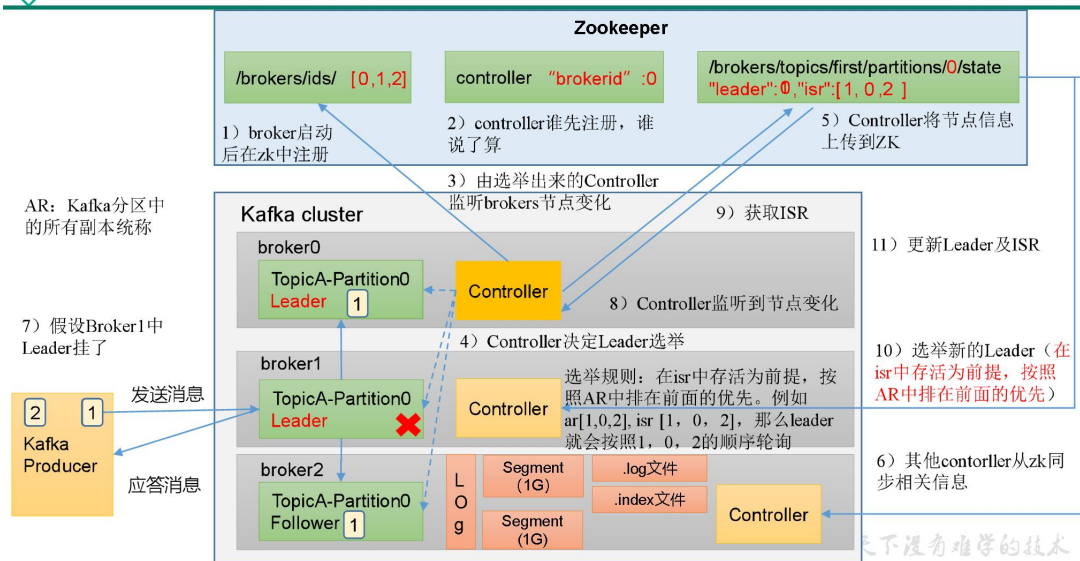
生产者、Broker、消费者、Zookeeper。

注意：Zookeeper 中保存 Broker id 和 controller 等信息，但是没有生产者信息。

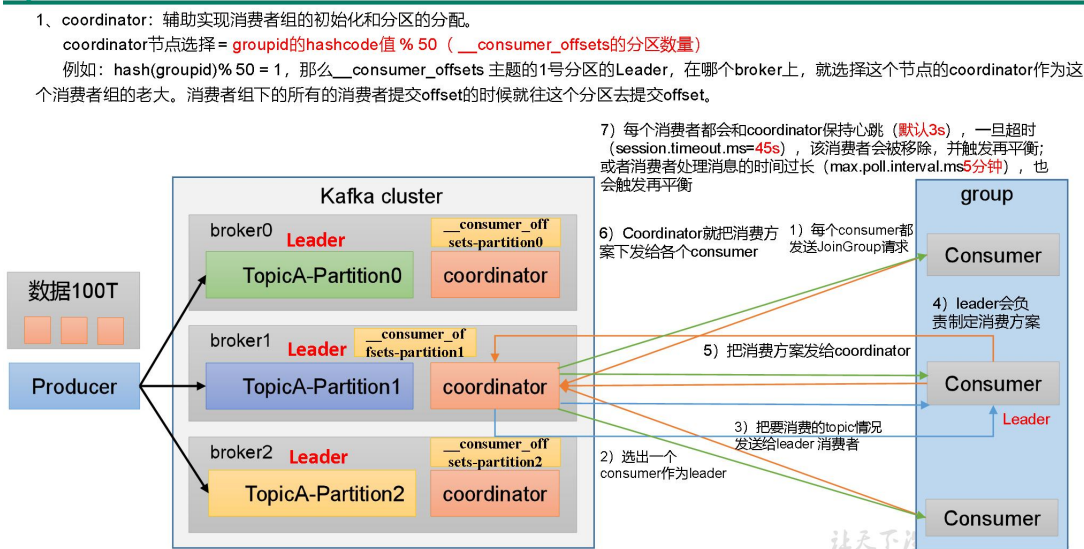
发送流程



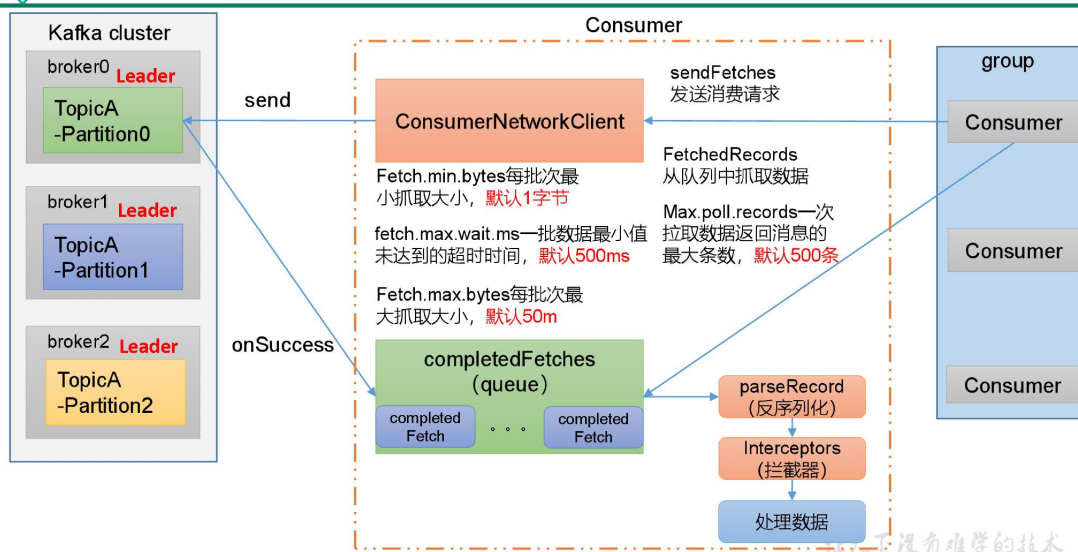
Kafka Broker总体工作流程



消费者组初始化流程

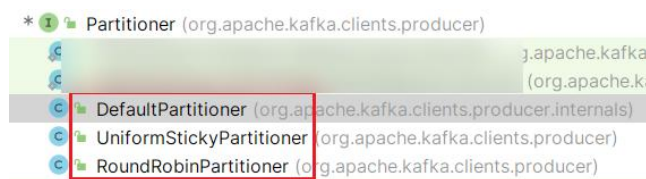


消费者组详细消费流程



1.5.2 Kafka 生产端分区分配策略

Kafka 官方为我们实现了三种 Partitioner (分区器), 分别是 DefaultPartitioner (当未指定分区器时候所使用的默认分区器)、UniformStickyPartitioner、RoundRobinPartitioner。



1) DefaultPartitioner 默认分区器

下图说明了默认分区器的分区分配策略:

Kafka 原则

在IDEA中全局查找 (ctrl + n) **ProducerRecord**类, 在类中可以看到如下构造方法:

```
public ProducerRecord(String topic, Integer partition, Long timestamp, K key, V value, Iterable<Header> headers) {
    ...
}
public ProducerRecord(String topic, Integer partition, Long timestamp, K key, V value) {
    ...
}
public ProducerRecord(String topic, Integer partition, K key, V value, Iterable<Header> headers) {
    ...
}
public ProducerRecord(String topic, Integer partition, K key, V value) {
    ...
}
public ProducerRecord(String topic, K key, V value) {
    ...
}
public ProducerRecord(String topic, V value) {
    ...
}
```

(1) 指明partition的情况下, 直接将指明的值作为partition值, 例如partition=0, 所有数据写入分区0

(2) 没有指明partition值但有key的情况下, 将key的hash值与topic的partition数进行取余得到partition值;
例如: key1的hash值=5, key2的hash值=6, topic的partition数=2, 那么key1对应的value1写入1号分区, key2对应的value2写入0号分区。

(3) 既没有partition值又没有key值的情况下, Kafka采用Sticky Partition (黏性分区器), 会随机选择一个分区, 并尽可能一直使用该分区, 待该分区的batch已满或者已完成, Kafka再随机一个分区进行使用 (和上一次的分区分不同)。

例如: 第一次随机选择0号分区, 等0号分区当前批次满了 (默认16k) 或者linger.ms设置的时间到, Kafka再随机一个分区进行使用。此时分以下三种情况: ①可用分区<1; 那么选择分区的逻辑是在所有分区中随机选择。②可用分区=1; 那么直接选择这个分区。③可用分区>1; 那么在所有可用分区中随机选择。

让天下没有难学的技术

2) UniformStickyPartitioner 纯粹的粘性分区器

- (1) 如果指定了分区号，则会按照指定的分区号进行分配
- (2) 若没有指定分区号，则使用粘性分区器

3) RoundRobinPartitioner 轮询分区器

- (1) 如果在消息中指定了分区则使用指定分区。
- (2) 如果未指定分区，都会将消息轮询每个分区，将数据平均分配到每个分区中。

4) 自定义分区器

自定义分区策略：可以通过实现 `org.apache.kafka.clients.producer.Partitioner` 接口，重写 `partition` 方法来达到自定义分区效果。

例如我们想要实现随机分配，只需要以下代码：

```
List<PartitionInfo> partitions = cluster.partitionsForTopic(topic);
return ThreadLocalRandom.current().nextInt(partitions.size());
```

先计算出该主题总的分区数，然后随机地返回一个小于它的正整数。

在项目中，如果希望把 MySQL 中某张表的数据发送到一个分区。可以以表名为 key 进行发送。

1.5.3 Kafka 丢不丢数据

1) Producer 角度

`acks=0`，生产者发送过来数据就不管了，可靠性差，效率高；

`acks=1`，生产者发送过来数据 Leader 应答，可靠性中等，效率中等；

`acks=-1`，生产者发送过来数据 Leader 和 ISR 队列里面所有 Follower 应答，可靠性高，效率低；

在生产环境中，`acks=0` 很少使用；`acks=1`，一般用于传输普通日志，允许丢个别数据；`acks=-1`，一般用于传输和钱相关的数据，对可靠性要求比较高的场景。

2) Broker 角度

副本数大于等于 2。

`min.insync.replicas` 大于等于 2。

1.5.4 Kafka 的 ISR 副本同步队列

ISR (In-Sync Replicas)，副本同步队列。如果 Follower 长时间未向 Leader 发送通信请求或同步数据，则该 Follower 将被踢出 ISR。该时间阈值由 `replica.lag.time.max.ms` 参数设

定，默认 30s。

任意一个维度超过阈值都会把 Follower 剔除出 ISR，存入 OSR（Outof-Sync Replicas）列表，新加入的 Follower 也会先存放在 OSR 中。

Kafka 分区中的所有副本统称为 AR = ISR + OSR

1.5.5 Kafka 数据重复

去重 = 幂等性 + 事务

1) 幂等性原理



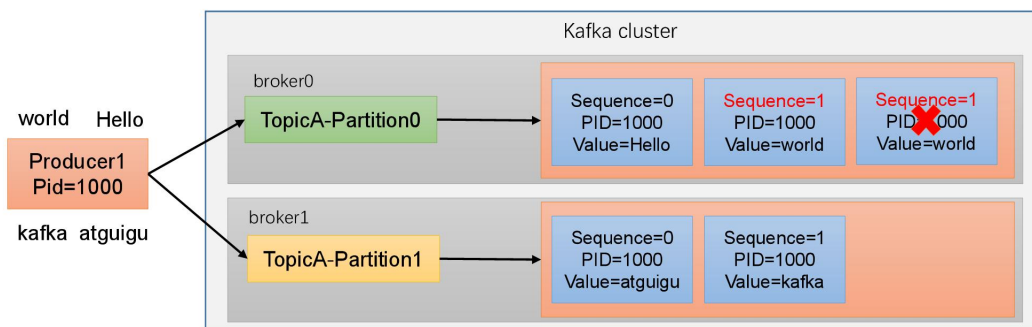
幂等性原理



幂等性就是指Producer不论向Broker发送多少次重复数据，Broker端都只会持久化一条，保证了不重复。

重复数据的判断标准：具有<PID, Partition, SeqNumber>相同主键的消息提交时，Broker只会持久化一条。其中PID是Kafka每次重启都会分配一个新的；Partition 表示分区号；Sequence Number是单调自增的。

所以幂等性只能保证的是在单分区单会话内不重复。



2) 幂等性配置参数

参数名称	描述
enable.idempotence	是否开启幂等性，默认 true，表示开启幂等性。
max.in.flight.requests.per.connection	1.0.X 版本前，需设置为 1，1.0.X 之后，小于等于 5
retries	失败重试次数，需要大于 0
acks	需要设置为 all

3) Kafka 的事务一共有如下 5 个 API

```
// 1 初始化事务
void initTransactions();

// 2 开启事务
void beginTransaction() throws ProducerFencedException;

// 3 在事务内提交已经消费的偏移量（主要用于消费者）
void sendOffsetsToTransaction(Map<TopicPartition, OffsetAndMetadata> offsets,
                               String consumerGroupId) throws
```

```

ProducerFencedException;

// 4 提交事务
void commitTransaction() throws ProducerFencedException;

// 5 放弃事务（类似于回滚事务的操作）
void abortTransaction() throws ProducerFencedException;

```

4) 总结

(1) 生产者角度

- acks 设置为-1 (acks=-1)。
- 幂等性 (enable.idempotence = true) + 事务。

(2) broker 服务端角度

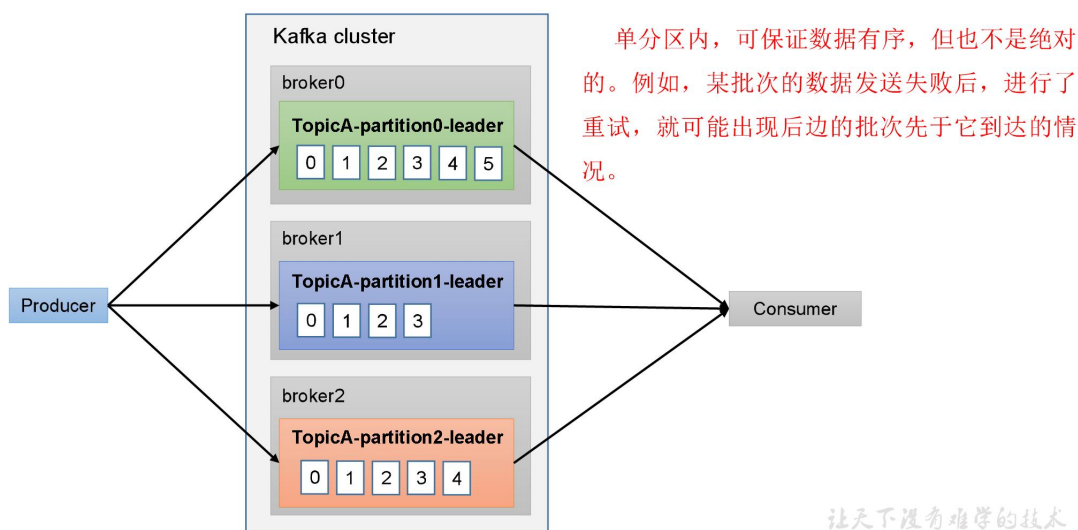
- 分区副本大于等于 2 (--replication-factor 2)。
- ISR 里应答的最小副本数量大于等于 2 (min.insync.replicas = 2)。

(3) 消费者

- 事务 + 手动提交 offset (enable.auto.commit = false)。
- 消费者输出的目的地必须支持事务 (MySQL、Kafka)。

1.5.6 Kafka 如何保证数据有序 or 怎么解决乱序

1) Kafka 最多只保证单分区内的消息是有序的，所以如果要保证业务全局严格有序，就要设置 Topic 为单分区。



2) 如何保证单分区内数据有序?

方案一：

禁止重试，需设置以下参数

设置retries等于0

说明：数据出现乱序的根本原因是，失败重试，关闭重试，则可保证数据是有序的。但是这样做，可能会导致数据的丢失。

方案二：

启用幂等，需设置以下参数

设置enable.idempotence=true，启用幂等

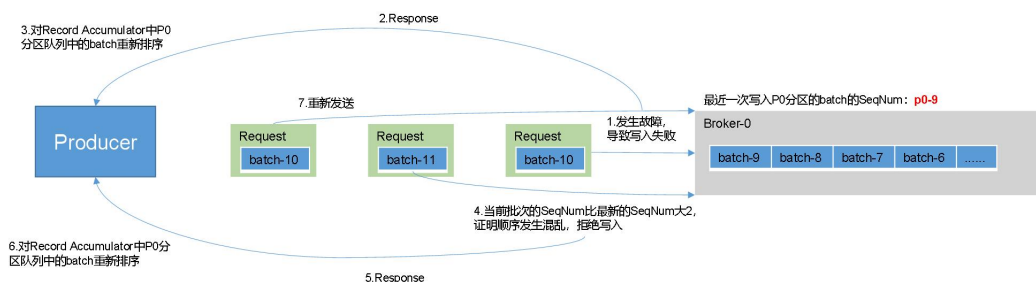
设置max.in.flight.requests.per.connection，1.0.X之后，小于等于5

设置retries，保证其大于0

设置acks，保证其为-1

让天下没有难学的技术

注：幂等机制保证数据有序的原理如下：



让天下没有难学的技术

1.5.7 Kafka 分区 Leader 选举规则

在 ISR 中存活为前提，按照 AR 中排在前面的优先。例如 AR[1,0,2]，ISR [1, 0, 2]，那么 Leader 就会按照 1, 0, 2 的顺序轮询。

1.5.8 Kafka 中 AR 的顺序

如果 Kafka 服务器只有 4 个节点，那么设置 Kafka 的分区数大于服务器台数，在 Kafka 底层如何分配存储副本呢？

1) 创建 16 分区，3 个副本

(1) 创建一个新的 Topic，名称为 second。

```
[atguigu@hadoop102 kafka]$ bin/kafka-topics.sh --bootstrap-server
hadoop102:9092 --create --partitions 16 --replication-factor 3
--topic second
```

(2) 查看分区和副本情况。

```
[atguigu@hadoop102 kafka]$ bin/kafka-topics.sh --bootstrap-server
hadoop102:9092 --describe --topic second
```

```
Topic: second4 Partition: 0 Leader: 0 Replicas: 0,1,2 Isr: 0,1,2
Topic: second4 Partition: 1 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3
Topic: second4 Partition: 2 Leader: 2 Replicas: 2,3,0 Isr: 2,3,0
Topic: second4 Partition: 3 Leader: 3 Replicas: 3,0,1 Isr: 3,0,1

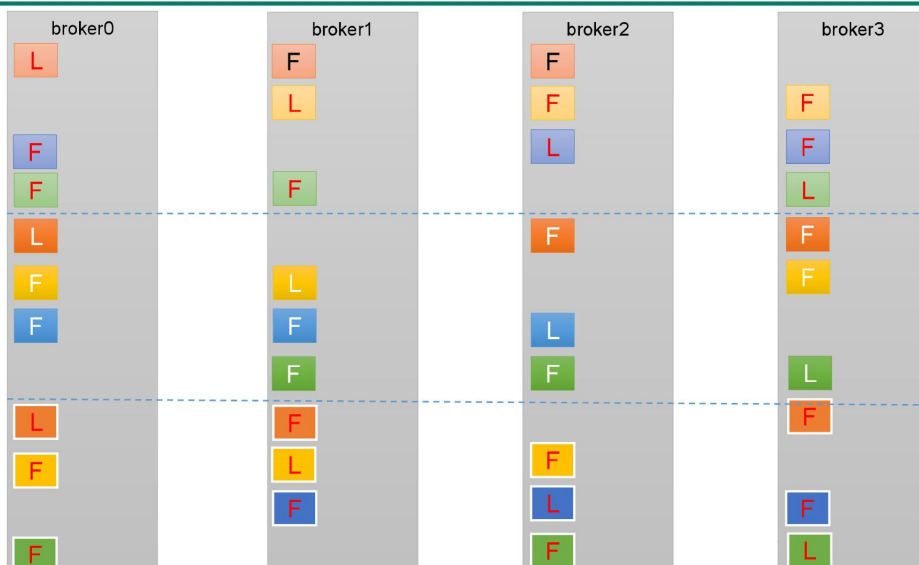
Topic: second4 Partition: 4 Leader: 0 Replicas: 0,2,3 Isr: 0,2,3
Topic: second4 Partition: 5 Leader: 1 Replicas: 1,3,0 Isr: 1,3,0
Topic: second4 Partition: 6 Leader: 2 Replicas: 2,0,1 Isr: 2,0,1
Topic: second4 Partition: 7 Leader: 3 Replicas: 3,1,2 Isr: 3,1,2

Topic: second4 Partition: 8 Leader: 0 Replicas: 0,3,1 Isr: 0,3,1
Topic: second4 Partition: 9 Leader: 1 Replicas: 1,0,2 Isr: 1,0,2
Topic: second4 Partition: 10 Leader: 2 Replicas: 2,1,3 Isr: 2,1,3
Topic: second4 Partition: 11 Leader: 3 Replicas: 3,2,0 Isr: 3,2,0

Topic: second4 Partition: 12 Leader: 0 Replicas: 0,1,2 Isr: 0,1,2
Topic: second4 Partition: 13 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3
Topic: second4 Partition: 14 Leader: 2 Replicas: 2,3,0 Isr: 2,3,0
Topic: second4 Partition: 15 Leader: 3 Replicas: 3,0,1 Isr: 3,0,1
```



分区副本分配



1.5.9 Kafka 日志保存时间

默认保存 7 天；生产环境建议 3 天。

1.5.10 Kafka 过期数据清理

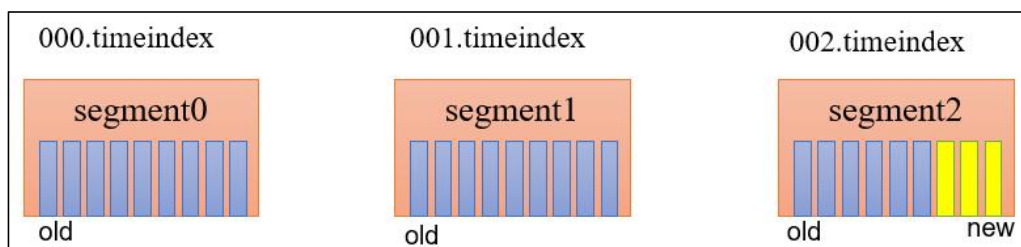
日志清理的策略只有 **delete** 和 **compact** 两种。

1) delete 日志删除：将过期数据删除

- `log.cleanup.policy = delete`，所有数据启用删除策略
 - (1) **基于时间**：默认打开。以 **segment** 中所有记录中的最大时间戳作为该文件时间戳。
 - (2) **基于大小**：默认关闭。超过设置的所有日志总大小，删除最早的 **segment**。

`log.retention.bytes`，默认等于-1，表示无穷大。

思考：如果一个 **segment** 中有一部分数据过期，一部分没有过期，怎么处理？



2) compact 日志压缩



compact 日志压缩



compact 日志压缩：对于相同 **key** 的不同 **value** 值，只保留最后一个版本。

- `log.cleanup.policy = compact` 所有数据启用压缩策略

Offset	0	1	2	3	4	5	6	7	8
key	K1	K2	K1	K1	K3	K4	K5	K5	K2
value	V1	V2	V3	V4	V5	V6	V7	V8	V9

压缩之前的数据

Offset	3	4	5	7	8
keys	k1	K3	K4	K5	K2
values	V4	V5	V6	V8	V9

压缩之后的数据

压缩后的 **offset** 可能是不连续的，比如上图中没有 6，当从这些 **offset** 消费消息时，将会拿到比这个 **offset** 大的 **offset** 对应的消息，实际上会拿到 **offset** 为 7 的消息，并从这个位置开始消费。

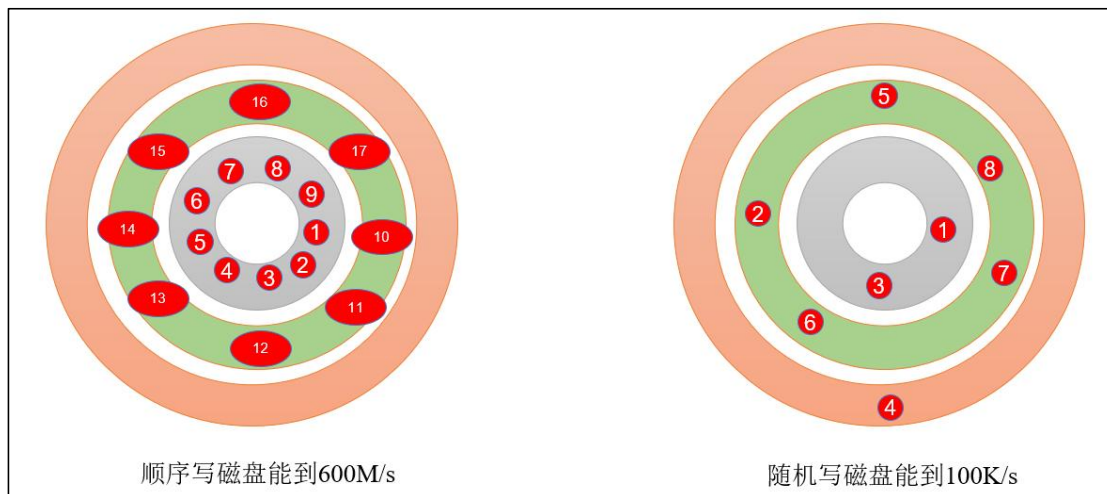
这种策略只适合特殊场景，比如消息的 **key** 是用户 ID，**value** 是用户的资料，通过这种压缩策略，整个消息集里就保存了所有用户最新的资料。

让天下没有难学的技术

1.5.11 Kafka 为什么能高效读写数据

- 1) **Kafka** 本身是分布式集群，可以采用分区技术，并行度高
- 2) 读数据采用稀疏索引，可以快速定位要消费的数据
- 3) 顺序写磁盘

Kafka 的 producer 生产数据，要写入到 log 文件中，写的过程是一直追加到文件末端，为顺序写。官网有数据表明，同样的磁盘，顺序写能到 600M/s，而随机写只有 100K/s。这与磁盘的机械机构有关，顺序写之所以快，是因为其省去了大量磁头寻址的时间。



4) 页缓存 + 零拷贝技术

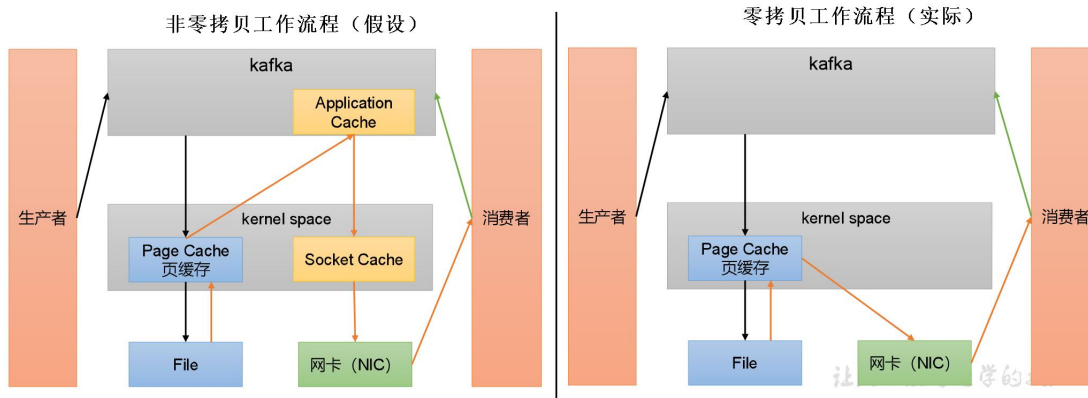


页缓存 + 零拷贝技术



零拷贝：Kafka 的数据加工处理操作交由 Kafka 生产者和 Kafka 消费者处理。Kafka Broker 应用层不关心存储的数据，所以就不用走应用层，传输效率高。

PageCache 页缓存：Kafka 重度依赖底层操作系统提供的 PageCache 功能。当上层有写操作时，操作系统只是将数据写入 PageCache。当读操作发生时，先从 PageCache 中查找，如果找不到，再去磁盘中读取。实际上 PageCache 是把尽可能多的空闲内存都当做了磁盘缓存来使用。



1.5.12 自动创建主题

如果 Broker 端配置参数 `auto.create.topics.enable` 设置为 `true`（默认值是 `true`），那么当生产者向一个未创建的主题发送消息时，会自动创建一个分区数为 `num.partitions`（默认值为 1）、副本因子为 `default.replication.factor`（默认值为 1）的主题。除此之外，当一个消费者开始从未知主题中读取消息时，或者当任意一个客户端向未知主题发送元数据请求时，都会自动创建一个相应主题。这种创建主题的方式是非预期的，增加了主题管理和维护的难度。生产环

境建议将该参数设置为 `false`。

(1) 向一个没有提前创建 `five` 主题发送数据

```
[atguigu@hadoop102 kafka]$ bin/kafka-console-producer.sh
--bootstrap-server hadoop102:9092 --topic five
>hello world
```

(2) 查看 `five` 主题的详情

```
[atguigu@hadoop102 kafka]$ bin/kafka-topics.sh --bootstrap-server
hadoop102:9092 --describe --topic five
```

1.5.13 副本数设定

一般我们设置成 2 个或 3 个，很多企业设置为 2 个。

副本的优势：提高可靠性；副本劣势：增加了网络 IO 传输。

1.5.14 Kafka 分区数

(1) 创建一个只有 1 个分区的 Topic。

(2) 测试这个 Topic 的 Producer 吞吐量和 Consumer 吞吐量。

(3) 假设他们的值分别是 T_p 和 T_c ，单位可以是 MB/s。

(4) 然后假设总的目标吞吐量是 T_t ，那么分区数 $= T_t / \min(T_p, T_c)$ 。

例如：Producer 吞吐量 = 20m/s；Consumer 吞吐量 = 50m/s，期望吞吐量 100m/s；

分区数 $= 100 / 20 = 5$ 分区

分区数一般设置为：3-10 个

分区数不是越多越好，也不是越少越好，需要搭建完集群，进行压测，再灵活调整分区个数。

1.5.15 Kafka 增加分区

1) 可以通过命令行的方式增加分区，但是分区数只能增加，不能减少。

2) 为什么分区数只能增加，不能减少？

(1) 按照 Kafka 现有的代码逻辑而言，此功能完全可以实现，不过也会使得代码的复杂度急剧增大。

(2) 实现此功能需要考虑的因素很多，比如删除掉的分区中的消息该作何处理？

- 如果随着分区一起消失则消息的可靠性得不到保障；
- 如果需要保留则又需要考虑如何保留，直接存储到现有分区的尾部，消息的时间戳就不会递增，如此对于 Spark、Flink 这类需要消息时间戳（事件时间）的组件将会受到影响；

● 如果分散插入到现有的分区中，那么在消息量很大的时候，内部的数据复制会占用很大的资源，而且在复制期间，此主题的可用性又如何得到保障？

● 同时，顺序性问题、事务性问题、以及分区和副本的状态机切换问题都是不得不面对的。

(3) 反观这个功能的收益点却是很低，如果真的需要实现此类功能，完全可以重新创建一个分区数较小的主题，然后将现有主题中的消息按照既定的逻辑复制过去即可。

1.5.16 Kafka 中多少个 Topic

ODS 层：2 个

DWD 层：20 个

1.5.17 Kafka 消费者是拉取数据还是推送数据

拉取数据。

1.5.18 Kafka 消费端分区分配策略

分区分配策略之Range



Range 是对每个 topic 而言的。

首先对同一个 topic 里面的分区按照序号进行排序，并对消费者按照字母顺序进行排序。

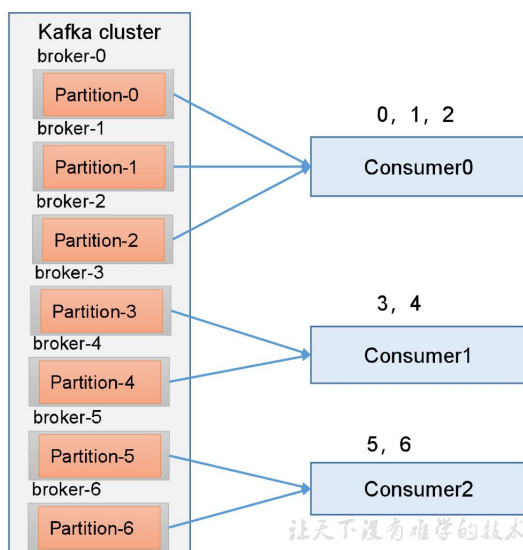
假如现在有 7 个分区，3 个消费者，排序后的分区将会是 0,1,2,3,4,5,6；消费者排序完之后将会是 C0,C1,C2。

通过 $\text{partitions数} / \text{consumer数}$ 来决定每个消费者应该消费几个分区。如果除不尽，那么前面几个消费者将会多消费 1 个分区。

例如， $7/3 = 2 \text{ 余 } 1$ ，除不尽，那么消费者 C0 便会多消费 1 个分区。 $8/3 = 2 \text{ 余 } 2$ ，除不尽，那么 C0 和 C1 分别多消费一个。

注意：如果只是针对 1 个 topic 而言，C0 消费者多消费 1 个分区影响不是很大。但是如果有 N 多个 topic，那么针对每个 topic，消费者 C0 都将多消费 1 个分区，topic 越多，C0 消费的分区的会比其他消费者明显多消费 N 个分区。

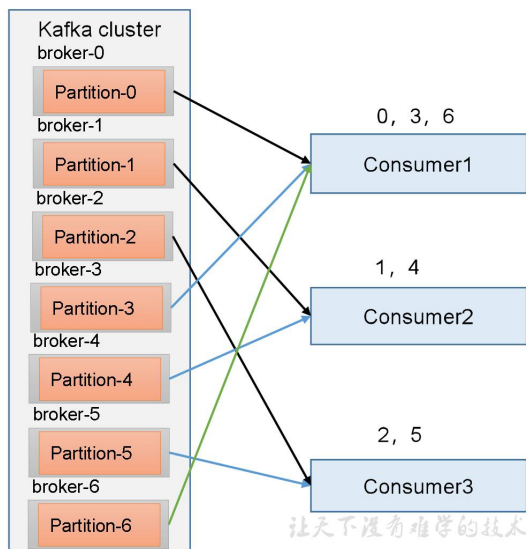
容易产生数据倾斜！



分区分配策略之RoundRobin

RoundRobin 针对集群中所有Topic而言。

RoundRobin 轮询分区策略，是把所有的 partition 和所有的 consumer 都列出来，然后按照 hashcode 进行排序，最后通过轮询算法来分配 partition 给到各个消费者。



粘性分区：

该分区分配算法是最复杂的一种，可以通过 `partition.assignment.strategy` 参数去设置，从 0.11 版本开始引入，目的就是在执行新分配时，尽量在上一次分配结果上少做调整，其主要实现了以下 2 个目标：

- (1) Topic Partition 的分配要尽量均衡。
- (2) 当 Rebalance 发生时，尽量与上一次分配结果保持一致。

注意：当两个目标发生冲突的时候，优先保证第一个目标，这样可以使分配更加均匀，其中第一个目标是 3 种分配策略都尽量去尝试完成的，而第二个目标才是该算法的精髓所在。

1.5.19 消费者再平衡的条件

1) Rebalance 的触发条件有三种

- (1) 当 Consumer Group 组成员数量发生变化（主动加入、主动离组或者故障下线等）。
- (2) 当订阅主题的数量或者分区发生变化。

1.5.22 Kafka 监控

公司自己开发的监控器。

开源的监控器：KafkaManager、KafkaMonitor、KafkaEagle。

1.5.23 Kafka 数据积压

1) 发现数据积压

通过 Kafka 的监控器 Eagle，可以看到消费 lag，就是积压情况：

Consumers Offsets details						
List the current consumers's offsets of topic.						
bigdata02						
Partition	LogSize	Offset	Lag	Owner	Created	Modify
2	4	4	0	--	2022-08-21 15:14:09	2022-08-21 15:14:09
0	2004	1004	1000	--	2022-08-21 15:14:09	2022-08-21 15:14:09
1	2	2	0	--	2022-08-21 15:14:09	2022-08-21 15:14:09
Showing 1 to 3 of 3 entries						
Previous 1 Next						

2) 解决

(1) 消费者消费能力不足

①可以考虑增加 Topic 的分区数，并且同时提升消费组的消费者数量，消费者数 = 分区数。（两者缺一不可）。

增加分区数：

```
[atguigu@hadoop102 kafka]$ bin/kafka-topics.sh --bootstrap-server hadoop102:9092 --alter --topic first --partitions 3
```

②提高每批次拉取的数量，提高单个消费者的消费能力。

参数名称	描述
fetch.max.bytes	默认 Default: 52428800 (50 m) 。消费者获取服务器端一批消息最大的字节数。如果服务器端一批次的数据大于该值（50m）仍然可以拉取回来这批数据，因此，这不是一个绝对最大值。一批次的大小受 message.max.bytes （broker config）or max.message.bytes （topic config）影响。
max.poll.records	一次 poll 拉取数据返回消息的最大条数，默认是 500 条

(2) 消费者处理能力不行

①消费者，调整 `fetch.max.bytes` 大小，默认是 50m。

②消费者，调整 `max.poll.records` 大小，默认是 500 条。

如果下游是 Spark、Flink 等计算引擎，消费到数据之后还要进行计算分析处理，当处理能力跟不上消费能力时，会导致背压的出现，从而使消费的速率下降。

需要对计算性能进行调优（看 Spark、Flink 优化）。

(3) 消息积压后如何处理

某时刻，突然开始积压消息且持续上涨。这种情况下需要你在短时间内找到消息积压的原因，迅速解决问题。

导致消息积压突然增加，只有两种：发送变快了或者消费变慢了。

假如赶上大促或者抢购时，短时间内不太可能优化消费端的代码来提升消费性能，此时唯一的办法是通过扩容消费端的实例数来提升总体的消费能力。如果短时间内没有足够的服务器资源进行扩容，只能降级一些不重要的业务，减少发送方发送的数据量，最低限度让系统还能正常运转，保证重要业务服务正常。

假如通过内部监控到消费变慢了，需要你检查消费实例，分析一下是什么原因导致消费变慢？

①优先查看日志是否有大量的消费错误。

②此时如果没有错误的话，可以通过打印堆栈信息，看一下你的消费线程卡在哪里「触发死锁或者卡在某些等待资源」。

1.5.24 如何提升吞吐量

如何提升吞吐量？

1) 提升生产吞吐量

(1) `buffer.memory`：发送消息的缓冲区大小，默认值是 32m，可以增加至 64m。

(2) `batch.size`：默认是 16k。如果 `batch` 设置太小，会导致频繁网络请求，吞吐量下降；如果 `batch` 太大，会导致一条消息需要等待很久才能被发送出去，增加网络延时。

(3) `linger.ms`，这个值默认是 0，意思就是消息必须立即被发送。一般设置一个 5-100 毫秒。如果 `linger.ms` 设置的太小，会导致频繁网络请求，吞吐量下降；如果 `linger.ms` 太长，会导致一条消息需要等待很久才能被发送出去，增加网络延时。

(4) `compression.type`: 默认是 `none`, 不压缩, 但是也可以使用 `lz4` 压缩, 效率还是不错的, 压缩之后可以减小数据量, 提升吞吐量, 但是会加大 `producer` 端的 CPU 开销。

2) 增加分区

3) 消费者提高吞吐量

(1) 调整 `fetch.max.bytes` 大小, 默认是 50m。

(2) 调整 `max.poll.records` 大小, 默认是 500 条。

1.5.25 Kafka 中数据量计算

每天总数据量 100g, 每天产生 1 亿条日志, $10000 \text{ 万} / 24 / 60 / 60 = 1150 \text{ 条/每秒钟}$

平均每秒钟: 1150 条

低谷每秒钟: 50 条

高峰每秒钟: $1150 \text{ 条} * (2-20 \text{ 倍}) = 2300 \text{ 条} - 23000 \text{ 条}$

每条日志大小: 0.5k - 2k (取 1k)

每秒多少数据量: 2.0M - 20MB

1.5.26 Kafka 如何压测?

用 Kafka 官方自带的脚本, 对 Kafka 进行压测。

- 生产者压测: `kafka-producer-perf-test.sh`
- 消费者压测: `kafka-consumer-perf-test.sh`

1) Kafka Producer 压力测试

(1) 创建一个 `test` Topic, 设置为 3 个分区 3 个副本

```
[atguigu@hadoop102 kafka]$ bin/kafka-topics.sh --bootstrap-server
hadoop102:9092 --create --replication-factor 3 --partitions 3 --topic
test
```

(2) 在 `/opt/module/kafka/bin` 目录下面有这两个文件。我们来测试一下

```
[atguigu@hadoop105 kafka]$ bin/kafka-producer-perf-test.sh --topic
test --record-size 1024 --num-records 1000000 --throughput 10000
--producer-props
bootstrap.servers=hadoop102:9092,hadoop103:9092,hadoop104:9092
batch.size=16384 linger.ms=0
```

参数说明:

- `record-size` 是一条信息有多大, 单位是字节, 本次测试设置为 1k。
- `num-records` 是总共发送多少条信息, 本次测试设置为 100 万条。
- `throughput` 是每秒多少条信息, 设成 -1, 表示不限流, 尽可能快的生产数据, 可测

出生产者最大吞吐量。本次实验设置为每秒钟 1 万条。

- `producer.props` 后面可以配置生产者相关参数，`batch.size` 配置为 16k。

输出结果：

```
ap.servers=hadoop102:9092,hadoop103:9092,hadoop104:9092      batch.size=16384
linger.ms=0
37021 records sent, 7401.2 records/sec (7.23 MB/sec), 1136.0 ms avg latency, 1453.0
ms max latency.
ooo   ooo
33570 records sent, 6714.0 records/sec (6.56 MB/sec), 4549.0 ms avg latency, 5049.0
ms max latency.
1000000 records sent, 9180.713158 records/sec (8.97 MB/sec), 1894.78 ms avg
latency, 5049.00 ms max latency, 1335 ms 50th, 4128 ms 95th, 4719 ms 99th, 5030
ms 99.9th.
```

- (3) 调整 `batch.size` 大小
- (4) 调整 `linger.ms` 时间
- (5) 调整压缩方式
- (6) 调整缓存大小

2) Kafka Consumer 压力测试

- (1) 修改 `/opt/module/kafka/config/consumer.properties` 文件中的一次拉取条数为 500

```
max.poll.records=500
```

- (2) 消费 100 万条日志进行压测

```
[atguigu@hadoop105 kafka]$ bin/kafka-consumer-perf-test.sh
--bootstrap-server  hadoop102:9092,hadoop103:9092,hadoop104:9092
--topic  test      --messages  1000000      --consumer.config
config/consumer.properties
```

参数说明：

- `--bootstrap-server` 指定 Kafka 集群地址
- `--topic` 指定 topic 的名称
- `--messages` 总共要消费的消息个数。本次实验 100 万条。

输出结果：

```
start.time, end.time, data.consumed.in.MB, MB.sec, data.consumed.in.nMsg,
nMsg.sec, rebalance.time.ms, fetch.time.ms, fetch.MB.sec, fetch.nMsg.sec
2022-01-20 09:58:26:171, 2022-01-20 09:58:33:321, 977.0166, 136.6457, 1000465,
139925.1748, 415, 6735, 145.0656, 148547.1418
```

- (3) 一次拉取条数为 2000
- (4) 调整 `fetch.max.bytes` 大小为 100m

1.5.27 磁盘选择

kafka 底层主要是顺序写，固态硬盘和机械硬盘的顺序写速度差不多。

建议选择普通的机械硬盘。

每天总数据量：1 亿条 * 1k $\approx$ 100g

100g * 副本 2 * 保存时间 3 天 / 0.7 $\approx$ 1T

建议三台服务器硬盘总大小，大于等于 1T。

1.5.28 内存选择

Kafka 内存组成：堆内存 + 页缓存

1) Kafka 堆内存建议每个节点：10g ~ 15g

在 kafka-server-start.sh 中修改

```
if [ "x$KAFKA_HEAP_OPTS" = "x" ]; then
    export KAFKA_HEAP_OPTS="-Xmx10G -Xms10G"
fi
```

(1) 查看 Kafka 进程号

```
[atguigu@hadoop102 kafka]$ jps
2321 Kafka
5255 Jps
1931 QuorumPeerMain
```

(2) 根据 Kafka 进程号，查看 Kafka 的 GC 情况

```
[atguigu@hadoop102 kafka]$ jstat -gc 2321 1s 10
```

S0C	S1C	S0U	S1U	EC	EU	OC	OU	MC	MU	CCSC	CCSU	YGC	YGCT	FGC	FGCT	GCT
0.0	7168.0	0.0	7168.0	103424.0	60416.0	1986560.0	148433.5	52092.0	46656.1	6780.0	6202.2	13	0.531	0	0.000	0.531
0.0	7168.0	0.0	7168.0	103424.0	60416.0	1986560.0	148433.5	52092.0	46656.1	6780.0	6202.2	13	0.531	0	0.000	0.531
0.0	7168.0	0.0	7168.0	103424.0	60416.0	1986560.0	148433.5	52092.0	46656.1	6780.0	6202.2	13	0.531	0	0.000	0.531
0.0	7168.0	0.0	7168.0	103424.0	60416.0	1986560.0	148433.5	52092.0	46656.1	6780.0	6202.2	13	0.531	0	0.000	0.531
0.0	7168.0	0.0	7168.0	103424.0	60416.0	1986560.0	148433.5	52092.0	46656.1	6780.0	6202.2	13	0.531	0	0.000	0.531
0.0	7168.0	0.0	7168.0	103424.0	61440.0	1986560.0	148433.5	52092.0	46656.1	6780.0	6202.2	13	0.531	0	0.000	0.531
0.0	7168.0	0.0	7168.0	103424.0	61440.0	1986560.0	148433.5	52092.0	46656.1	6780.0	6202.2	13	0.531	0	0.000	0.531
0.0	7168.0	0.0	7168.0	103424.0	61440.0	1986560.0	148433.5	52092.0	46656.1	6780.0	6202.2	13	0.531	0	0.000	0.531
0.0	7168.0	0.0	7168.0	103424.0	61440.0	1986560.0	148433.5	52092.0	46656.1	6780.0	6202.2	13	0.531	0	0.000	0.531
0.0	7168.0	0.0	7168.0	103424.0	61440.0	1986560.0	148433.5	52092.0	46656.1	6780.0	6202.2	13	0.531	0	0.000	0.531
0.0	7168.0	0.0	7168.0	103424.0	61440.0	1986560.0	148433.5	52092.0	46656.1	6780.0	6202.2	13	0.531	0	0.000	0.531

参数说明：

YGC：年轻代垃圾回收次数；

(3) 根据 Kafka 进程号，查看 Kafka 的堆内存

```
[atguigu@hadoop102 kafka]$ jmap -heap 2321
... ..
Heap Usage:
G1 Heap:
  regions    = 2048
  capacity   = 2147483648 (2048.0MB)
  used       = 246367744 (234.95458984375MB)
  free       = 1901115904 (1813.04541015625MB)
11.472392082214355% used
```

2) 页缓存：

页缓存是 Linux 系统服务器的内存。我们只需要保证 1 个 segment (1g) 中 25% 的数据在内存中就好。

每个节点页缓存大小 = (分区数 * 1g * 25%) / 节点数。例如 10 个分区，页缓存大小 = $(10 * 1g * 25\%) / 3 \approx 1g$

建议服务器内存大于等于 11G。

1.5.29 CPU 选择

1) 默认配置

num.io.threads = 8 负责写磁盘的线程数。

num.replica.fetchers = 1 副本拉取线程数。

num.network.threads = 3 数据传输线程数。

2) 建议配置

此外还有后台的一些其他线程，比如清理数据线程，Controller 负责感知和管控整个集群的线程等等，这样算，每个 Broker 都会有上百个线程存在。根据经验，4 核 CPU 处理几十个线程在高峰期会打满，8 核勉强够用，而且再考虑到集群上还要运行其他的服务，所以部署 Kafka 的服务器一般建议在 16 核以上可以应对一两百个线程的工作，如果条件允许，给到 24 核甚至 32 核就更好。

num.io.threads = 16 负责写磁盘的线程数。

num.replica.fetchers = 2 副本拉取线程数。

num.network.threads = 6 数据传输线程数。

服务器建议购买 32 核 CPU

1.5.30 网络选择

网络带宽 = 峰值吞吐量 $\approx 20\text{MB/s}$ ，选择千兆网卡即可。

100Mbps 单位是 bit; 10M/s 单位是 byte; 1byte = 8bit, $100\text{Mbps}/8 = 12.5\text{M/s}$ 。

一般百兆的网卡 (100Mbps=12.5m/s)、千兆的网卡 (1000Mbps=125m/s)、万兆的网卡 (1250m/s)。

一般百兆的网卡 (100Mbps)、千兆的网卡 (1000Mbps)、万兆的网卡 (10000Mbps)。

100Mbps 单位是 bit; 10M/s 单位是 byte; 1byte = 8bit, $100\text{Mbps}/8 = 12.5\text{M/s}$ 。

通常选用千兆或者是万兆网卡。

1.5.31 Kafka 挂掉

在生产环境中，如果某个 Kafka 节点挂掉。

正常处理办法：

- (1) 先看日志，尝试重新启动一下，如果能启动正常，那直接解决。
- (2) 如果重启不行，检查内存、CPU、网络带宽。调优=》调优不行增加资源
- (3) 如果将 Kafka 整个节点误删除，如果副本数大于等于 2，可以按照服役新节点的方式重新服役一个新节点，并执行负载均衡。

1.5.32 Kafka 的机器数量



Kafka 的机器数量



- 假定生产者写入的速度为：W (MB/s)
- 写入的Topic的副本数为：R
- 同一Topic的消费者组的个数为：C

因为副本同步以及多个消费者组都会占用整个集群的吞吐量，综合考虑后，可得到如下结论：

- 总的写入速度应为 $W \times R$ (考虑副本同步)
- 总的读取速度应为 $W \times (R-1+C)$ (考虑副本同步和多个消费者组) (读取速度比写入速度快)

所以可得到以下结论：

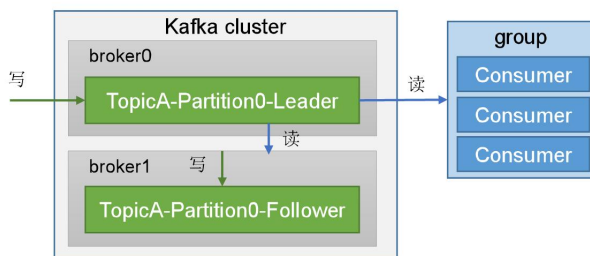
- 磁盘的总吞吐量为 $W \times R + W \times (R-1+C)$
- 网络的写吞吐量为 $W \times R$
- 网络的读吞吐量为 $W \times (R-1+C)$

说明：Kafka 充分利用了 PageCache 缓存生产者写入的数据，所以读数据 (包括消费者和副本同步) 时，不一定发生磁盘 IO。所以实际情况下，磁盘的总吞吐量一定是小于 $W \times R + W \times (R-1+C)$ 的，此处按最坏的情况进行估算。

假如单个服务器的配置如下：

网卡：全双工千兆网卡 (可提供 125M/s 的读写速率)

磁盘：机械硬盘 (可提供 200M/s 的读写速率)



若 $W=100\text{M/s}$, $R=2$, $C=1$ ，则可得到

- 磁盘的总吞吐量为 $W \times R + W \times (R-1+C) = 400\text{M/s}$
- 网络的写吞吐量为 $W \times R = 200\text{M/s}$
- 读吞吐量为 $W \times (R-1+C) = 200\text{M/s}$

所以可计算出至少需要两台节点，考虑到网络协议的开销 (除了发送实际数据，还会有一些其他信息，例如 ACK 等)，通常可将计算结果乘 2。所以需要的节点数为：2~4。

1.5.33 服役新节点退役旧节点

可以通过 `bin/kafka-reassign-partitions.sh` 脚本服役和退役节点。

1.5.34 Kafka 单条日志传输大小

Kafka 对于消息体的大小默认为单条最大值是 1M 但是在我们的应用场景中，常常会出现一条消息大于 1M，如果不对 Kafka 进行配置。则会出现生产者无法将消息推送到 Kafka 或消费者无法去消费 Kafka 里面的数据，这时我们就要对 Kafka 进行以下配置：`server.properties`。

参数名称	描述
<code>message.max.bytes</code>	默认 1m，Broker 端接收每个批次消息最大值。

max.request.size	默认 1m，生产者发往 Broker 每个请求消息最大值。针对 Topic 级别设置消息体的大小。
replica.fetch.max.bytes	默认 1m，副本同步数据，每个批次消息最大值。
fetch.max.bytes	默认 Default: 52428800 (50 m) 。消费者获取服务器端一批消息最大的字节数。如果服务器端一批次的的数据大于该值（50m）仍然可以拉取回来这批数据，因此，这不是一个绝对最大值。 一批次的大小受 message.max.bytes （broker config） or max.message.bytes （topic config）影响。

1.5.35 Kafka 参数优化

重点调优参数：

- （1）buffer.memory 32m
- （2）batch.size: 16k
- （3）linger.ms 默认 0 调整 5-100ms
- （4）compression.type 采用压缩 snappy
- （5）消费者端调整 fetch.max.bytes 大小，默认是 50m。
- （6）消费者端调整 max.poll.records 大小，默认是 500 条。
- （7）单条日志大小：message.max.bytes、max.request.size、replica.fetch.max.bytes 适

当调整 2-10m

- （8）Kafka 堆内存建议每个节点：10g ~ 15g

在 kafka-server-start.sh 中修改

```
if [ "$KAFKA_HEAP_OPTS" = "x" ]; then
    export KAFKA_HEAP_OPTS="-Xmx10G -Xms10G"
fi
```

- （9）增加 CPU 核数

num.io.threads = 8 负责写磁盘的线程数

num.replica.fetchers = 1 副本拉取线程数

num.network.threads = 3 数据传输线程数

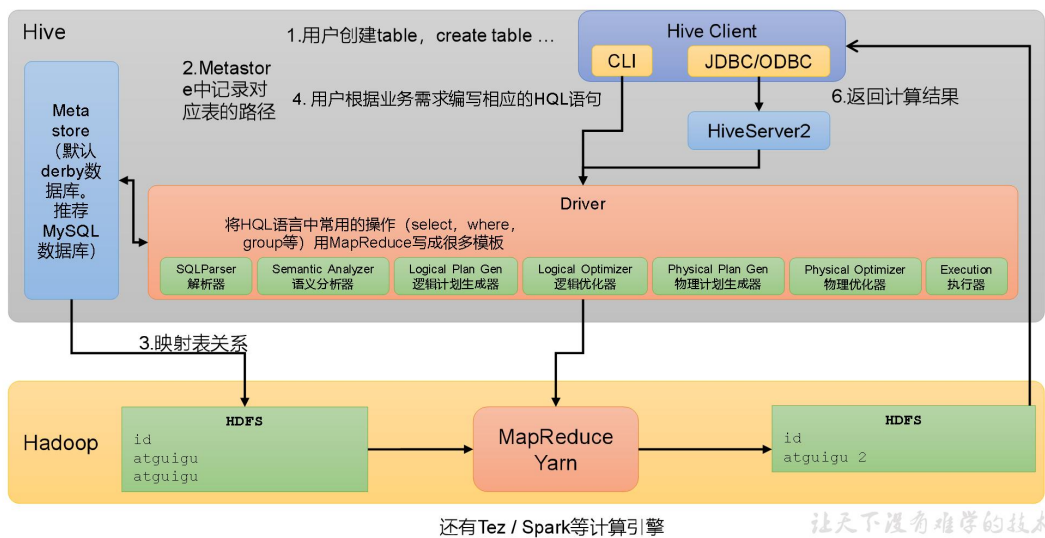
- （10）日志保存时间 log.retention.hours 3 天

- （11）副本数，调整为 2

1.6 Hive

1.6.1 Hive 的架构

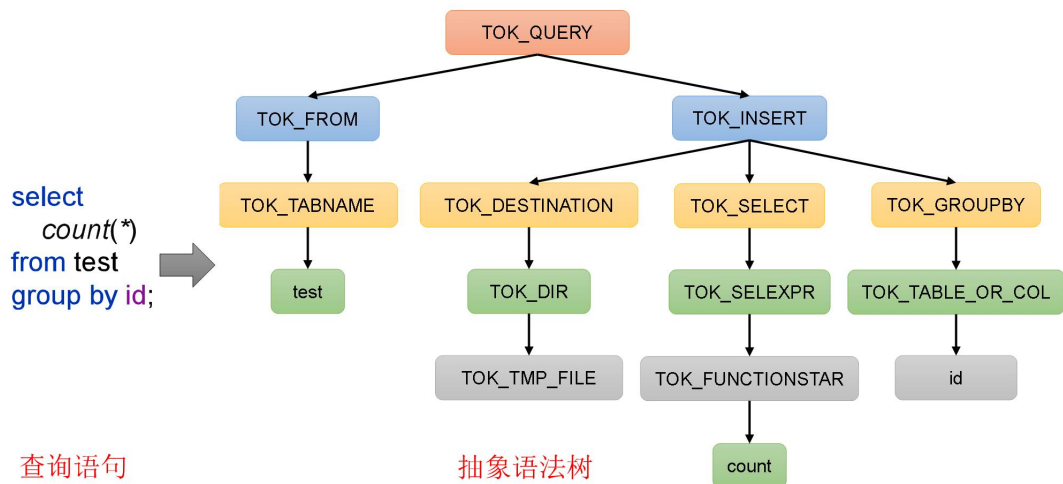
Hive架构原理



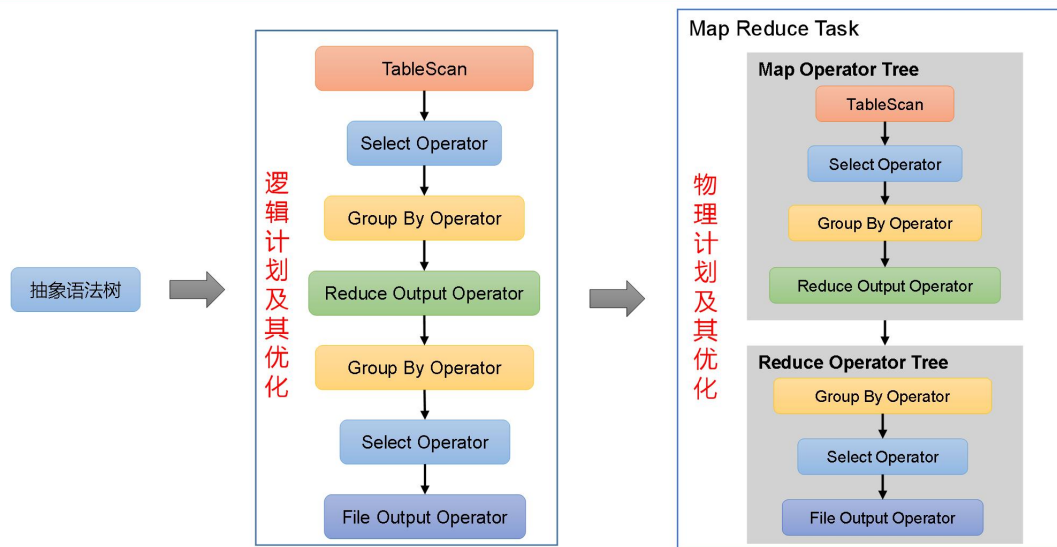
让天下没有难学的技术

1.6.2 HQL 转换为 MR 流程

抽象语法树



让天下没有难学的技术



- (1) 解析器 (SQLParser): 将 SQL 字符串转换成抽象语法树 (AST)
- (2) 语义分析器 (Semantic Analyzer): 将 AST 进一步抽象为 QueryBlock (可以理解为一个子查询划分成一个 QueryBlock)
- (2) 逻辑计划生成器 (Logical Plan Gen): 由 QueryBlock 生成逻辑计划
- (3) 逻辑优化器 (Logical Optimizer): 对逻辑计划进行优化
- (4) 物理计划生成器 (Physical Plan Gen): 根据优化后的逻辑计划生成物理计划
- (5) 物理优化器 (Physical Optimizer): 对物理计划进行优化
- (6) 执行器 (Execution): 执行该计划, 得到查询结果并返回给客户端

1.6.3 Hive 和数据库比较

Hive 和数据库除了拥有类似的查询语言, 再无类似之处。

1) 数据存储位置

Hive 存储在 HDFS。数据库将数据保存在块设备或者本地文件系统中。

2) 数据更新

Hive 中不建议对数据的改写。而数据库中的数据通常是需要经常进行修改的。

3) 执行延迟

Hive 执行延迟较高。数据库的执行延迟较低。当然, 这个是有条件的, 即数据规模较小, 当数据规模大到超过数据库的处理能力的时候, Hive 的并行计算显然能体现出优势。

4) 数据规模

Hive 支持很大规模的数据计算；数据库可以支持的数据规模较小。

1.6.4 内部表和外部表

元数据、原始数据

1) 删除数据时

内部表：元数据、原始数据，全删除

外部表：元数据 只删除

2) 在公司生产环境下，什么时候创建内部表，什么时候创建外部表？

在公司中绝大多数场景都是外部表。

自己使用的临时表，才会创建内部表；

1.6.5 系统函数

1) 数值函数

(1) round：四舍五入；(2) ceil：向上取整；(3) floor：向下取整

2) 字符串函数

(1) substring：截取字符串；(2) replace：替换；(3) regexp_replace：正则替换

(4) regexp：正则匹配；(5) repeat：重复字符串；(6) split：字符串切割

(7) nvl：替换 null 值；(8) concat：拼接字符串；

(9) concat_ws：以指定分隔符拼接字符串或者字符串数组；

(10) get_json_object：解析 JSON 字符串

3) 日期函数

(1) unix_timestamp：返回当前或指定时间的时间戳

(2) from_unixtime：转化 UNIX 时间戳（从 1970-01-01 00:00:00 UTC 到指定时间的秒数）到当前时区的时间格式

(3) current_date：当前日期

(4) current_timestamp：当前的日期加时间，并且精确的毫秒

(5) month：获取日期中的月；(6) day：获取日期中的日

(7) datediff：两个日期相差的天数（结束日期减去开始日期的天数）

(8) date_add：日期加天数；(9) date_sub：日期减天数

(10) date_format：将标准日期解析成指定格式字符串

4) 流程控制函数

- (1) case when: 条件判断函数
- (2) if: 条件判断, 类似于 Java 中三元运算符

5) 集合函数

- (1) array: 声明 array 集合
- (2) map: 创建 map 集合
- (3) named_struct: 声明 struct 的属性和值
- (4) size: 集合中元素的个数
- (5) map_keys: 返回 map 中的 key
- (6) map_values: 返回 map 中的 value
- (7) array_contains: 判断 array 中是否包含某个元素
- (8) sort_array: 将 array 中的元素排序

6) 聚合函数

- (1) collect_list: 收集并形成 list 集合, 结果不去重
- (2) collect_set: 收集并形成 set 集合, 结果去重

1.6.6 自定义 UDF、UDTF 函数

1) 在项目中是否自定义过 UDF、UDTF 函数, 以及用他们处理了什么问题, 及自定义步骤?

- (1) 目前项目中逻辑不是特别复杂就没有用自定义 UDF 和 UDTF
- (2) 自定义 UDF: 继承 G..UDF, 重写核心方法 evaluate
- (3) 自定义 UDTF: 继承自 GenericUDTF, 重写 3 个方法: initialize (自定义输出的列名和类型), process (将结果返回 forward (result)), close

2) 企业中一般什么场景下使用 UDF/UDTF?

- (1) 因为自定义函数, 可以将自定义函数内部任意计算过程打印输出, 方便调试。
- (2) 引入第三方 jar 包时, 也需要。

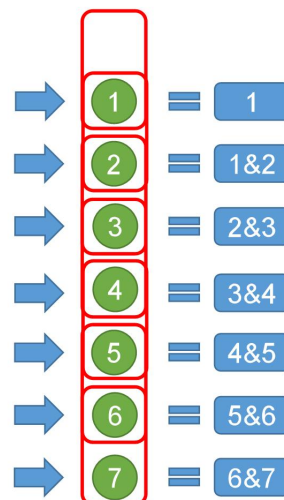
1.6.7 窗口函数

一般在场景题中出现手写: 分组 TopN、行转列、列转行。



定义

窗口函数，能为每行数据划分一个窗口，然后对窗口范围内的数据进行计算，最后将计算结果返回给该行数据。



按照功能，常用窗口可划分为如下几类：聚合函数、跨行取值函数、排名函数。

1) 聚合函数

max: 最大值。

min: 最小值。

sum: 求和。

avg: 平均值。

count: 计数。

2) 跨行取值函数

(1) lead 和 lag



常用窗口函数——lead和lag

功能：获取当前行的上/下边某行、某个字段的值。

语法：

```
select
  order_id,
  user_id,
  order_date,
  amount,
  lag(order_date, 1, '1970-01-01') over (partition by user_id order by order_date) last_date,
  lead(order_date, 1, '9999-12-31') over (partition by user_id order by order_date) next_date
from order_info;
```

字段名 偏移量 默认值

思考：该查询语句的结果是什么？

order_id	user_id	order_date	amount	order_id	user_id	order_date	amount	last_date	next_date
1	1001	2022-01-01	10	1	1001	2022-01-01	10	1970-01-01	2022-01-02
2	1001	2022-01-02	20	2	1001	2022-01-02	20	2022-01-01	2022-01-03
3	1001	2022-01-03	10	3	1001	2022-01-03	10	2022-01-02	9999-12-31
4	1002	2022-01-04	30	4	1002	2022-01-04	30	1970-01-01	2022-01-05
5	1002	2022-01-05	40	5	1002	2022-01-05	40	2022-01-05	2022-01-06
6	1002	2022-01-06	20	6	1002	2022-01-06	20	2022-01-06	9999-12-31

注：lag 和 lead 函数不支持自定义窗口。

(2) first_value 和 last_value



窗口函数(window functions)



常用窗口函数——first_value和last_value

功能：获取窗口内某一列的第一个值/最后一个值

语法：

```
select
  order_id,
  user_id,
  order_date,
  amount,
  first_value(order_date,false) over (partition by user_id order by order_date) first_date,
  last_value(order_date,false) over (partition by user_id order by order_date) last_date
from order_info;
```

字段名 是否跳过null

思考：该查询语句的结果是什么？

order_id	user_id	order_date	amount	order_id	user_id	order_date	amount	first_date	last_date
1	1001	2022-01-01	10	1	1001	2022-01-01	10	2022-01-01	2022-01-03
2	1001	2022-01-02	20	2	1001	2022-01-02	20	2022-01-01	2022-01-03
3	1001	2022-01-03	10	3	1001	2022-01-03	10	2022-01-01	2022-01-03
4	1002	2022-01-04	30	4	1002	2022-01-04	30	2022-01-04	2022-01-06
5	1002	2022-01-05	40	5	1002	2022-01-05	40	2022-01-04	2022-01-06
6	1002	2022-01-06	20	6	1002	2022-01-06	20	2022-01-04	2022-01-06

3) 排名函数



窗口函数(window functions)



常用窗口函数——rank、dense_rank、row_number

功能：计算排名

语法：

```
select
  stu_id,
  course,
  score,
  rank() over(partition by course order by score desc) rk,
  dense_rank() over(partition by course order by score desc) dense_rk,
  row_number() over(partition by course order by score desc) rn
from score_info;
```

stu_id	course	score	stu_id	course	score	rk	dense_rk	rn
1	语文	99	1	语文	99	1	1	1
2	语文	98	2	语文	98	2	2	2
3	语文	95	3	语文	95	3	3	3
4	数学	100	4	数学	100	1	1	1
5	数学	100	5	数学	100	1	1	2
6	数学	99	6	数学	99	3	2	3

注：rank 、dense_rank、row_number 不支持自定义窗口。

1.6.8 Hive 优化

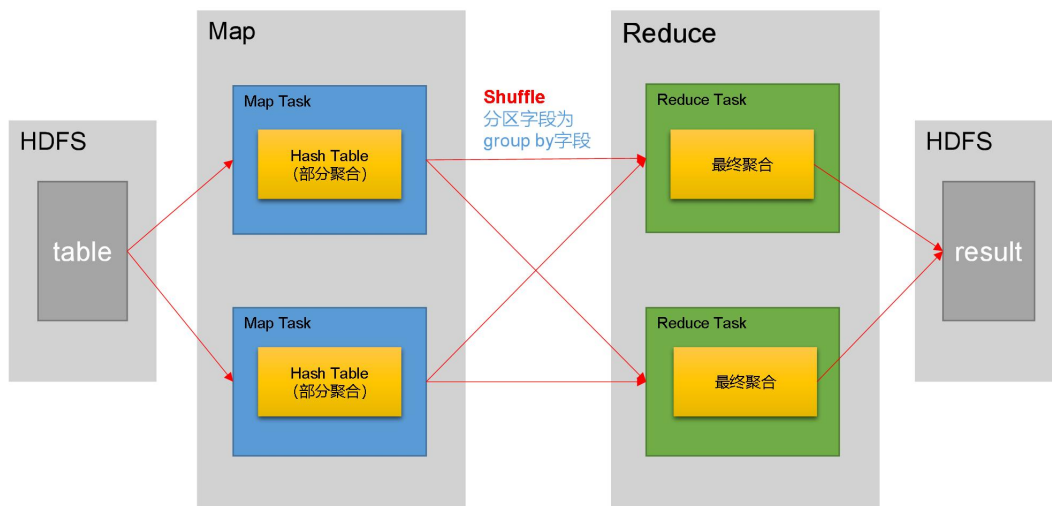
1.6.8.1 分组聚合

一个分组聚合的查询语句，默认是通过一个 MapReduce Job 完成的。Map 端负责读取数据，并按照分组字段分区，通过 Shuffle，将数据发往 Reduce 端，各组数据在 Reduce 端完成最终的聚合运算。

分组聚合的优化主要围绕着减少 Shuffle 数据量进行，具体做法是 map-side 聚合。所谓 map-side 聚合，就是在 map 端维护一个 Hash Table，利用其完成部分的聚合，然后将部分聚合的结果，按照分组字段分区，发送至 Reduce 端，完成最终的聚合。



map-side聚合原理



相关参数如下：

```
--启用 map-side 聚合，默认是 true
set hive.map.aggr=true;

--用于检测源表数据是否适合进行 map-side 聚合。检测的方法是：先对若干条数据进行
map-side 聚合，若聚合后的条数和聚合前的条数比值小于该值，则认为该表适合进行
map-side 聚合；否则，认为该表数据不适合进行 map-side 聚合，后续数据便不再进行
map-side 聚合。
set hive.map.aggr.hash.min.reduction=0.5;

--用于检测源表是否适合 map-side 聚合的条数。
set hive.groupby.mapaggr.checkinterval=100000;

--map-side 聚合所用的 hash table，占用 map task 堆内存的最大比例，若超出该值，
则会对 hash table 进行一次 flush。
set hive.map.aggr.hash.force.flush.memory.threshold=0.9;
```

1.6.8.2 Map Join

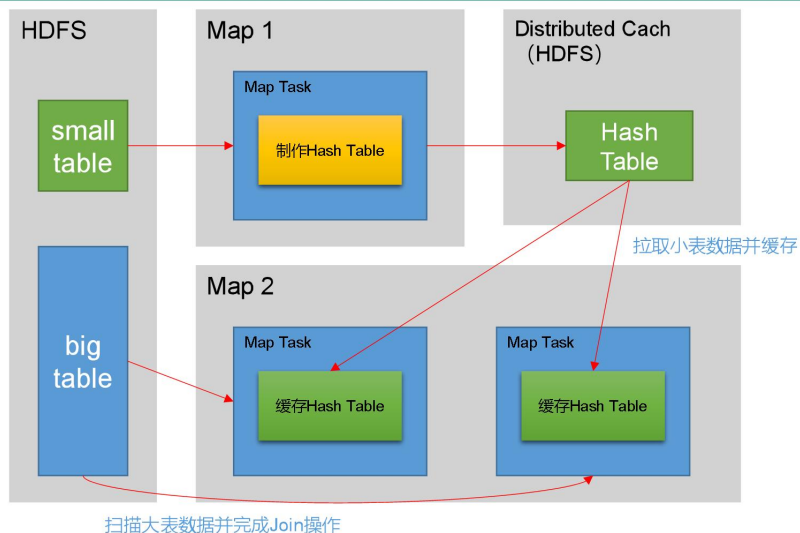
Hive 中默认最稳定的 Join 算法是 Common Join。其通过一个 MapReduce Job 完成一个 Join 操作。Map 端负责读取 Join 操作所需表的数据，并按照关联字段进行分区，通过 Shuffle，将其发送到 Reduce 端，相同 key 的数据在 Reduce 端完成最终的 Join 操作。

优化 Join 的最为常用的手段就是 Map Join，其可通过两个只有 Map 阶段的 Job 完成一个 join 操作。第一个 Job 会读取小表数据，将其制作为 Hash Table，并上传至 Hadoop 分

布式缓存（本质上是上传至 HDFS）。第二个 Job 会先从分布式缓存中读取小表数据，并缓存在 Map Task 的内存中，然后扫描大表数据，这样在 map 端即可完成关联操作。

注：由于 Map Join 需要缓存整个小表的数据，故只适用于大表 Join 小表的场景。

Map Join原理



相关参数如下：

```
--启动 Map Join 自动转换
set hive.auto.convert.join=true;
--开启无条件转 Map Join
set hive.auto.convert.join.noconditionaltask=true;
--无条件转 Map Join 小表阈值，默认值 10M，推荐设置为 Map Task 总内存的三分之一到二分之一
set hive.auto.convert.join.noconditionaltask.size=10000000;
```

1.6.8.3 SMB Map Join

上节提到，Map Join 只适用于大表 Join 小表的场景。若想提高大表 Join 大表的计算效率，可使用 Sort Merge Bucket Map Join。

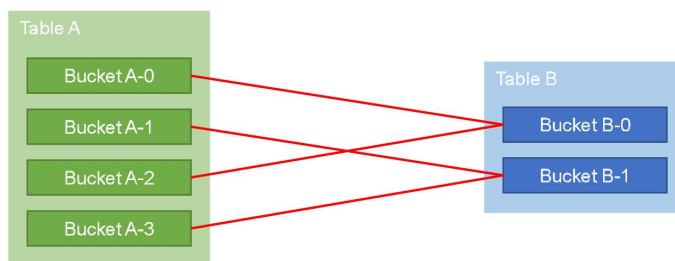
需要注意的是 SMB Map Join 有如下要求：

- (1) 参与 Join 的表均为分桶表，且分桶字段为 Join 的关联字段。
- (2) 两表分桶数呈倍数关系。
- (3) 数据在分桶内是按关联字段有序的。

SMB Join 的核心原理如下：只要保证了上述三点要求的前两点，就能保证参与 Join 的两张表的分桶之间具有明确的关联关系，因此就可以在两表的分桶间进行 Join 操作了。



Bucket对应关系



让天下没有难学的技术

若能保证第三点,也就是参与 Join 的数据是有序的,这样就能使用数据库中常用的 Join 算法之一——Sort Merge Join 了, Merge Join 原理如下:



Sort Merge Join原理

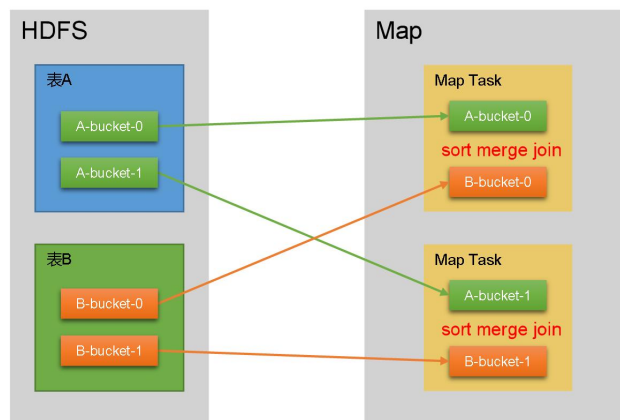


order_detail				join	user_info			on
order_id	user_id	product_id	total_amount		user_id	gender	name	
1001	1	10	25.20		1	男	张三	order_detail.user_id=user_info.user_id
1002	2	11	33.10		2	女	李四	
1003	2	12	45.10		3	女	王五	
1004	3	13	44.90		4	男	赵六	
1005	3	14	33.02		5	女	小红	
1006	4	15	100.10					

order_id	user_id	product_id	total_amount	user_id	gender	name
1001	1	10	25.20	1	男	张三
1002	2	11	33.10	2	女	李四
1003	2	12	45.10	2	女	李四
1004	3	13	44.90	3	女	王五
1005	3	14	33.02	3	女	王五
1006	4	15	100.10	4	男	赵六

让天下没有难学的技术

在满足了上述三点要求之后,就能使用 SMB Map Join 了。



由于 SMB Map Join 无需构建 Hash Table 也无需缓存小表数据，故其对内存要求很低。适用于大表 Join 大表的场景。

1.6.8.4 Reduce 并行度

Reduce 端的并行度，也就是 Reduce 个数，可由用户自己指定，也可由 Hive 自行根据该 MR Job 输入的文件大小进行估算。

Reduce 端的并行度的相关参数如下：

```
--指定 Reduce 端并行度，默认值为-1，表示用户未指定
set mapreduce.job.reduces;
--Reduce 端并行度最大值
set hive.exec.reducers.max;
--单个 Reduce Task 计算的数据量，用于估算 Reduce 并行度
set hive.exec.reducers.bytes.per.reducer;
```

Reduce 端并行度的确定逻辑如下：

若指定参数 `mapreduce.job.reduces` 的值为一个非负整数，则 Reduce 并行度为指定值。

否则，Hive 自行估算 Reduce 并行度，估算逻辑如下：

假设 Job 输入的文件大小为 `totalInputBytes`

参数 `hive.exec.reducers.bytes.per.reducer` 的值为 `bytesPerReducer`。

参数 `hive.exec.reducers.max` 的值为 `maxReducers`。

则 Reduce 端的并行度为：

$$\min \left(\text{ceil} \left(\frac{\text{totalInputBytes}}{\text{bytesPerReducer}} \right), \text{maxReducers} \right)$$

根据上述描述，可以看出，Hive 自行估算 Reduce 并行度时，是以整个 MR Job 输入的

文件大小作为依据的。因此，在某些情况下其估计的并行度很可能并不准确，此时就需要用户根据实际情况来指定 Reduce 并行度了。

需要说明的是：若使用 Tez 或者是 Spark 引擎，Hive 可根据计算统计信息（Statistics）估算 Reduce 并行度，其估算的结果相对更加准确。

1.6.8.5 小文件合并

若 Hive 的 Reduce 并行度设置不合理，或者估算不合理，就可能导致计算结果出现大量的小文件。该问题可由小文件合并任务解决。其原理是根据计算任务输出文件的平均大小进行判断，若符合条件，则单独启动一个额外的任务进行合并。

相关参数为：

```
--开启合并 map only 任务输出的小文件
set hive.merge.mapfiles=true;

--开启合并 map reduce 任务输出的小文件
set hive.merge.mapredfiles=true;

--合并后的文件大小
set hive.merge.size.per.task=256000000;

--触发小文件合并任务的阈值，若某计算任务输出的文件平均大小低于该值，则触发合并
set hive.merge.smallfiles.avgsize=16000000;
```

1.6.8.6 谓词下推

谓词下推（predicate pushdown）是指，尽量将过滤操作前移，以减少后续计算步骤的数据量。开启谓词下推优化后，无需调整 SQL 语句，Hive 就会自动将过滤操作尽可能的前移动。

相关参数为：

```
--是否启动谓词下推（predicate pushdown）优化
set hive.optimize.ppd = true;
```

1.6.8.7 并行执行

Hive 会将一个 SQL 语句转化成一個或者多个 Stage，每个 Stage 对应一个 MR Job。默认情况下，Hive 同时只会执行一个 Stage。但是某 SQL 语句可能会包含多个 Stage，但这多个 Stage 可能并非完全互相依赖，也就是说有些 Stage 是可以并行执行的。此处提到的并行执行就是指这些 Stage 的并行执行。相关参数如下：

```
--启用并行执行优化，默认是关闭的
set hive.exec.parallel=true;

--同一个 sql 允许最大并行度，默认为 8
set hive.exec.parallel.thread.number=8;
```

1.6.8.8 CBO 优化

CBO 是指 Cost based Optimizer，即基于计算成本的优化。

在 Hive 中，计算成本模型考虑到了：数据的行数、CPU、本地 IO、HDFS IO、网络 IO 等方面。Hive 会计算同一 SQL 语句的不同执行计划的计算成本，并选出成本最低的执行计划。目前 CBO 在 Hive 的 MR 引擎下主要用于 Join 的优化，例如多表 Join 的 Join 顺序。

相关参数为：

```
--是否启用 cbo 优化
set hive.cbo.enable=true;
```

1.6.8.9 列式存储

采用 ORC 列式存储加快查询速度。

	id	name	age
	1	zs	18
	2	lishi	19
行: 1	zs	18	2 lishi 19
列: 1 2	zs	lishi	18 19

select name from user

1.6.8.10 压缩

压缩减少磁盘 IO：因为 Hive 底层计算引擎默认是 MR，可以在 Map 输出端采用 Snappy 压缩。

Map (Snappy) Reduce

1.6.8.11 分区和分桶

- (1) 创建分区表 防止后续全表扫描
- (2) 创建分桶表 对未知的复杂的数据进行提前采样

1.6.8.12 更换引擎

1) MR/Tez/Spark 区别：

MR 引擎：多 Job 串联，基于磁盘，落盘的地方比较多。虽然慢，但一定能跑出结果。
一般处理，周、月、年指标。

Spark 引擎：虽然在 Shuffle 过程中也落盘，但是并不是所有算子都需要 Shuffle，尤其是多算子过程，中间过程不落盘 DAG 有向无环图。兼顾了可靠性和效率。一般处理天指标。

2) Tez 引擎的优点

(1) 使用 DAG 描述任务，可以减少 MR 中不必要的中间节点，从而减少磁盘 IO 和网络 IO。

(2) 可更好的利用集群资源，例如 Container 重用、根据集群资源计算初始任务的并行度等。

(3) 可在任务运行时，根据具体数据量，动态的调整后续任务的并行度。

1.6.8.13 几十张表 join 如何优化

(1) 减少 join 的表数量：不影响业务前提，可以考虑将一些表进行预处理和合并，从而减少 join 操作。

(2) 使用 Map Join：将小表加载到内存中，从而避免了 Reduce 操作，提高了性能。通过设置 `hive.auto.convert.join` 为 `true` 来启用自动 Map Join。

(3) 使用 Bucketed Map Join：通过设置 `hive.optimize.bucketmapjoin` 为 `true` 来启用 Bucketed Map Join。

(4) 使用 Sort Merge Join：这种方式在 Map 阶段完成排序，从而减少了 Reduce 阶段的计算量。通过设置 `hive.auto.convert.sortmerge.join` 为 `true` 来启用。

(5) 控制 Reduce 任务数量：通过合理设置 `hive.exec.reducers.bytes.per.reducer` 和 `mapreduce.job.reduces` 参数来控制 Reduce 任务的数量。

(6) 过滤不需要的数据：join 操作之前，尽量过滤掉不需要的数据，从而提高性能。

(7) 选择合适的 join 顺序：将小表放在前面可以减少中间结果的数据量，提高性能。

(8) 使用分区：可以考虑使用分区技术。只需要读取与查询条件匹配的分区数据，从而减少数据量和计算量。

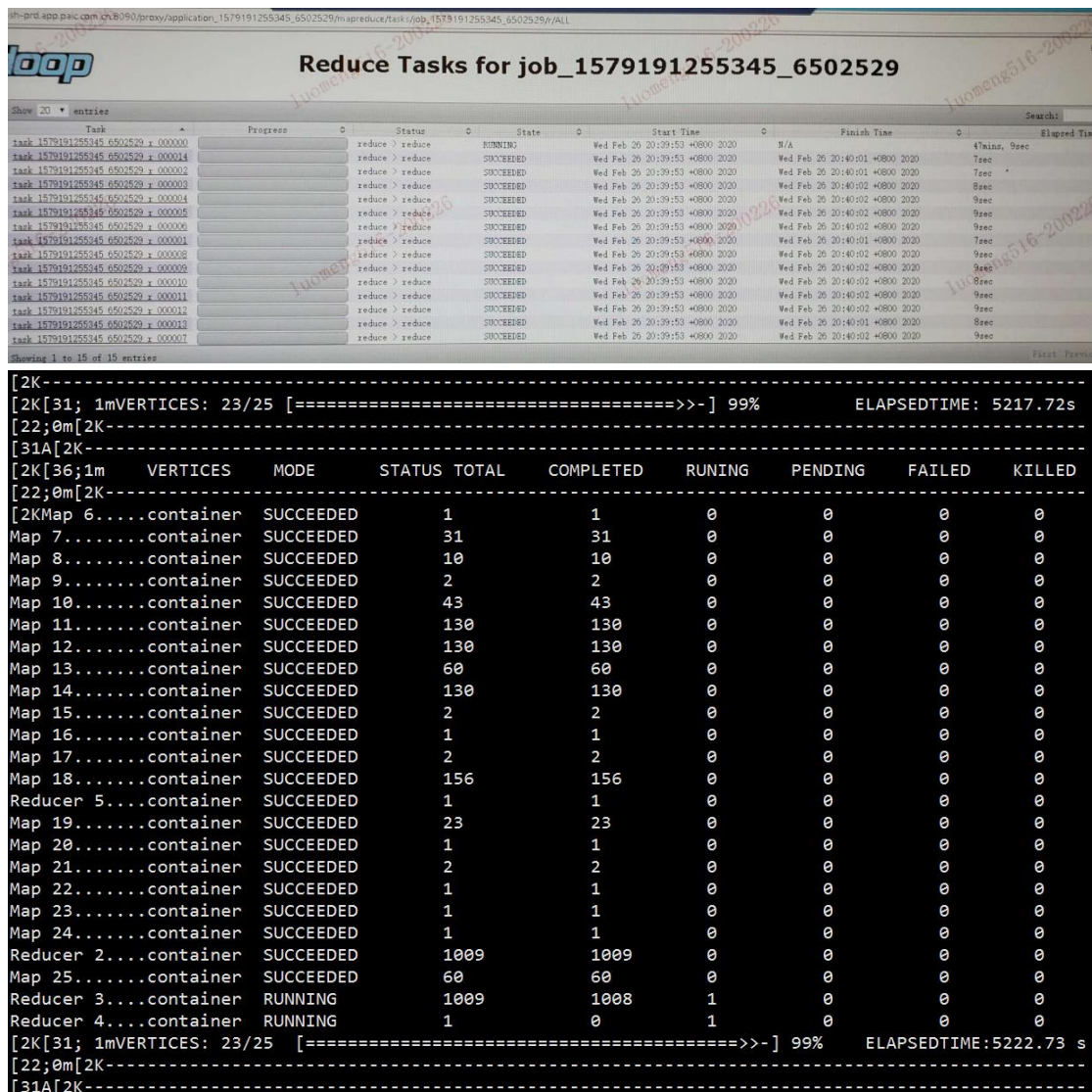
(9) 使用压缩：通过对数据进行压缩，可以减少磁盘和网络 IO，提高性能。注意选择合适的压缩格式和压缩级别。

(10) 调整 Hive 配置参数：根据集群的硬件资源和实际需求，合理调整 Hive 的配置参数，如内存、CPU、IO 等，以提高性能。

1.6.9 Hive 解决数据倾斜方法

数据倾斜问题，通常是指参与计算的数据分布不均，即某个 key 或者某些 key 的数据量远超其他 key，导致在 shuffle 阶段，大量相同 key 的数据被发往同一个 Reduce，进而导致

该 Reduce 所需的时间远超其他 Reduce，成为整个任务的瓶颈。以下为生产环境中数据倾斜的现象：



Hive 中的数据倾斜常出现在分组聚合和 join 操作的场景中，下面分别介绍在上述两种场景下的优化思路。

1) 分组聚合导致的数据倾斜

前文提到过，Hive 中的分组聚合是由一个 MapReduce Job 完成的。Map 端负责读取数据，并按照分组字段分区，通过 Shuffle，将数据发往 Reduce 端，各组数据在 Reduce 端完成最终的聚合运算。若 group by 分组字段的值分布不均，就可能导致大量相同的 key 进入同一 Reduce，从而导致数据倾斜。

由分组聚合导致的数据倾斜问题，有如下解决思路：

(1) 判断倾斜的值是否为 null

若倾斜的值为 null，可考虑最终结果是否需要这部分数据，若不需要，只要提前将 null 过滤掉，就能解决问题。若需要保留这部分数据，考虑以下思路。

(2) Map-Side 聚合

开启 Map-Side 聚合后，数据会现在 Map 端完成部分聚合工作。这样一来即便原始数据是倾斜的，经过 Map 端的初步聚合后，发往 Reduce 的数据也就不再倾斜了。最佳状态下，Map 端聚合能完全屏蔽数据倾斜问题。

相关参数如下：

```
set hive.map.aggr=true;
set hive.map.aggr.hash.min.reduction=0.5;
set hive.groupby.mapaggr.checkinterval=100000;
set hive.map.aggr.hash.force.flush.memory.threshold=0.9;
```

(3) Skew-GroupBy 优化

Skew-GroupBy 是 Hive 提供的一个专门用来解决分组聚合导致的数据倾斜问题的方案。其原理是启动两个 MR 任务，第一个 MR 按照随机数分区，将数据分散发送到 Reduce，并完成部分聚合，第二个 MR 按照分组字段分区，完成最终聚合。

相关参数如下：

```
--启用分组聚合数据倾斜优化
set hive.groupby.skewindata=true;
```

2) Join 导致的数据倾斜

若 Join 操作使用的是 Common Join 算法，就会通过一个 MapReduce Job 完成计算。Map 端负责读取 Join 操作所需表的数据，并按照关联字段进行分区，通过 Shuffle，将其发送到 Reduce 端，相同 key 的数据在 Reduce 端完成最终的 Join 操作。

如果关联字段的值分布不均，就可能导致大量相同的 key 进入同一 Reduce，从而导致数据倾斜问题。

由 Join 导致的数据倾斜问题，有如下解决思路：

(1) Map Join

使用 Map Join 算法，Join 操作仅在 Map 端就能完成，没有 Shuffle 操作，没有 Reduce 阶段，自然不会产生 Reduce 端的数据倾斜。该方案适用于大表 Join 小表时发生数据倾斜的场景。

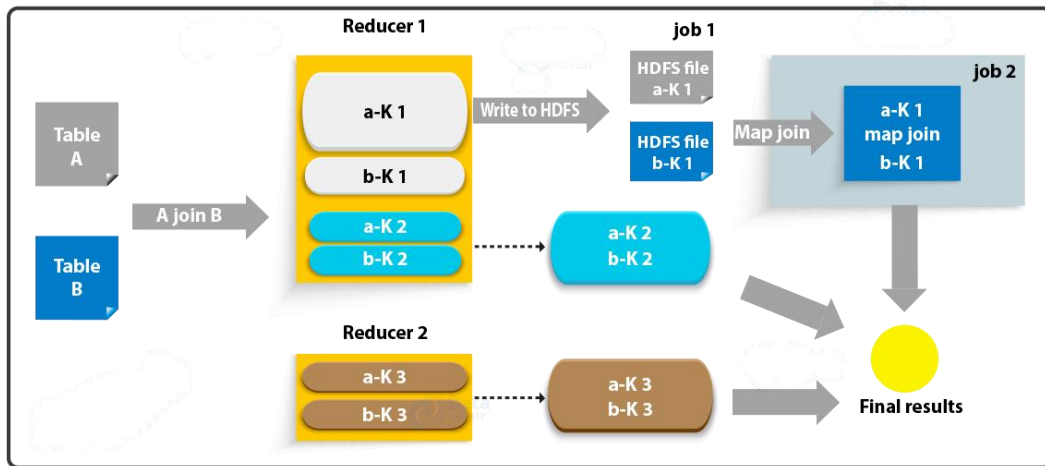
相关参数如下：

```
set hive.auto.convert.join=true;
set hive.auto.convert.join.noconditionaltask=true;
set hive.auto.convert.join.noconditionaltask.size=10000000;
```

(2) Skew Join

若参与 Join 的两表均为大表，Map Join 就难以应对了。此时可考虑 Skew Join，其核心原理是 Skew Join 的原理是，为倾斜的大 key 单独启动一个 Map Join 任务进行计算，其余 key 进行正常的 Common Join。原理图如下：

Skew Join



相关参数如下：

```
--启用 skew join 优化
set hive.optimize.skewjoin=true;
--触发 skew join 的阈值，若某个 key 的行数超过该参数值，则触发
set hive.skewjoin.key=100000;
```

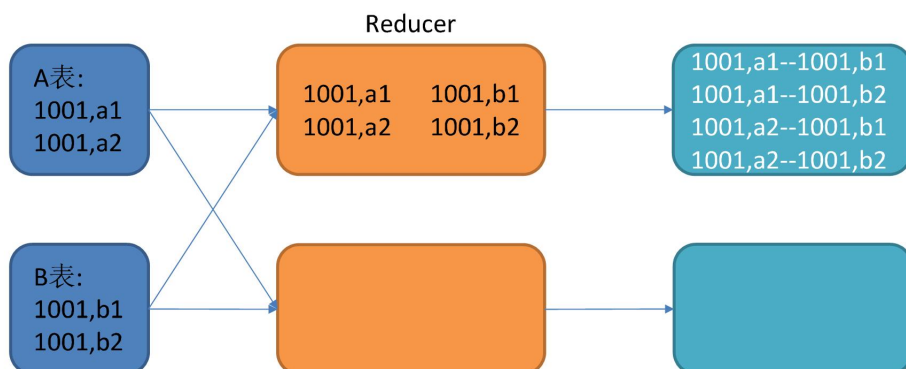
3) 调整 SQL 语句

若参与 Join 的两表均为大表，其中一张表的数据是倾斜的，此时也可通过以下方式对 SQL 语句进行相应的调整。

假设原始 SQL 语句如下：A, B 两表均为大表，且其中一张表的数据是倾斜的。

```
hive (default)>
select
  *
from A
join B
on A.id=B.id;
```

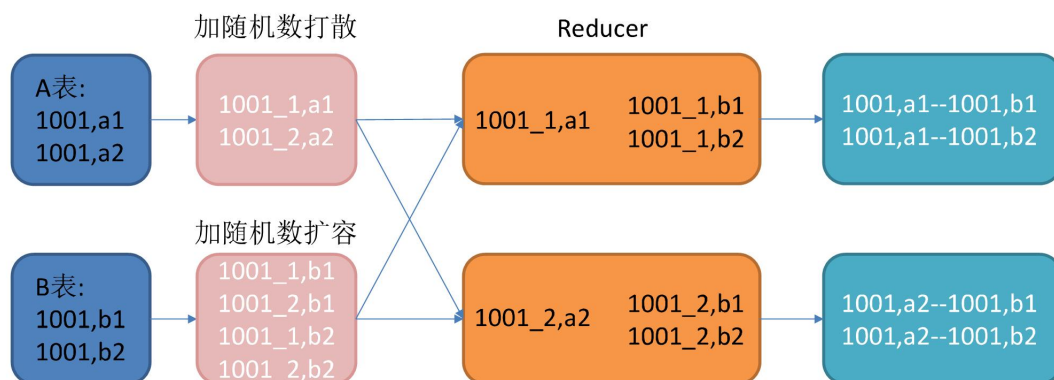
其 Join 过程如下：



让天下没有难学的技术

图中 1001 为倾斜的大 key，可以看到，其被发往了同一个 Reduce 进行处理。

调整之后的 SQL 语句执行计划如下图所示：



让天下没有难学的技术

调整 SQL 语句如下：

```
hive (default)>
select
  *
from(
  select --打散操作
    concat(id, '_', cast(rand()*2 as int)) id,
    value
  from A
)ta
join(
  select --扩容操作
```

```

        concat(id, '_', 1) id,
        value
    from B
    union all
    select
        concat(id, '_', 2) id,
        value
    from B
)tb
on ta.id=tb.id;
```

1.6.10 Hive 的数据中含有字段的分隔符怎么处理？

Hive 默认的字段分隔符为 Ascii 码的控制符\001(^A),建表的时候用 fields terminated by '\001'。注意：如果采用\t或者\001等为分隔符，需要要求前端埋点和 JavaEE 后台传递过来的数据必须不能出现该分隔符，通过代码规范约束。

一旦传输过来的数据含有分隔符，需要在前一级数据中转义或者替换（ETL）。通常采用 Sqoop 和 DataX 在同步数据时预处理。

```

id  name  age
1   zs    18

2   li 分隔符 si    19
```

1.6.11 MySQL 元数据备份

元数据备份（重点，如数据损坏，可能整个集群无法运行，至少要保证每日零点之后备份到其它服务器两个复本）。

（1）MySQL 备份数据脚本（建议每天定时执行一次备份元数据）

```

#!/bin/bash
#常量设置
MYSQL_HOST='hadoop102'
MYSQL_USER='root'
MYSQL_PASSWORD='000000'
# 备份目录，需提前创建
BACKUP_DIR='/root/mysql-backup'
# 备份天数，超过这个值，最旧的备份会被删除
FILE_ROLL_COUNT='7'

# 备份 MySQL 数据库
[ -d "${BACKUP_DIR}" ] || exit 1
mysqldump \
--all-databases \
--opt \
--single-transaction \
--source-data=2 \
```

```
--default-character-set=utf8 \
-h"${MYSQL_HOST}" \
-u"${MYSQL_USER}" \
-p"${MYSQL_PASSWORD}" | gzip > "${BACKUP_DIR}/${date +%F}.gz"

if [ "$(ls "${BACKUP_DIR}" | wc -l )" -gt "${FILE_ROLL_COUNT}" ]
then
    ls "${BACKUP_DIR}" | sort | sed -n 1p | xargs -I {} -n1 rm -rf
    "${BACKUP_DIR}/${}"
fi
```

(2) MySQL 恢复数据脚本

```
#!/bin/bash
#常量设置
MYSQL_HOST='hadoop102'
MYSQL_USER='root'
MYSQL_PASSWORD='000000'
BACKUP_DIR='/root/mysql-backup'
# 恢复指定日期，不指定就恢复最新数据
RESTORE_DATE=''

[ "${RESTORE_DATE}" ] && BACKUP_FILE="${RESTORE_DATE}.gz" ||
BACKUP_FILE="$(ls ${BACKUP_DIR} | sort -r | sed -n 1p)"
gunzip "${BACKUP_DIR}/${BACKUP_FILE}" --stdout | mysql \
-h"${MYSQL_HOST}" \
-u"${MYSQL_USER}" \
-p"${MYSQL_PASSWORD}"
```

1.6.12 如何创建二级分区表？

```
create table dept_partition2(
    deptno int,    -- 部门编号
    dname string, -- 部门名称
)
partitioned by (day string, hour string)
row format delimited fields terminated by '\t';
```

1.6.13 Union 与 Union all 区别

- (1) union 会将联合的结果集去重
- (2) union all 不会对结果集去重

1.7 Datas

1.7.1 DataX 与 Sqoop 区别

- 1) DataX 与 Sqoop 都是主要用于离线系统中批量同步数据处理场景。
- 2) DataX 和 Sqoop 区别如下：

- (1) DataX 底层是单进程多线程；Sqoop 底层是 4 个 Map；

(2) 数据量大的场景优先考虑 Sqoop 分布式同步；数据量小的场景优先考虑 DataX，完全基于内存；DataX 数据量大，可以使用多个 DataX 实例，每个实例负责一部分（手动划分）。

(3) Sqoop 是为 Hadoop 而生的，对 Hadoop 相关组件兼容性比较好；Datax 是插件化开发，支持的 Source 和 Sink 更多一些。

(4) Sqoop 目前官方不在升级维护；DataX 目前阿里在升级维护

(5) 关于运行日志与统计信息，DataX 更丰富，Sqoop 基于 Yarn 不容易采集

1.7.2 速度控制

1) 关键优化参数如下：

参数	说明
<code>job.setting.speed.channel</code>	总并发数
<code>job.setting.speed.record</code>	总 record 限速
<code>job.setting.speed.byte</code>	总 byte 限速
<code>core.transport.channel.speed.record</code>	单个 channel 的 record 限速，默认值为 10000（10000 条/s）
<code>core.transport.channel.speed.byte</code>	单个 channel 的 byte 限速，默认值 1024*1024（1M/s）

2) 生效优先级：

(1) 全局 Byte 限速 / 单 Channel Byte 限速

(2) 全局 Record 限速 / 单 Channel Record 限速

两个都设置，取结果小的

(3) 上面都没设置，总 Channel 数的设置生效

3) 项目配置

只设置 总 channel 数=5，基本可以跑满网卡带宽。

1.7.3 内存调整

建议将内存设置为 4G 或者 8G，这个也可以根据实际情况来调整。

调整 JVM xms xmx 参数的两种方式：一种是直接更改 datax.py 脚本；另一种是在启动的时候，加上对应的参数，如下：

```
python datax/bin/datax.py --jvm="-Xms8G -Xmx8G" /path/to/your/job.json
```

1.7.4 空值处理

1) MySQL (null) => Hive (\N) 要求 Hive 建表语句

解决该问题的方案有两个：

(1) 修改 DataX HDFS Writer 的源码，增加自定义 null 值存储格式的 logic，可参考 <https://blog.csdn.net/u010834071/article/details/105506580>。

(2) 在 Hive 中建表时指定 null 值存储格式为空字符串 (")，例如：

```
DROP TABLE IF EXISTS base_province;
CREATE EXTERNAL TABLE base_province
(
    `id`          STRING COMMENT '编号',
    `name`        STRING COMMENT '省份名称',
    `region_id`   STRING COMMENT '地区 ID',
    `area_code`   STRING COMMENT '地区编码',
    `iso_code`    STRING COMMENT '旧版 ISO-3166-2 编码，供可视化使用',
    `iso_3166_2`  STRING COMMENT '新版 IOS-3166-2 编码，供可视化使用'
) COMMENT '省份表'
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'
NULL DEFINED AS "
LOCATION '/base_province/';
```

2) Hive (\N) => MySQL (null)

```
"reader": {
    "name": "hdfsreader",
    "parameter": {
        "defaultFS": "hdfs://hadoop102:8020",
        "path": "/base_province",
        "column": [
            "*"
        ],
        "fileType": "text",
        "compress": "gzip",
        "encoding": "UTF-8",
        "nullFormat": "\\N",
        "fieldDelimiter": "\t",
    }
}
```

1.7.5 配置文件生成脚本

(1) 一个表一个配置，如果有几千张表，怎么编写的配置？

(2) 脚本使用说明

```
python gen_import_config.py -d database -t table
```

1.7.6 DataX 一天导入多少数据

1) 全量同步的表如下

- 活动表、优惠规则表、优惠券表、SKU 平台属性表、SKU 销售属性表
- SPU 商品表（1-2 万）、SKU 商品表（10-20 万）、品牌表、商品一级分类、商品二级分类、商品三级分类
- 省份表、地区表
- 编码字典表

以上全部加一起 30 万条，约等于 300m。

加购表（每天增量 20 万、全量 100 万 => 1g）

所以 Datax 每天全量同步的数据 1-2g 左右。

注意：金融、保险（平安、民生银行），只有业务数据数据量大一些。

2) 增量同步的表如下

- 加购表（20 万）、订单表（10 万）、订单详情表（15 万）、订单状态表、支付表（9 万）、退单表（1000）、退款表（1000）
- 订单明细优惠券关联表、优惠券领用表
- 商品评论表、收藏表
- 用户表、订单明细活动关联表

增量数据每天 1-2g

1.7.7 Datax 如何实现增量同步

获取今天新增和变化的数据：通过 sql 过滤，创建时间是今天或者操作时间等于今天。

1.8 Maxwell

1.8.1 Maxwell 与 Canal、FlinkCDC 的对比

1) FlinkCDC、Maxwell、Canal 都是主要用于实时系统中实时数据同步处理场景。

	FlinkCDC	Maxwell	Canal
SQL 与数据条数关系	SQL 影响几条出现几条	SQL 影响几条出现几条	只有一整条（后续可能需要炸开）
数据初始化功能（同步全量数据）	有（支持多库多表同时做）	有（单表）	无

断点续传功能	有（放在 CK）	有（存在 MySQL）	有（本地）
--------	----------	-------------	-------

1.8.2 Maxwell 好处

支持断点续传。

全量初始化同步。

自动根据库名和表名把数据发往 Kafka 的对应主题。

1.8.3 Maxwell 底层原理

MySQL 主从复制。

1.8.4 全量同步速度如何

同步速度慢，全量同步建议采用 Sqoop 或者 DataX。

1.8.5 Maxwell 数据重复问题

同步历史数据时，bootstrap 会扫描所有数据。

同时 maxwell 会监听 binlog 变化。

例如：用 bootstrap 同步历史数据库时，历史数据库中新插入一条数据，这时 bootstrap 扫描到，maxwell 进程也监控到了，这时就会出现数据重复问题。

1.9 DolphinScheduler 调度器

1.3.9 版本，支持邮件、企业微信。

2.0.3 版本，支持的报警信息更全一些，配置更容易。

3.0.0 以上版本，支持数据质量监控。

1.9.1 每天集群运行多少指标？

每天跑 100 多个指标，有活动时跑 200 个左右。

1.9.2 任务挂了怎么办？

- （1）运行成功或者失败都会发邮件、发钉钉、集成自动打电话。
- （2）最主要的解决方案就是，看日志，解决问题。
- （3）报警网站睿象云，<http://www.onealert.com/>
- （4）双 11 和 618 活动需要 24 小时值班

1.9.3 DS 挂了怎么办？

看日志报错原因：直接重启，资源不够增加资源在重启。

1.10 Spark Core & SQL

1.10.1 Spark 运行模式

(1) Local：运行在一台机器上。**测试用。**

(2) Standalone：是 Spark 自身的一个调度系统。对集群性能要求非常高时用。国内很少使用。

(3) Yarn：采用 Hadoop 的资源调度器。国内大量使用。

Yarn-client 模式：Driver 运行在 Client 上（不在 AM 里）

Yarn-cluster 模式：Driver 在 AM 上

(4) Mesos：国内很少使用。

(5) K8S：趋势，但是目前不成熟，需要的配置信息太多。

1.10.2 Spark 常用端口号

(1) 4040 spark-shell 任务端口

(2) 7077 内部通讯端口。类比 Hadoop 的 8020/9000

(3) 8080 查看任务执行情况端口。类比 Hadoop 的 8088

(4) 18080 历史服务器。类比 Hadoop 的 19888

注意：由于 Spark 只负责计算，所有并没有 Hadoop 中存储数据的端口 9870/50070。

1.10.3 RDD 五大属性



RDD特性



* Internally, each RDD is characterized by five main properties:

- * - A list of partitions
- * - A function for computing each split
- * - A list of dependencies on other RDDs
- * - Optionally, a Partitioner for key-value RDDs (e.g. to say that the RDD is hash-partitioned)
- * - Optionally, a list of preferred locations to compute each split on (e.g. block locations for an HDFS file)

(1) 一组分区 (Partition)，即是数据集的基本组成单位，标记数据是哪个分区的；

```
protected def getPartitions: Array[Partition]
```

(2) 一个计算每个分区的函数；

```
def compute(split: Partition, context: TaskContext): Iterator[T]
```

(3) RDD之间的依赖关系；

```
protected def getDependencies: Seq[Dependency[_]] = deps
```

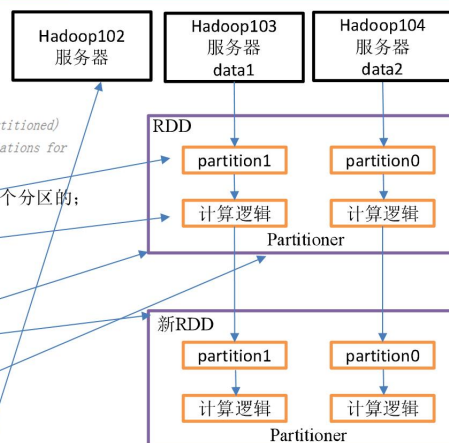
(4) 一个Partitioner，即RDD的分片函数；控制分区的数据流向(键值对)

```
val partitioner: scala.Option[org.apache.spark.Partitioner]
```

(5) 一个列表，存储存取每个Partition的优先位置 (preferred location)。

移动数据不如移动计算，除非资源不够。

```
protected def getPreferredLocations(split: Partition): scala.Seq[String]
```



让天下没有难学的技术

1.10.4 RDD 弹性体现在哪里

主要表现为存储弹性、计算弹性、任务 (Task、Stage) 弹性、数据位置弹性，具体如下：

- (1) 自动进行内存和磁盘切换
- (2) 基于 lineage 的高效容错
- (3) Task 如果失败会特定次数的重试
- (4) Stage 如果失败会自动进行特定次数的重试，而且只会只计算失败的分片
- (5) Checkpoint 【每次对 RDD 操作都会产生新的 RDD，如果链条比较长，计算比较笨重，就把数据放在硬盘中】和 persist 【内存或磁盘中对数据进行复用】(检查点、持久化)
- (6) 数据调度弹性：DAG Task 和资源管理无关
- (7) 数据分片的高度弹性 repartition

1.10.5 Spark 的转换算子 (8 个)

1) 单 Value

- (1) map
- (2) mapPartitions
- (3) mapPartitionsWithIndex
- (4) flatMap

- (5) groupBy
- (6) filter
- (7) distinct
- (8) coalesce
- (9) repartition
- (10) sortBy

2) 双 value

- (1) intersection
- (2) union
- (3) subtract
- (4) zip

3) Key-Value

- (1) partitionBy
- (2) reduceByKey
- (3) groupByKey
- (4) sortByKey
- (5) mapValues
- (6) join

1.10.6 Spark 的行动算子 (5 个)

- (1) reduce
- (2) collect
- (3) count
- (4) first
- (5) take
- (6) save
- (7) foreach

1.10.7 map 和 mapPartitions 区别

- (1) map: 每次处理一条数据

(2) mapPartitions: 每次处理一个分区数据

1.10.8 Repartition 和 Coalesce 区别

1) 关系:

两者都是用来改变 RDD 的 partition 数量的, repartition 底层调用的就是 coalesce 方法: coalesce (numPartitions, shuffle = true)。

2) 区别:

repartition 一定会发生 Shuffle, coalesce 根据传入的参数来判断是否发生 Shuffle。

一般情况下增大 rdd 的 partition 数量使用 repartition, 减少 partition 数量时使用 coalesce。

1.10.9 reduceByKey 与 groupByKey 的区别

reduceByKey: 具有预聚合操作。

groupByKey: 没有预聚合。

在不影响业务逻辑的前提下, 优先采用 reduceByKey。

1.10.10 Spark 中的血缘

宽依赖和窄依赖。有 Shuffle 的是宽依赖。

1.10.11 Spark 任务的划分

(1) Application: 初始化一个 SparkContext 即生成一个 Application;

(2) Job: 一个 Action 算子就会生成一个 Job;

(3) Stage: Stage 等于宽依赖的个数加 1;

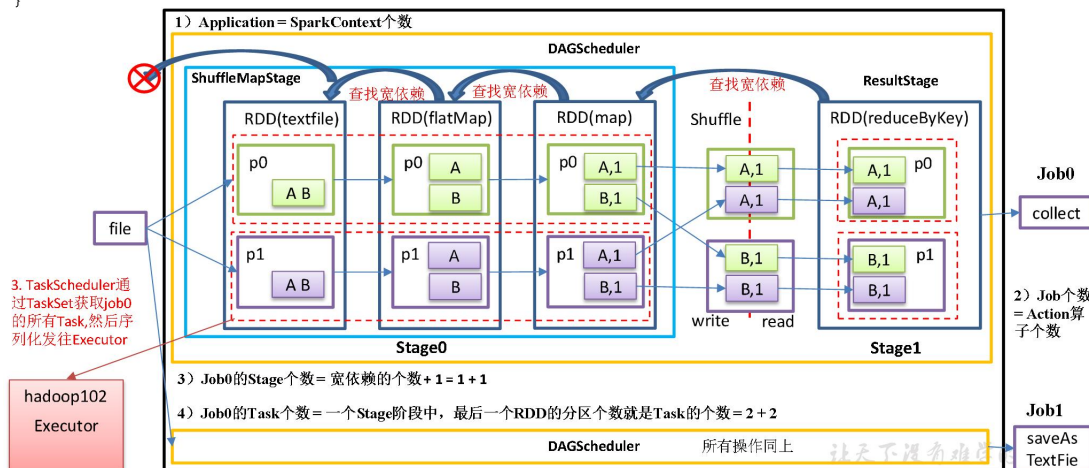
(4) Task: 一个 Stage 阶段中, 最后一个 RDD 的分区个数就是 Task 的个数。



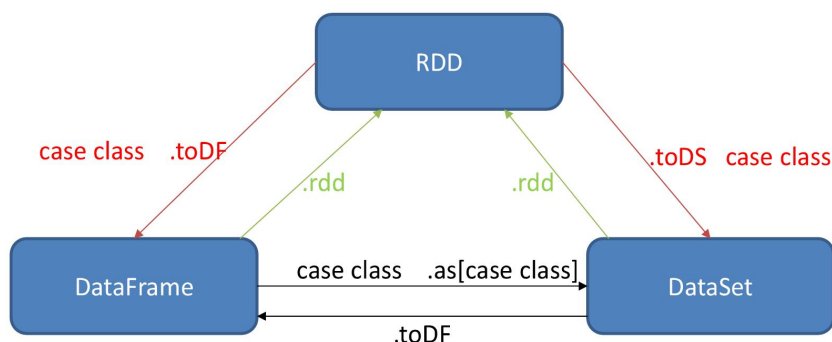
Stage任务划分

```
def main(args: Array[String]): Unit = {
    val sc: SparkContext = new SparkContext(new SparkConf().setAppName("SparkCoreTest").setMaster("local[*]"))
    val resultRDD = sc.textFile("input").flatMap(_._split(" ")).map(_._1).reduceByKey(_+_ )
    resultRDD.collect()
    resultRDD.saveAsTextFile("output")
}
```

1. 执行main方法->初始化sc->执行到Action算子1
2. DAGScheduler对job0切分Stage, Stage产生Task



1.10.12 SparkSQL 中 RDD、DataFrame、DataSet 三者的转换及区别



让天下没有难学的技术

DataFrame 和 DataSet 的区别：前者是 row 类型

RDD 和 DataSet 及 DataSet 的区别：前者没有字段和表信息

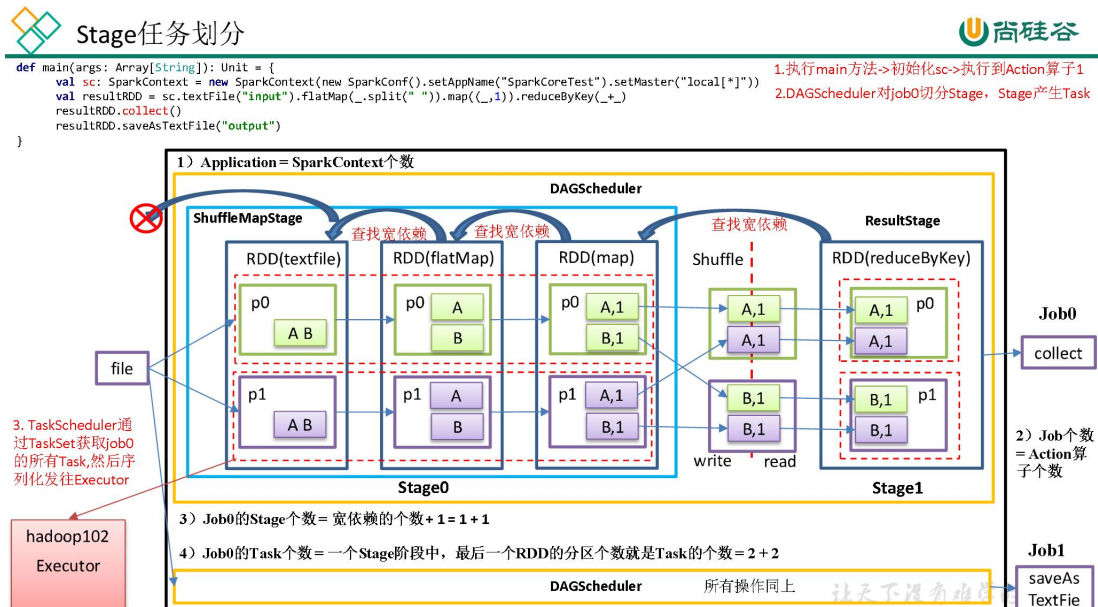
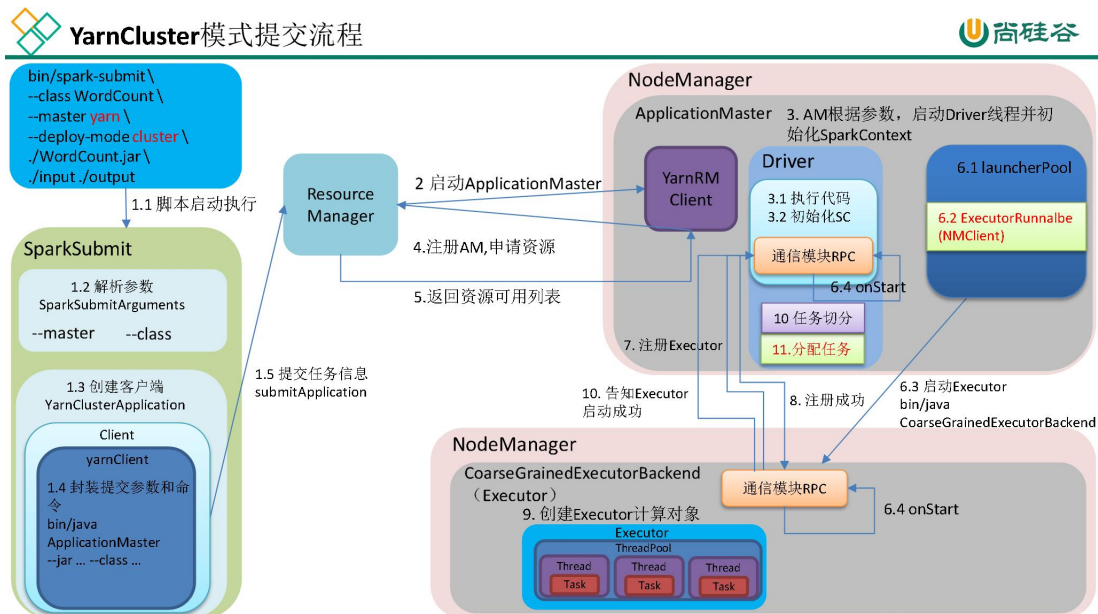
1.10.13 Hive on Spark 和 Spark on Hive 区别

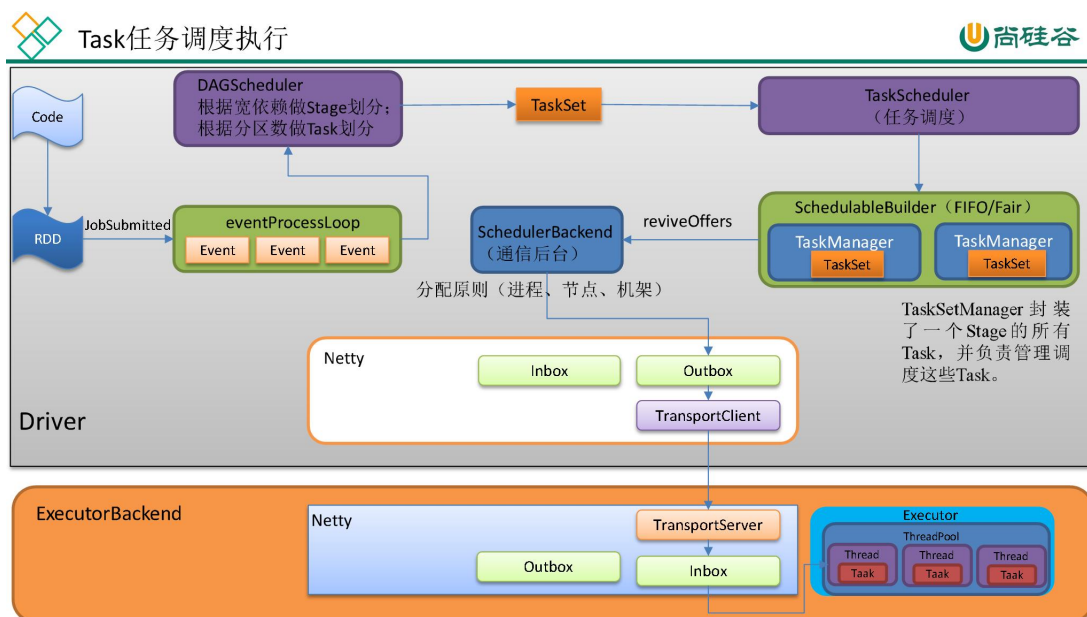
	元数据	执行引擎	语法	生态
Hive on Spark	MySQL	rdd	HQL	更加完善
Spark on Hive (Spark SQL)	MySQL	df ds	Spark SQL	有欠缺（权限管理、元数据管理）

内置 Hive	derby			
外置 Hive	MySQL			

1.10.14 Spark 内核源码（重点）

1) 提交流程（重点）



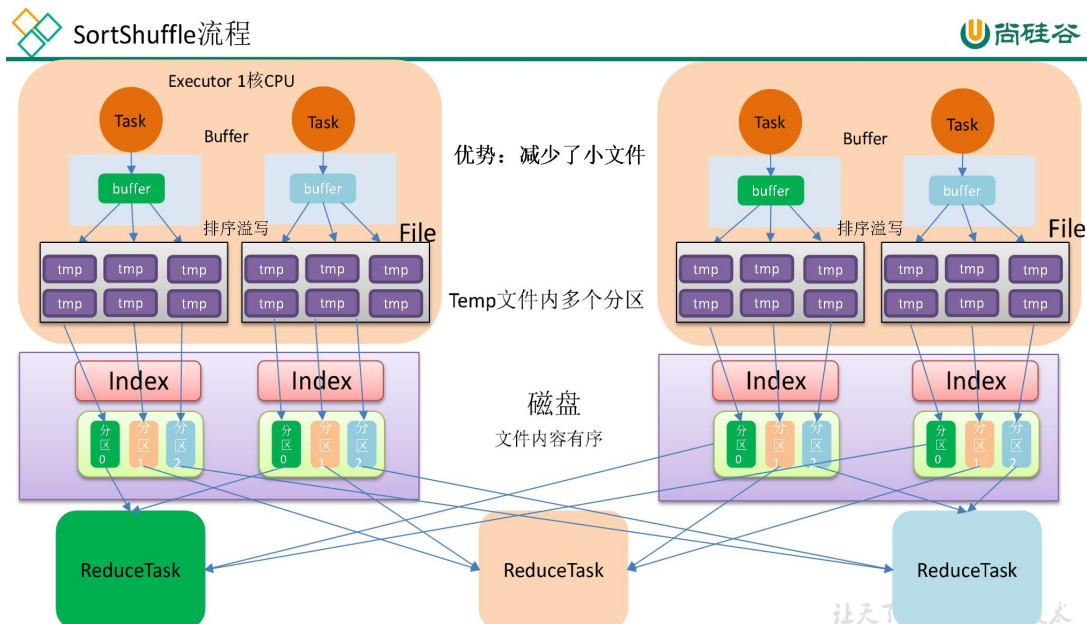


2) Shuffle 流程（重点）

(1) SortShuffle: 减少了小文件。

中间落盘应该是本地磁盘

生成的文件数 = Task 数量*2



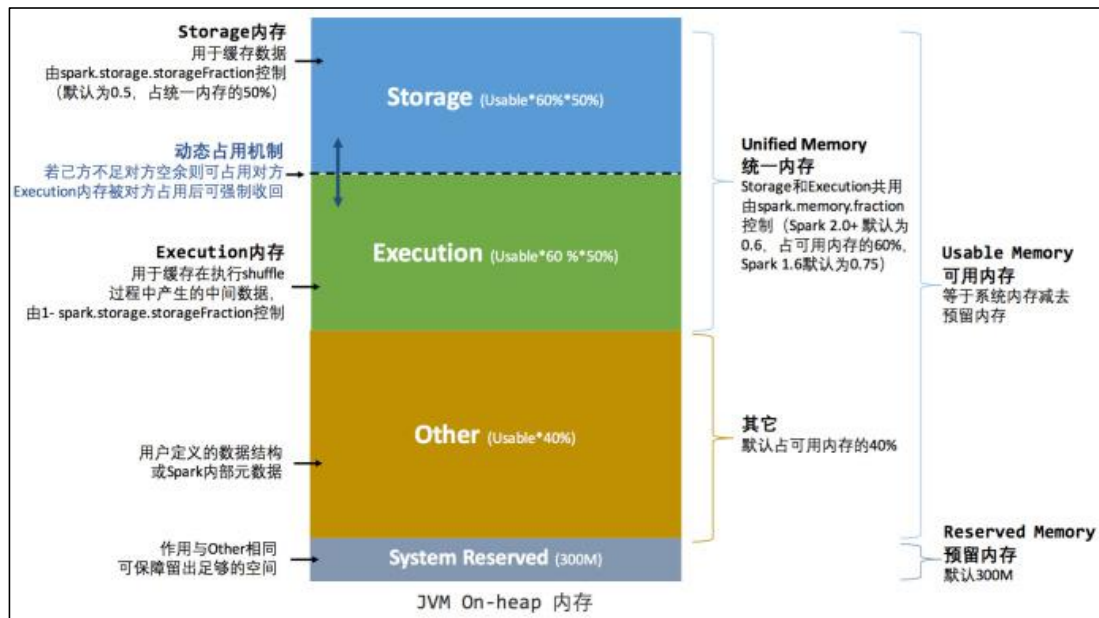
在溢写磁盘前，先根据 **key** 进行排序，排序过后的数据，会分批写入到磁盘文件中。默认批次为 **10000** 条，数据会以每批一万条写入到磁盘文件。写入磁盘文件通过缓冲区溢写的方式，每次溢写都会产生一个磁盘文件，也就是说一个 **Task** 过程会产生多个临时文件。最后在每个 **Task** 中，将所有的临时文件合并，这就是 **merge** 过程，此过程将所有临时文件读

取出来，一次写入到最终文件。

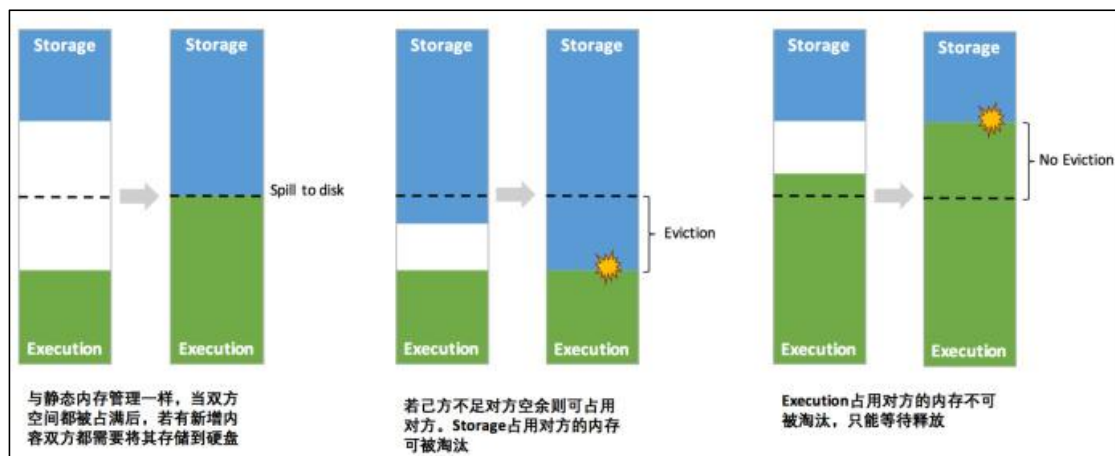
(4) bypassShuffle: 减少了小文件，不排序，效率高。在不需要排序的场景使用。

1.10.15 Spark 统一内存模型

1) 统一内存管理的堆内存结构如下图



2) 统一内存管理的动态占用机制如下图



1.10.16 Spark 为什么比 MR 快?

1) 内存&硬盘

(1) MR 在 Map 阶段会在溢写阶段将中间结果频繁的写入磁盘，在 Reduce 阶段再从磁盘拉取数据。频繁的磁盘 IO 消耗大量时间。

(2) Spark 不需要将计算的中间结果写入磁盘。这得益于 Spark 的 RDD，在各个 RDD 的分区中，各自处理自己的中间结果即可。在迭代计算时，这一优势更为明显。

2) Spark DAG 任务划分减少了不必要的 Shuffle

(1) 对 MR 来说，每一个 Job 的结果都会落地到磁盘。后续依赖于次 Job 结果的 Job，会从磁盘中读取数据再进行计算。

(2) 对于 Spark 来说，每一个 Job 的结果都可以保存到内存中，供后续 Job 使用。配合 Spark 的缓存机制，大大的减少了不必要的 Shuffle。

3) 资源申请粒度：进程&线程

开启和调度进程的代价一般情况下大于线程的代价。

(1) MR 任务以进程的方式运行在 Yarn 集群中。N 个 MapTask 就要申请 N 个进程

(2) Spark 的任务是以线程的方式运行在进程中。N 个 MapTask 就要申请 N 个线程。

1.10.17 Spark Shuffle 和 Hadoop Shuffle 区别？

(1) Hadoop 不用等所有的 MapTask 都结束后开启 ReduceTask；Spark 必须等到父 Stage 都完成，才能去 Fetch 数据。

(2) Hadoop 的 Shuffle 是必须排序的，那么不管是 Map 的输出，还是 Reduce 的输出，都是分区内有序的，而 Spark 不要求这一点。

1.10.18 Spark 提交作业参数（重点）

参考答案：

https://blog.csdn.net/gamer_gyt/article/details/79135118

1) 在提交任务时的几个重要参数

executor-cores —— 每个 executor 使用的内核数，默认为 1，官方建议 2-5 个，我们企业是 4 个

num-executors —— 启动 executors 的数量，默认为 2

executor-memory —— executor 内存大小，默认 1G

driver-cores —— driver 使用内核数，默认为 1

driver-memory —— driver 内存大小，默认 512M

2) 给一个提交任务的样式

spark-submit \

```
--master local[5] \
--driver-cores 2 \
--driver-memory 8g \
--executor-cores 4 \
--num-executors 10 \
--executor-memory 8g \
--class PackageName.ClassName XXXX.jar \
--name "Spark Job Name" \
InputPath \
OutputPath
```

1.10.19 Spark 任务使用什么进行提交，JavaEE 界面还是脚本

Shell 脚本。海豚调度器可以通过页面提交 Spark 任务。

1.10.20 请列举会引起 Shuffle 过程的 Spark 算子，并简述功能。

reduceByKey:

groupByKey:

...ByKey:

1.10.21 Spark 操作数据库时，如何减少 Spark 运行中的数据库连接数？

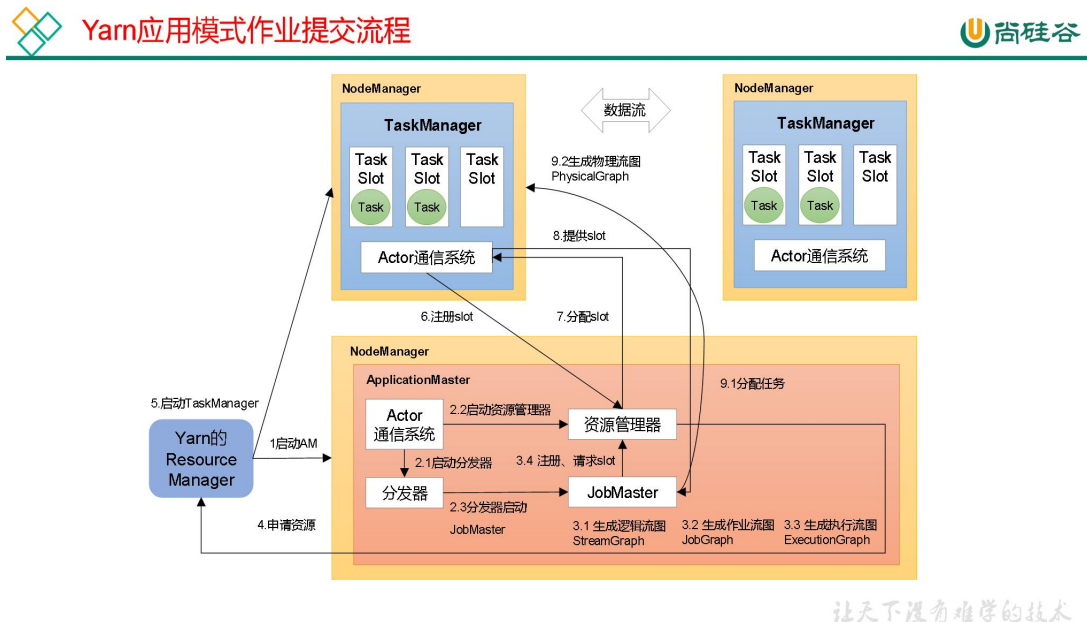
使用 foreachPartition 代替 foreach，在 foreachPartition 内获取数据库的连接。

1.10.22 Spark 数据倾斜

详见 Hive on Spark 数据倾斜讲解。

1.12 Flink

1.12.1 Flink 基础架构组成？



让天下没有难学的技术

Flink 程序在运行时主要有 TaskManager，JobManager，Client 三种角色。

JobManager 是集群的老大，负责接收 Flink Job，协调检查点，Failover 故障恢复等，同时管理 TaskManager。包含：Dispatcher、ResourceManager、JobMaster。

TaskManager 是执行计算的节点，每个 TaskManager 负责管理其所在节点上的资源信息，如内存、磁盘、网络。内部划分 slot 隔离内存，不隔离 cpu。同一个 slot 共享组的不同算子的 subtask 可以共享 slot。

Client 是 Flink 程序提交的客户端，将 Flink Job 提交给 JobManager。

1.12.2 Flink 和 Spark Streaming 的区别？

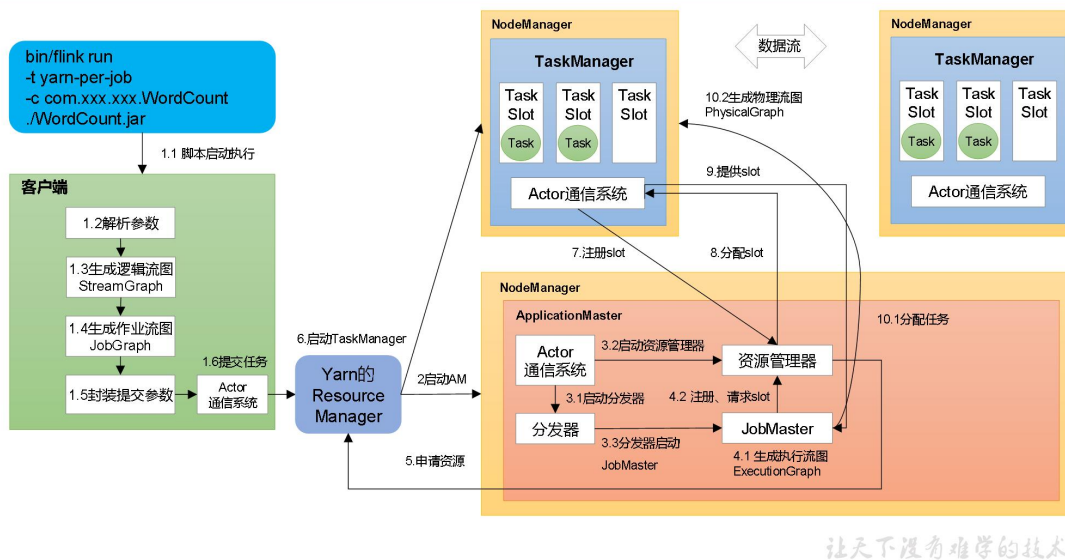
	Flink	Spark Streaming
计算模型	流计算	微批次
时间语义	三种	没有，处理时间
乱序	有	没有
窗口	多、灵活	少、不灵活（窗口长度必须是 批次的整数倍）
checkpoint	异步分界线快照	弱
状态	有，多	没有（updatestatebykey）

流式 sql	有	没有
--------	---	----

1.12.3 Flink 提交作业流程及核心概念

1) Flink 提交流程 (Yarn-Per-Job)

Yarn单作业模式作业提交流程



2) 算子链路: Operator Chain

Flink 自动做的优化, 要求 One-to-one, 并行度相同。

代码 disableOperatorChaining() 禁用算子链。

3) Graph 生成与传递

	在哪里生成	传递给谁	做了什么事
逻辑流图 StreamGraph	Client	Client	最初的 DAG 图
作业流图 JobGraph	Client	JobManager	算子链路优化
执行流图 ExecutionGraph	JobManager	JobManager	并行度的细化
物理流图			

4) Task、Subtask 的区别

Subtask: 算子的一个并行实例。

Task: Subtask 运行起来之后, 就叫 Task。

5) 并行度和 Slot 的关系

Slot 是静态的概念, 是指 TaskMangaer 具有的并发执行能力。

并行度是动态的概念，指程序运行时实际使用的并发能力。

设置合适的并行度能提高运算效率，太多太少都不合适。

6) Slot 共享组了解吗，如何独享 Slot 插槽

默认共享组时 default，同一共享组的 task 可以共享 Slot。

通过 `slotSharingGroup()` 设置共享组。

1.12.4 Flink 的部署模式及区别？

1) **Local**: 本地模式，Flink 作业在单个 JVM 进程中运行，适用于测试阶段

2) **Standalone**: Flink 作业在一个专门的 Flink 集群上运行，独立模式不依赖于其他集群管理器（Yarn 或者 Kubernetes）

3) **Yarn**:

Per-job: 独享资源，代码解析在 Client

Application: 独享资源，代码解析在 JobMaster

Session: 共享资源，一套集群多个 job

4) **K8s**: 支持云原生，未来的趋势

5) **Mesos**: 国外使用，仅作了解

1.12.5 Flink 任务的并行度优先级设置？资源一般如何配置？

设置并行度有多种方式，优先级：算子 > 全局 Env > 提交命令行 > 配置文件

1) 并行度根据任务设置：

(1) 常规任务：Source，Transform，Sink 算子都与 Kafka 分区保持一致

(2) 计算偏大任务：Source，Sink 算子与 Kafka 分区保持一致，Transform 算子可设置成 2 的 n 次方，64，128...

2) 资源设置：通用经验 1CU = 1CPU + 4G 内存

Taskmanager 的 Slot 数：1 拖 1（独享资源）、1 拖 N（节省资源，减少网络传输）

TaskManager 的内存数：4~8G

TaskManager 的 CPU：Flink 默认一个 Slot 分配一个 CPU

JobManager 的内存：2~4G

JobManager 的 CPU：默认是 1

3) 资源是否足够：

资源设置，然后压测，看每个并行度处理上限，是否会出现反压

例如：每个并行度处理 5000/s，开始出现反压，比如我们设置三个并行度，我们程序处理上限 15000/s

1.12.6 Flink 的三种时间语义

事件时间 Event Time：是事件创建的时间。数据本身携带的时间。

进入时间 Ingestion Time：是数据进入 Flink 的时间。

处理时间 Processing Time：是每一个执行基于时间操作的算子的本地系统时间，与机器相关，默认的时间属性就是 Processing Time。

1.12.7 你对 Watermark 的认识

水位线是 Flink 流处理中保证结果正确性的核心机制，它往往会跟窗口一起配合，完成对乱序数据的正确处理。

- 水位线是插入到数据流中的一个标记，可以认为是一个特殊的数据
- 水位线主要的内容是一个时间戳，用来表示当前事件时间的进展
- 水位线是基于数据的时间戳生成的
- 水位线的时间戳必须单调递增，以确保任务的事件时间时钟一直向前推进
- 水位线可以通过设置延迟，来保证正确处理乱序数据
- 一个水位线 $Watermark(t)$ ，表示在当前流中事件时间已经达到了时间戳 t ，这代表 t 之前的所有数据都到齐了，之后流中不会出现时间戳 $t' \leq t$ 的数据

1.12.8 Watermark 多并行度下的传递、生成原理

1) 分类：

间歇性：来一条数据，更新一次 Watermark。

周期性：固定周期更新 Watermark。

官方提供的 API 是基于周期的，默认 200ms，因为间歇性会给系统带来压力。

2) 生成原理：

$Watermark = \text{当前最大事件时间} - \text{乱序时间} - 1ms$

3) 传递：

Watermark 是一条携带时间戳的特殊数据，从代码指定生成的位置，插入到流里面。

一对多：广播。

多对一：取最小。

多对多：拆分来看，其实就是上面两种的结合。

1.12.9 Flink 怎么处理乱序和迟到数据？

在 Apache Flink 中，迟到时间（lateness）和乱序时间（out-of-orderness）是两个与处理时间和事件时间相关的概念。它们在流处理过程中，尤其是在处理不按事件时间排序的数据时非常重要。

（1）迟到时间（lateness）：迟到时间可以影响窗口，在窗口计算完成后，仍然可以接收迟到的数据

迟到时间是指事件到达流处理系统的延迟时间，即事件的实际接收时间与其事件时间的差值。在某些场景下，由于网络延迟、系统故障等原因，事件可能会延迟到达。为了处理这些迟到的事件，Flink 提供了一种机制，允许在窗口计算完成后仍然接受迟到的数据。设置迟到时间后，Flink 会在窗口关闭之后再等待一段时间，以便接收并处理这些迟到的事件。

设置迟到时间的方法如下：

在定义窗口时，使用`allowedLateness`方法设置迟到时间。例如，设置迟到时间为 10 分钟：

```
```java
DataStream<T> input = ...;
input
 .keyBy(<key selector>)
 .window(<window assigner>)
 .allowedLateness(Time.minutes(10))
 .<window function>;
```
```

（2）乱序时间（out-of-orderness）

乱序时间是通过影响水印来影响数据的摄入，它表示的是数据的混乱程度。

乱序时间是指事件在流中不按照事件时间的顺序到达。在某些场景下，由于网络延迟或数据源的特性，事件可能会乱序到达。

Flink 提供了处理乱序事件的方法，即水位线（watermark）。

水位线是一种表示事件时间进展的机制，它告诉系统当前处理到哪个事件时间。

当水位线到达某个值时，说明所有时间戳小于该值的事件都已经处理完成。

为了处理乱序事件，可以为水位线设置一个固定的延迟。

设置乱序时间的方法如下：

在定义数据源时，使用`assignTimestampsAndWatermarks`方法设置水位线策略。例如，设置水位线延迟为 5 秒：

```
```java
DataStream<T> input = env.addSource(<source>);
input
 .assignTimestampsAndWatermarks(
 WatermarkStrategy
 .<T>forBoundedOutOfOrderness(Duration.ofSeconds(5))
 .withTimestampAssigner(<timestamp assigner>))
 .<other operations>;
```
```

1.12.10 说说 Flink 中的窗口（分类、生命周期、触发、划分）

1) 窗口分类：

Keyed Window 和 Non-keyed Window

基于时间：滚动、滑动、会话。

基于数量：滚动、滑动。

2) Window 口的 4 个相关重要组件：

assigner（分配器）：如何将元素分配给窗口。

function（计算函数）：为窗口定义的计算。其实是一个计算函数，完成窗口内容的计算。

trigger（触发器）：在什么条件下触发窗口的计算。

可以使用自定义触发器，解决事件时间，没有数据到达，窗口不触发计算问题，还可以使用持续性触发器，实现一个窗口多次触发输出结果，详细看连接

问题展示：

https://www.bilibili.com/video/BV1Gv4y1H7F8/?spm_id_from=333.999.0.0&vd_source=891aa1a363111d4914eb12ace2e039af

问题解决：

https://www.bilibili.com/video/BV1mM411N7uP/?spm_id_from=333.999.0.0&vd_source=891aa1a363111d4914eb12ace2e039af

evictor（退出器）：定义从窗口中移除数据。

3) 窗口的划分：如，基于事件时间的滚动窗口

Start = 按照数据的事件时间向下取窗口长度的整数倍。

end = start + size

比如开了一个 10s 的滚动窗口，第一条数据是 857s，那么它属于[850s,860s)。

4) 窗口的创建：当属于某个窗口的第一个元素到达，Flink 就会创建一个窗口，并且放入单例集合

5) 窗口的销毁：时间进展 >= 窗口最大时间戳 + 窗口允许延迟时间

(Flink 保证只删除基于时间的窗口，而不能删除其他类型的窗口，例如全局窗口)。

6) 窗口为什么左闭右开：属于窗口的最大时间戳 = end - 1ms

7) 窗口什么时候触发：如基于事件时间的窗口 watermark >= end - 1ms

1.12.11 Flink 的 keyby 怎么实现的分区？分区、分组的区别是什么？

分组和分区在 Flink 中具有不同的含义和作用：

分区：分区 (Partitioning) 是将数据流划分为多个子集，这些子集可以在不同的任务实例上进行处理，以实现数据的并行处理。

数据具体去往哪个分区，是通过指定的 key 值先进行一次 hash 再进行一次 murmurHash，通过上述计算得到的值再与并行度进行相应的计算得到。

分组：分组 (Grouping) 是将具有相同键值的数据元素归类到一起，以便进行后续操作 (如聚合、窗口计算等)。key 值相同的数据将进入同一个分组中。

注意：数据如果具有相同的 key 将一定去往同一个分组和分区，但是同一分区中的数据不一定属于同一组。

1.12.12 Flink 的 Interval Join 的实现原理？Join 不上的怎么办？

底层调用的是 keyby + connect，处理逻辑：

- (1) 判断是否迟到 (迟到就不处理了，直接 return)
- (2) 每条流都存了一个 Map 类型的状态 (key 是时间戳，value 是 List 存数据)
- (3) 任一条流，来了一条数据，遍历对方的 map 状态，能匹配上就发往 join 方法
- (4) 使用定时器，超过有效时间范围，会删除对应 Map 中的数据 (不是 clear，是 remove)

Interval join 不会处理 join 不上的数据，如果需要没 join 上的数据，可以用 coGroup+join 算子实现，或者直接使用 flinksql 里的 left join 或 right join 语法。

1.12.13 介绍一下 Flink 的状态编程、状态机制？

- (1) 算子状态：作用范围是算子，算子的多个并行实例各自维护一个状态
- (2) 键控状态：每个分组维护一个状态
- (3) 状态后端：两件事=》 本地状态存哪里、checkpoint 存哪里

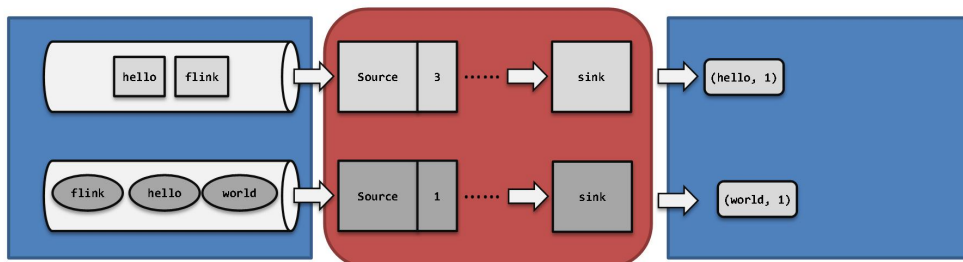
1.13 版本之前

| | 本地状态 | Checkpoint |
|----|-----------------|---------------|
| 内存 | TaskManager 的内存 | JobManager 内存 |

| | | |
|---------------|-----------------|------|
| 文件 | TaskManager 的内存 | HDFS |
| RocksDB | RocksDB | HDFS |
| 1.13 版本之后 | | |
| | 本地状态 | |
| Hashmap() | TaskManager 的内存 | |
| RocksDB | RocksDB | |
| Checkpoint 存储 | 参数指定 | |

1.12.14 Flink 如何实现端到端一致性?

端到端精确一次



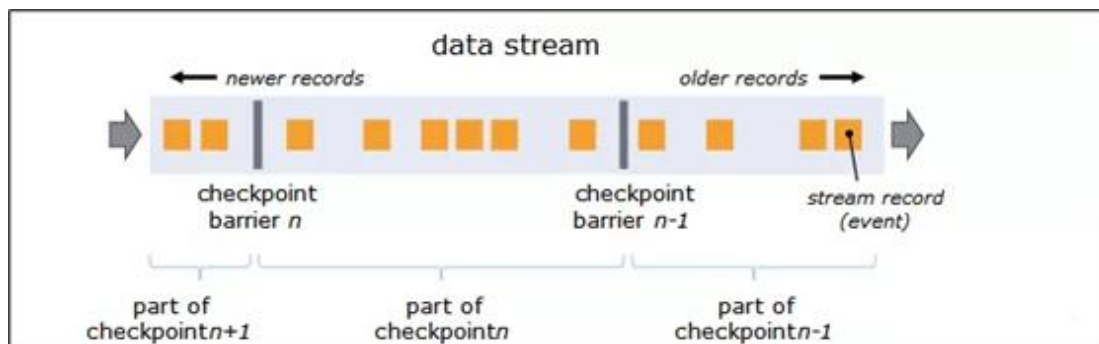
输入端：数据可重放
如Kafka，可重置读取数据偏移量

Flink处理：开启checkpoint且精准一次
① barrier对齐精准一次
② 非barrier对齐精准一次

输出端：幂等 或 事务
① 幂等：
利用mysql的主键upsert
hbase的rowkey唯一
② 事务（外部系统提供）：
两阶段提交写kafka
两阶段提交写MySQL（XA事务）

让天下没有难学的技术

1.12.15 Flink 分布式快照的原理是什么



barriers 在数据流源处被注入并行数据流中。快照 n 的 barriers 被插入的位置（我们称之

为 S_n) 是快照所包含的数据在数据源中最大位置。

例如, 在 Kafka 中, 此位置将是分区中最后一条记录的偏移量。将该位置 S_n 报告给 checkpoint 协调器 (Flink 的 JobManager)。

然后 barriers 向下游流动。当一个中间操作算子从其所有输入流中收到快照 n 的 barriers 时, 它会为快照 n 发出 barriers 进入其所有输出流中。

一旦 Sink 操作算子 (流式 DAG 的末端) 从其所有输入流接收到 barriers n , 它就向 Checkpoint 协调器确认快照 n 完成。

在所有 Sink 确认快照后, 意味快照着已完成。一旦完成快照 n , Job 将永远不再向数据源请求 S_n 之前的记录, 因为此时这些记录 (及其后续记录) 将已经通过整个数据流拓扑, 也即是已经被处理结束。

1.12.16 Checkpoint 的参数怎么设置的?

(1) 间隔: 兼顾性能和延迟, 一般任务设置分钟级 (1~5min), 要求延迟低的设置秒级

(2) Task 重启策略 (Failover):

固定延迟重启策略: 重试几次、每次间隔多久。

失败率重启策略: 重试次数、重试区间、重试间隔。

无重启策略: 一般在开发测试时使用。

Fallback 重启策略: 默认固定延迟重启策略。

1.12.17 Flink 内存模型（重点）



Flink内存模型(TaskManager)



Flink 1.10 对 TaskManager 内存模型做了较大改动

Flink使用了堆上内存和堆外内存

- Flink框架内存使用了堆上内存和堆外内存，不计入slot资源
- Task执行的内存使用了堆上内存和堆外内存
- 网络缓冲内存：网络数据交换所使用的堆外内存大小，如网络数据交换缓冲区

框架堆外内存、Task堆外内存、网络缓冲内存，都在堆外的直接内存里

- 管理内存：Flink管理的堆外内存。用于管理排序、哈希表、缓存中间结果及 RocksDB State Backend 的本地内存
- JVM特有内存：JVM本身占用的内存，包括元空间和执行开销

Flink使用内存=框架堆内存和堆外内存+Task堆内存和堆外内存+网络缓冲内存+管理内存

进程内存=Flink内存+JVM特有内存

让天下没有难学的技术

1.12.18 Flink 常见的维表 Join 方案

- (1) 预加载：open()方法，查询维表，存储下来 ==》 定时查询
- (2) 热存储：存在外部系统 Redis、HBase 等
- (3) 广播维表
- (4) Lookup Join：外部存储，connector 创建，SQL 用法

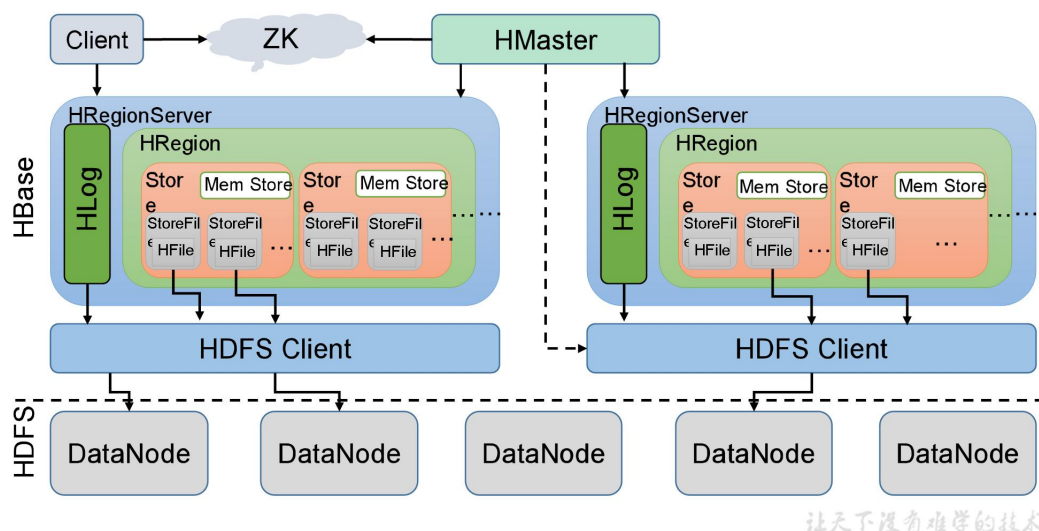
1.12.19 FlinkCDC 锁表问题

- (1) FlinkCDC 1.x 同步历史数据会锁表
设置参数不加锁，但只能保证至少一次。
- (2) 2.x 实现了无锁算法，同步历史数据的时候不会锁表
2.x 在全量同步阶段可以多并行子任务同步，在增量阶段只能单并行子任务同步

1.13 HBase

1.13.1 HBase 存储结构

 HBase架构图



让天下没有难学的技术

架构角色：

1) Master

实现类为 **HMaster**，负责监控集群中所有的 RegionServer 实例。主要作用如下：

- (1) 管理元数据表 `hbase:meta`，接收用户对表格创建修改删除的命令并执行
- (2) 监控 region 是否需要负载均衡，故障转移和 region 的拆分。

通过启动多个后台线程监控实现上述功能：

①LoadBalancer 负载均衡器

周期性监控 region 分布在 regionServer 上面是否均衡，由参数 `hbase.balancer.period` 控制周期时间，默认 5 分钟。

②CatalogJanitor 元数据管理器

定期检查和清理 HBase: meta 中的数据。meta 表内容在进阶中介绍。

③MasterProcWAL Master 预写日志处理器

把 Master 需要执行的任务记录到预写日志 WAL 中，如果 Master 宕机，让 backupMaster 读取日志继续干。

2) Region Server

Region Server 实现类为 **HRegionServer**，主要作用如下：

- (1) 负责数据 cell 的处理，例如写入数据 put，查询数据 get 等
- (2) 拆分合并 Region 的实际执行者，有 Master 监控，有 regionServer 执行。

3) Zookeeper

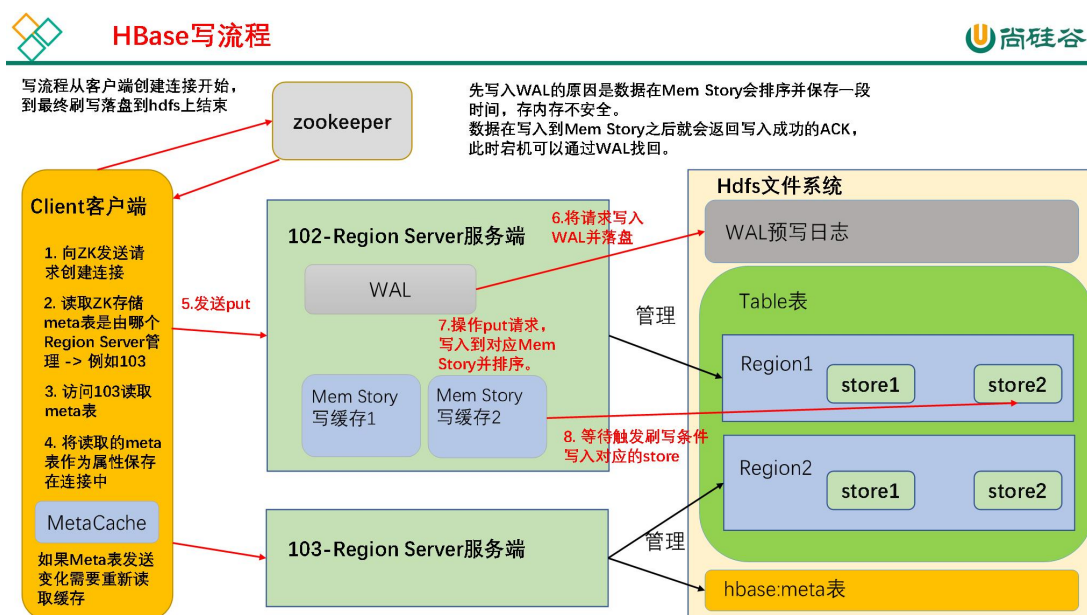
HBase 通过 Zookeeper 来做 Master 的高可用、记录 RegionServer 的部署信息、并且存储有 meta 表的位置信息。

HBase 对于数据的读写操作时直接访问 Zookeeper 的，在 2.3 版本推出 Master Registry 模式，客户端可以直接访问 Master。使用此功能，会加大对 Master 的压力，减轻对 Zookeeper 的压力。

4) HDFS

HDFS 为 HBase 提供最终的底层数据存储服务，同时为 HBase 提供高容错的支持。

1.13.2 HBase 的写流程



写流程:

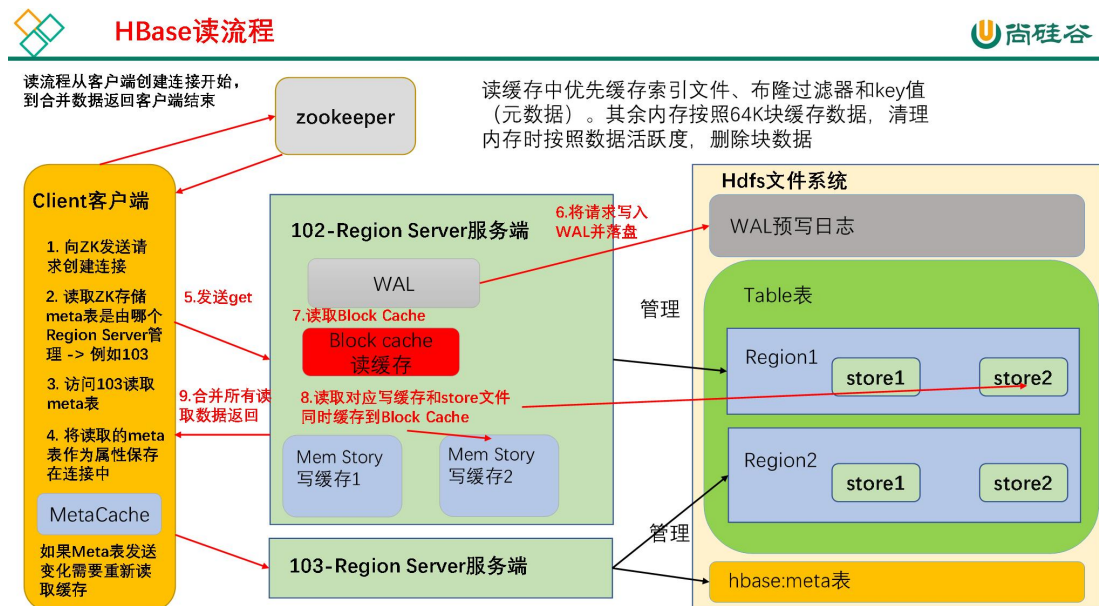
写流程顺序正如 API 编写顺序，首先创建 HBase 的重量级连接

- (1) 读取本地缓存中的 Meta 表信息；（第一次启动客户端为空）
- (2) 向 ZK 发起读取 Meta 表所在位置的请求；
- (3) ZK 正常返回 Meta 表所在位置；
- (4) 向 Meta 表所在位置的 RegionServer 发起请求读取 Meta 表信息；
- (5) 读取到 Meta 表信息并将其缓存在本地；

(6) 向待写入表发起写数据请求；

(7) 先写 WAL，再写 MemStore,并向客户端返回写入数据成功。

1.13.3 HBase 的读流程



创建连接同写流程。

(1) 读取本地缓存中的 Meta 表信息；（第一次启动客户端为空）

(2) 向 ZK 发起读取 Meta 表所在位置的请求；

(3) ZK 正常返回 Meta 表所在位置；

(4) 向 Meta 表所在位置的 RegionServer 发起请求读取 Meta 表信息；

(5) 读取到 Meta 表信息并将其缓存在本地；

(6) MemStore、StoreFile、BlockCache

同时构建 MemStore 与 StoreFile 的扫描器，

MemStore：正常读

StoreFile：

根据索引确定待读取文件；

再根据 BlockCache 确定读取文件；

(7) 合并多个位置读取到的数据,给用户返回最大版本的数据，如果最大版本数据为删除标记，则不给不返回任何数据。

1.13.6 HBase 的合并

Compaction 分为两种，分别是 **Minor Compaction** 和 **Major Compaction**。

1.13.7 RowKey 设计原则

- (1) rowkey 长度原则
- (2) rowkey 散列原则
- (3) rowkey 唯一原则

1.13.8 RowKey 如何设计

1) 使用场景:

大量用户信息保存在 HBase 中。

2) 热点问题:

由于用户的 id 是连续的，批量导入用户数据后，很有可能用户信息都集中在同一个 region 中。如果用户信息频繁访问，很有可能该 region 的节点成为热点。

3) 期望： 通过对 Rowkey 的设计，使用户数据能够分散到多个 region 中。

4) 步骤:

(1) 预分区

通过命令

```
create 'GMALL:DIM_USER_INFO', 'INFO', SPLITS=>['20', '40', '60', '80']
```

把用户信息表(GMALL:DIM_USER_INFO) 分为 5 个 region : [00-20), [20-40), [40-60), [60-80), [80-99]

(2) 写入时反转 ID

把用户 ID 左补零 10 位（根据最大用户数），然后反转顺序。

比如：用户 id 为 1457，反转处理后变为 7541000000；根据前两位分到 region [60-80)，

用户 id 为 1459，反转处理后变为 9541000000；根据前两位分到 region [80-99]

这样连续的用户 ID 反转后由于 Rowkey 开头并不连续，会进入不同的 region 中。

最终达到的效果可以通过 Web UI 进行观察：

| Name(s) | Region Server | ReadRequests
(4,182) | WriteRequests
(297,014) | Storefile Size
(0 B) | Num.Storefiles
(5) | MemSize
(0 B) | Locality | Start Key | End Key |
|----------------------------------|---------------|-------------------------|----------------------------|-------------------------|-----------------------|------------------|----------|-----------|---------|
| 0c7fb8384b979c0b657ad0cd9596b30c | hdp3:16030 | 869 | 59,876 | 0 B | 1 | 0 B | 1.0 | 20 | |
| 6d94f37dc38dc7bee2383ca0cf96aa12 | hdp1:16030 | 883 | 59,960 | 0 B | 1 | 0 B | 1.0 | 20 | 40 |
| ca1047b7c51e81abb9906454c8da4db3 | hdp3:16030 | 832 | 60,072 | 0 B | 1 | 0 B | 1.0 | 40 | 60 |
| 22d0ab48924385808fa629ea70a97522 | hdp2:16030 | 801 | 58,812 | 0 B | 1 | 0 B | 1.0 | 60 | 80 |
| 25db8ef6386fe30ce4e9ef64510d3ea5 | hdp2:16030 | 797 | 58,294 | 0 B | 1 | 0 B | 1.0 | 80 | |

如上图，用户数据会分散到多个分区中。

注意：在用户查询时，也同样根据需要把 ID 进行反转后进行查询。

1.13.9 HBase 二级索引原理

1) 原理

协处理器：协助处理数据，可以在向原始表中写入数据之后向索引表中写入一条索引数据。

2) 种类及用法

(1) 全局 读多写少

单独创建表专门用于存储索引，索引表数据量比原始表小，读取更快速。但是写操作会写两张表的数据，跨 Region，需要多个连接。

(2) 本地 写多读少

将索引数据与原表放在一起 (Region)，加在一起比原表数据量大，读取相对变慢，但是由于在一个 Region，所以写操作两条数据用的是同一个连接。

1.14 Clickhouse

1.14.1 Clickhouse 的优势

快：提供了丰富的表引擎，每个表引擎 都做了尽可能的优化。

为什么快？

- (1) 向量化
- (2) 列式
- (3) 尽可能使用本节点的 内存+cpu，不依赖其他组件，比如 Hadoop
- (4) 提供了 sql 化的语言
- (5) 支持自定义函数
- (6) 提供了丰富的表引擎，引擎都经过了优化

1.14.2 Clickhouse 的引擎

- (1) Log
- (2) Special: Memory、Distributed
- (3) MergeTree: replacingmergetree、summingmergetree

replicatedmergetree

(4) 集成引擎： 外部系统映射，如 MySQL

1.14.3 Flink 写入 Clickhouse 怎么保证一致性？

Clickhouse 没有事务，Flink 写入是至少一次语义。

利用 Clickhouse 的 ReplacingMergeTree 引擎会根据主键去重，但只能保证最终一致性。
查询时加上 final 关键字可以保证查询结果的一致性。

1.14.4 Clickhouse 存储多少数据？几张表？

10 几张宽表，每天平均 10 来 G，存储一年。

需要磁盘 $10G * 365 \text{ 天} * 2 \text{ 副本} / 0.7 = \text{约 } 11T$

1.14.5 Clickhouse 使用本地表还是分布式表

1) 我们用的本地表，2 个副本

2) 分布式表写入存在的问题：

假如现有一个 2 分片的集群，使用 clickhouse 插入分布式表。

(1) 资源消耗问题：在分片 2 的数据写入临时目录中会产生写放大现象，会大量消耗分片节点 1 的 CPU 和磁盘等资源。

(2) 数据准确性和一致性问题：在写入分片 2 的时候，节点 1 或节点 2 的不正常都会导致数据问题。(节点 1 挂了数据丢失、节点 2 挂了或者节点 2 表删了节点 1 会无限制重试，占用资源)。

(3) part 过多问题：每个节点每秒收到一个 Insert Query，N 个节点，分发 N-1 次，一共就是每秒生成 $N * (N-1)$ 个 part 目录。集群 shard 数越多，分发产生的小文件也会越多(如果写本地表就会相对集中些)，最后会导致写入到 MergeTree 的 Part 的数会特别多，最后会拖垮整个文件的系统。

1.14.6 Clickhouse 的物化视图

一种查询结果的持久化，记录了查询语句和对应的查询结果。

优点：查询速度快，要是把物化视图这些规则全部写好，它比原数据查询快了很多，总的行数少了，因为都预计算好了。

缺点：它的本质是一个流式数据的使用场景，是累加式的技术，所以要用历史数据做去

重、去核这样的分析，在物化视图里面是不太好用的。在某些场景的使用也是有限的。而且如果一张表加了好多物化视图，在写这张表的时候，就会消耗很多机器的资源，比如数据带宽占满、存储一下子增加了很多。

1.14.8 Clickhouse 的优化

1) 内存优化

`max_memory_usage`: 单个查询的内存上限，128G 内存的服务器==》 设为 100G

`max_bytes_before_external_group_by`: 设为 一半，50G

`max_bytes_before_external_sort`: 设为一半，50G

2) CPU

`max_concurrent_queries`: 默认 100/s ==> 300/s

3) 存储

SSD 更快

4) 物化视图

5) 写入时攒批，避免写入过快导致 too many parts

1.14.9 Clickhouse 的新特性 Projection

Projection 意指一组列的组合，可以按照与原表不同的排序存储，并且支持聚合函数的查询。ClickHouse Projection 可以看做是一种更加智能的物化视图，它有如下特点：

1) part-level 存储

相比普通物化视图是一张独立的表，Projection 物化的数据就保存在原表的分区目录中，支持明细数据的普通 Projection 和 预聚合 Projection。

2) 无感使用，自动命中

可以对一张 MergeTree 创建多个 Projection，当执行 Select 语句的时候，能根据查询范围，自动匹配最优的 Projection 提供查询加速。如果没有命中 Projection，就直接查询底表。

3) 数据同源、同生共死

因为物化的数据保存在原表的分区，所以数据的更新、合并都是同源的，也就不会出现不一致的情况了。

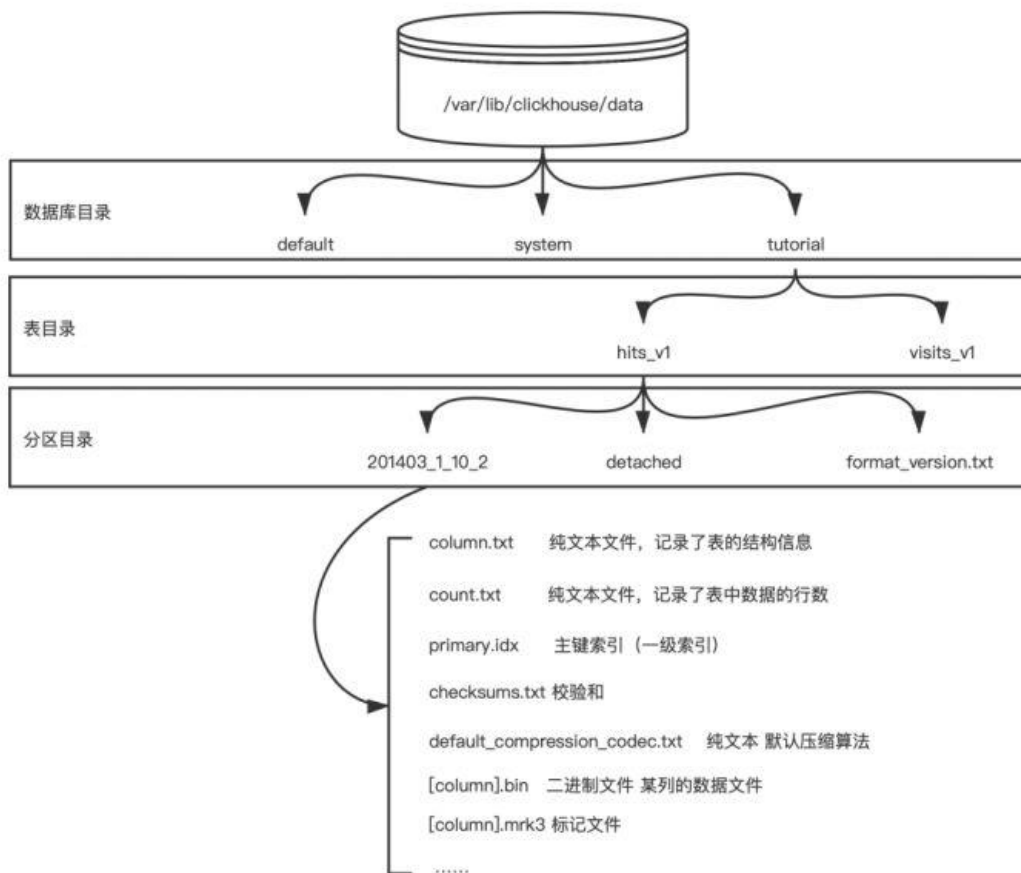
1.14.10 Clickhouse 的索引、底层存储

1) 索引

- (1) 一级索引：稀疏索引（主键索引） 粒度 8192
- (2) 二级索引：跳数索引 minmax、set、bloom_filter 等

2) 底层存储

Clickhouse 默认数据目录在 `/var/lib/clickhouse/data` 目录中。所有的数据库都会在该目录中创建一个子文件夹。下图展示了 Clickhouse 对数据文件的组织。



| | | |
|--------------------|------|-------------------------|
| 202103_1_10_2 | 目录 | 分区目录，由分区+LSM 生成的 |
| detached | 目录 | 通过 DETACH 语句卸载后的表分区存放位置 |
| format_version.txt | 文本文件 | 纯文本，记录存储的格式 |

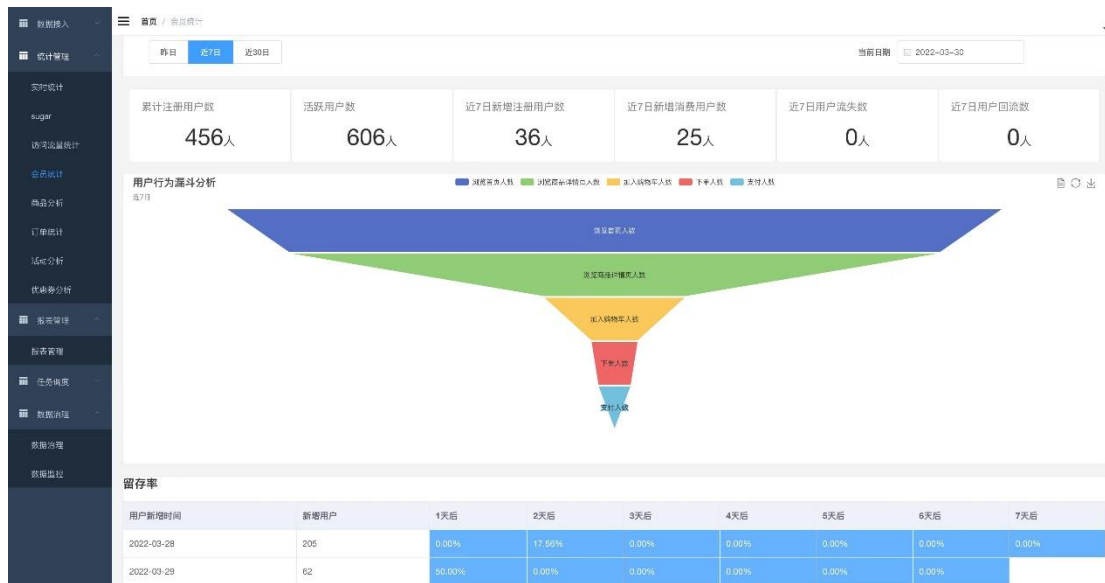
分区目录命名 = 分区 ID_最小数据块编号_最大数据块编号_层级构成。数据块编号从 1 开始自增，新创建的数据块最大和最小编号相同，当发生合并时会将其修改为合并的数据

块编号。同时每次合并都会将层级增加 1。

1.15 可视化报表工具

开源：Echarts（百度）、Kibana、Superset（功能一般）

收费：Tableau（功能强大）、QuickBI（阿里云面对实时）、DataV（阿里云面对实时）、Suga（百度实时）



1.16 Doris

1.17 Sqoop

1) Sqoop 参数

```
/opt/module/sqoop/bin/sqoop import \
--connect \
--username \
--password \
--target-dir \
--delete-target-dir \
--num-mappers \
--fields-terminated-by \
--query "$2" ' and $CONDITIONS;'
```

2) Sqoop 导入导出 Null 存储一致性问题

Hive 中的 Null 在底层是以 “\N” 来存储，而 MySQL 中的 Null 在底层就是 Null，为了保证数据两端的一致性。在导出数据时采用 --input-null-string 和 --input-null-non-string 两个参数。导入数据时采用 --null-string 和 --null-non-string。

3) Sqoop 数据导出一致性问题

场景 1：如 Sqoop 在导出到 MySQL 时，使用 4 个 Map 任务，过程中有 2 个任务失败，那此时 MySQL 中存储了另外两个 Map 任务导入的数据，此时老板正好看到了这个报表数据。而开发工程师发现任务失败后，会调试问题并最终将全部数据正确的导入 MySQL，那后面老板再次看报表数据，发现本次看到的数据与之前的不一致，这在生产环境是不允许的。

官网：<http://sqoop.apache.org/docs/1.4.6/SqoopUserGuide.html>

Since Sqoop breaks down export process into multiple transactions, it is possible that a failed export job may result in partial data being committed to the database. This can further lead to subsequent jobs failing due to insert collisions in some cases, or lead to duplicated data in others. You can overcome this problem by specifying a staging table via the `--staging-table` option which acts as an auxiliary table that is used to stage exported data. The staged data is finally moved to the destination table in a single transaction.

`--staging-table` 方式

```
sqoop export --connect jdbc:mysql://192.168.137.10:3306/user_behavior --username root --password 123456 --table app_cource_study_report --columns watch_video_cnt,complete_video_cnt,dt --fields-terminated-by "\t" --export-dir "/user/hive/warehouse/tmp.db/app_cource_study_analysis_${day}" --staging-table app_cource_study_report_tmp --clear-staging-table --input-null-string '\N'
```

4) Sqoop 底层运行的任务是什么

只有 Map 阶段，没有 Reduce 阶段的任务。默认是 4 个 MapTask。

5) Sqoop 一天导入多少数据

100 万日活=》10 万订单，1 人 10 条，每天 1g 左右业务数据

Sqoop 每天将 1G 的数据量导入到数仓。

6) Sqoop 数据导出的时候一次执行多长时间

每天晚上 00:10 开始执行，Sqoop 任务一般情况 20-30 分钟的都有。取决于数据量（11.11，6.18 等活动在 1 个小时左右）。

7) Sqoop 在导出数据的时候数据倾斜

Sqoop 参数撇嘴：`split-by`：按照自增主键来切分表的工作单元。

`num-mappers`：启动 N 个 map 来并行导入数据，默认 4 个；

8) Sqoop 数据导出 Parquet（项目中遇到的问题）

Ads 层数据用 Sqoop 往 MySql 中导入数据的时候，如果用了 orc（Parquet）不能导入，需转化成 text 格式

（1）创建临时表，把 Parquet 中表数据导入到临时表，把临时表导出到目标表用于可视

化

(2) ads 层建表的时候就不要建 Parquet 表

1.18 Azkaban

1) 集群每天跑多少指标?

每天跑 100 多个指标, 有活动时跑 200 个左右。

2) 任务挂了怎么办?

(1) 运行成功或者失败都会发邮件、发钉钉、集成自动打电话 (项目中遇到的问题)

(2) 最主要的解决方案就是重新跑。

(3) 报警网站 <http://www.onealert.com/>

1.19 JavaSE

1.19.1 什么是多线程&多线程的优点

多线程是指程序中包含多个执行流, 即一个程序中可以同时运行多个不同的线程来执行不同的任务。

优点: 可以提高 cpu 的利用率。多线程中, 一个线程必须等待的时候, cpu 可以运行其它的线程而不是等待, 这样大大提高了程序的效率。

1.19.2 如何创建多线程

Java 3 种常见创建多线程的方式

- (1) 继承 Thread 类, 重 run()方法
- (2) 实现 Runnable 接口, 重写 run()方法
- (3) 通过创建线程池实现

1.19.3 如何创建线程池

Executors 提供了线程工厂方法用于创建线程池, 返回的线程池都实现了 ExecutorServer 接口。

- newSingleThreadExecutor
- newFixedThreadPool
- newCachedThreadPool
- newScheduledThreadPool

虽然 Java 自带的工厂方法很便捷，但都有弊端，《阿里巴巴 Java 开发手册》中强制线程池不允许使用以上方法创建，而是通过 `ThreadPoolExecutor` 的方式，这样处理可以更加明确线程池运行规则，规避资源耗尽的风险。

1.19.4 ThreadPoolExecutor 构造函数参数解析

- (1) `corePoolSize` 创建线程池的线程数量
- (2) `maximumPoolSize` 线程池的最大线程数
- (3) `keepAliveTime` 当线程数量大于 `corePoolSize`，空闲的线程当空闲时间超过 `keepAliveTime` 时就会回收；
- (4) `unit {keepAliveTime}` 时间单位
- (5) `workQueue` 保留任务的队列

1.19.5 列举线程安全的 Map 集合

`SynchronizedMap`、`ConcurrentHashMap`

1.19.6 StringBuffer 和 StringBuilder 的区别

- (1) `StringBuffer` 中的方法大都采用 `synchronized` 关键字进行修饰，是线程安全的，效率低。
- (2) `StringBuilder` 是线程不安全的，效率高。

1.19.7 ArrayList 和 LinkedList 的区别

- (1) `ArrayList` 基于动态数据实现，`LinkedList` 基于链表实现，两者都是线程不安全的
- (2) `ArrayList` 基于数组，查询快；`LinkedList` 基于链表，新增和删除更快
- (3) `LinkedList` 不支持高效的随机访问

1.19.8 HashMap 和 Hashtable 的区别

1) 继承的父类不同

`HashMap` 继承 `AbstractMap` 类

`Hashtable` 继承 `Dictionary` 类（已经废弃的类），用比较少

2) 是否线程安全

`HashMap` 是线程不安全的效率高，`Hashtable` 是线程安全的，效率低。

3) key 和 value 是否允许 null 值

Hashtable 中，key 和 value 都不允许出现 null 值。HashMap 中，都可出现 null。

1.19.9 HashMap 的底层原理

1) HashMap 的实现原理

HashMap 实际上是一个数组和链表的结合体，HashMap 基于 Hash 算法实现的；

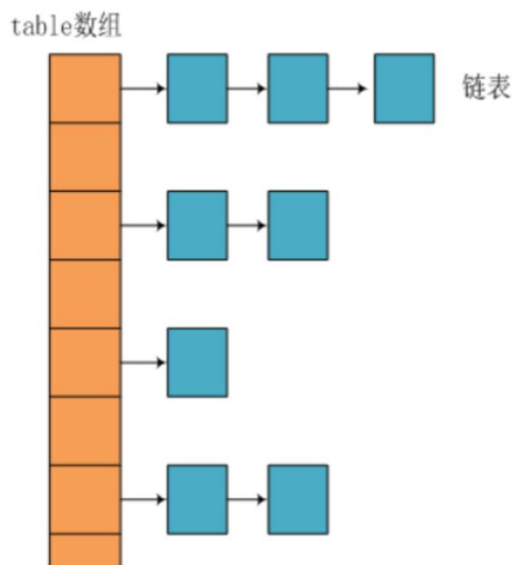
(1) 当我们向 HashMap 中 Put 元素时，利用 key 的 hashCode 重新计算出当前对象的元素在数组中的下标

(2) 写入时，如果出现 Hash 值相同的 key，此时分类，如果 key 相同，则覆盖原始值；如果 key 不同，value 则放入链表中

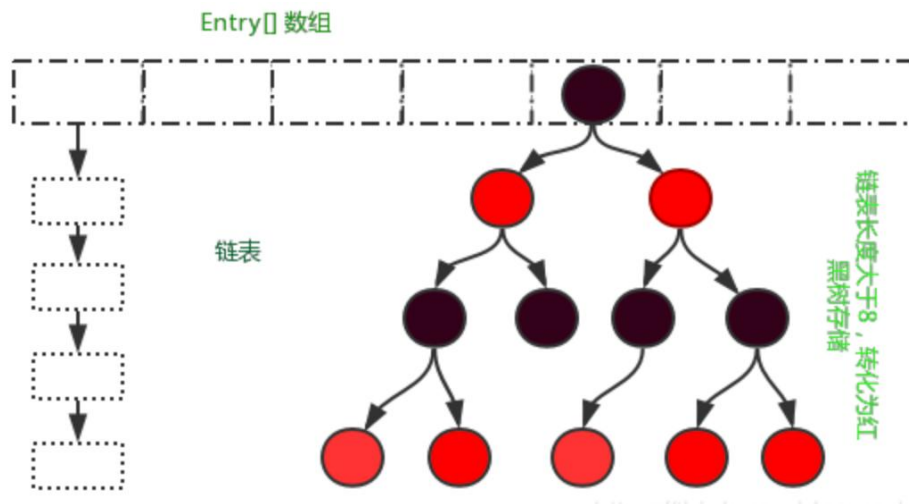
(3) 读取时，直接找到 hash 值对应的下标，在进一步判断 key 是否相同，进而找到对应值

2) HashMap 在 JDK1.7 和 JDK1.8 中有哪些区别

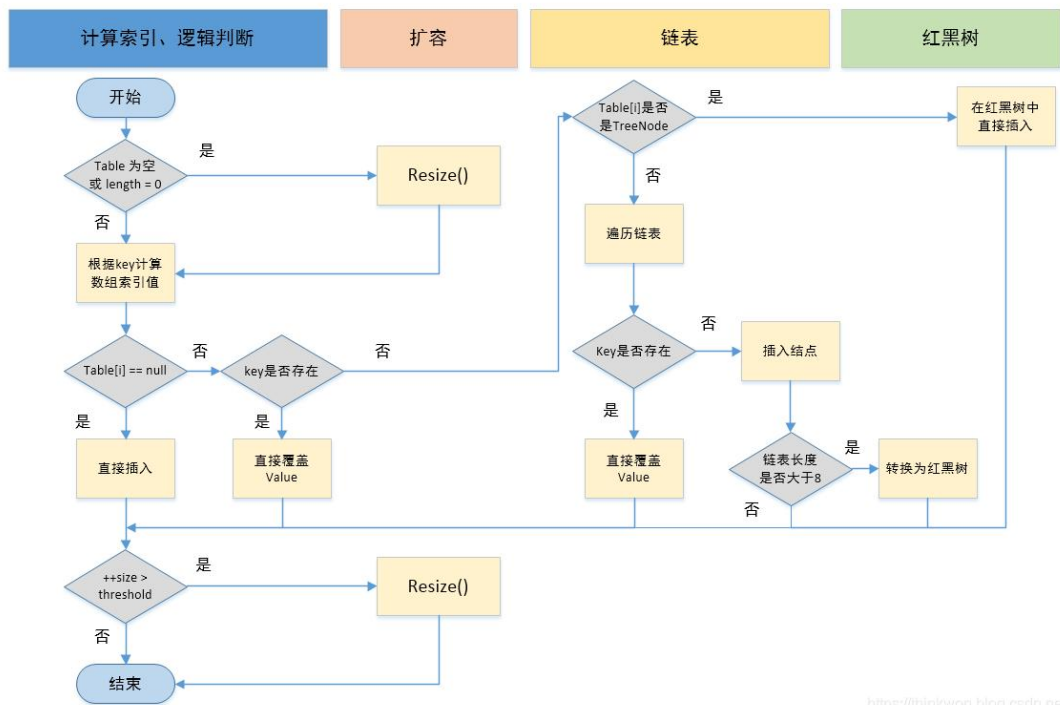
JDK1.7: 数组 + 链表



JDK1.8: 数组+红黑树



3) HashMap 的 Put 方法具体流程



4) HashMap 的扩容

HashMap 中的键值对大于阈值或者初始化时，就调用 `resize()` 进行扩容。

每次扩展的时候都是扩展 2 倍。

1.19.10 HashMap 里面放 100 条数据，初始化应该是多少

扩容因子 0.75

$100/0.75 \approx 133.3$

初始化 134

1.20 MySQL

1.20.1 MyISAM 与 InnoDB 的区别

| 对比项 | MyISAM | InnoDB |
|-----|------------------------------|---|
| 外键 | 不支持 | 支持 |
| 事务 | 不支持 | 支持 |
| 行表锁 | 表锁，即使操作一条记录也会锁住整个表，不适合高并发的操作 | 行锁，操作时只锁某一行，不对其它行有影响，适合高并发的操作 |
| 缓存 | 只缓存索引，不缓存真实数据 | 不仅缓存索引还要缓存真实数据，对内存要求较高，而且内存大小对性能有决定性的影响 |

1.20.2 MySQL 四种索引

1) 唯一索引

主键索引是唯一的，通常以表的 ID 设置为主键索引，一个表只能有一个主键索引，这是他跟唯一索引的区别。

2) 聚簇索引

聚簇索引的叶子节点都包含主键值、事务 ID、用于事务 MVCC 的回滚指针以及所有的剩余列。

3) 辅助索引（非聚簇索引|二级索引）

辅助索引也叫非聚簇索引，二级索引等，其叶子节点存储的不是行指针而是主键值，得到主键值再要查询具体行数据的话，要去聚簇索引中再查找一次，也叫回表。这样的策略优势是减少了当出现行移动或者数据页分裂时二级索引的维护工作。

4) 联合索引

两个或两个以上字段联合组成一个索引。使用时需要注意满足最左匹配原则！

1.20.3 MySQL 的事务

(1) 事务的基本要素（ACID）

(2) 事务的并发问题

脏读：事务 A 读取了事务 B 更新的数据，然后 B 回滚操作，那么 A 读取到的数据是脏

数据

不可重复读：事务 A 多次读取同一数据，事务 B 在事务 A 多次读取的过程中，对数据作了更新并提交，导致事务 A 多次读取同一数据时，结果 不一致

幻读：系统管理员 A 将数据库中所有学生的成绩从具体分数改为 ABCDE 等级，但是系统管理员 B 就在这个时候插入了一条具体分数的记录，当系统管理员 A 改结束后发现还有一条记录没有改过来，就好像发生了幻觉一样，这就叫幻读。

小结：不可重复读的和幻读很容易混淆，不可重复读侧重于修改，幻读侧重于新增或删除。解决不可重复读的问题只需锁住满足条件的行，解决幻读需要锁表

1.20.4 MySQL 事务隔离级别

| 事务隔离级别 | 脏读 | 不可重复读 | 幻读 |
|-------------------------|----|-------|----|
| 读未提交 (read-uncommitted) | 是 | 是 | 是 |
| 不可重复读 (read-committed) | 否 | 是 | 是 |
| 可重复读 (repeatable-read) | 否 | 否 | 是 |
| 串行化 (serializable) | 否 | 否 | 否 |

1.20.5 MyISAM 与 InnoDB 对比

(1) InnoDB 的数据文件本身就是索引文件，而 MyISAM 索引文件和数据文件是分离的：

①InnoDB 的表在磁盘上存储在以下文件中：.ibd（表结构、索引和数据都存在一起,MySQL5.7 表结构放在.frm 中）

②MyISAM 的表在磁盘上存储在以下文件中：*.sdi（描述表结构，MySQL5.7 是.frm）、*.MYD（数据），*.MYI（索引）

(2) InnoDB 中主键索引是聚簇索引，叶子节点中存储完整的数据记录；其他索引是非聚簇索引，存储相应记录主键的值。

(3)InnoDB 要求表必须有主键（MyISAM 可以没有）。如果没有显式指定，则 MySQL 系统会自动选择一个可以非空且唯一标识数据记录的列作为主键。如果不存在这种列，则 MySQL 自动为 InnoDB 表生成一个隐含字段作为主键。

(4) MyISAM 中无论是主键索引还是非主键索引都是非聚簇的，叶子节点记录的是数据的地址。

(5) MyISAM 的回表操作是十分快速的，因为是拿着地址偏移量直接到文件中取数据的，反观 InnoDB 是通过获取主键之后再去聚簇索引里找记录，虽然说也不慢，但还是比不上直接用地址去访问。

1.20.6 B 树和 B+树对比

1) B+ 树和 B 树的差异

(1) B+树中非叶子节点的关键字也会同时存在子节点中，并且是在子节点中所有关键字的最大值（或最小）。

(2) B+树中非叶子节点仅用于索引，不保存数据记录，跟记录有关的信息都放在叶子节点中。而 B 树中，非叶子节点既保存索引，也保存数据记录。

(3) B+树中所有关键字都在叶子节点出现，叶子节点构成一个有序链表，而且叶子节点本身按照关键字的大小从小到大顺序链接。

2) B+树为什么 IO 的次数会更少

真实环境中一个页存放的记录数量是非常大的（默认 16KB），假设指针与键值忽略不计（或看做 10 个字节），数据占 1 kb 的空间：

如果 B+树只有 1 层，也就是只有 1 个用于存放用户记录的节点，最多能存放 16 条记录。

如果 B+树有 2 层，最多能存放 $1600 \times 16 = 25600$ 条记录。

如果 B+树有 3 层，最多能存放 $1600 \times 1600 \times 16 = 40960000$ 条记录。

如果存储千万级别的数据，只需要三层就够了。

B+树的非叶子节点不存储用户记录，只存储目录记录，相对 B 树每个节点可以存储更多的记录，树的高度会更矮胖，IO 次数也会更少。

1.21 Redis

1.21.1 Redis 缓存穿透、缓存雪崩、缓存击穿

(1) 缓存穿透是指查询一个一定不存在的数据。由于缓存命中时会去查询数据库，查不到数据则不写入缓存，这将导致这个不存在的数据每次请求都要到数据库去查询，造成缓存穿透。

解决方案：

①是将空对象也缓存起来，并给它设置一个很短的过期时间，最长不超过 5 分钟。

②采用布隆过滤器，将所有可能存在的数据哈希到一个足够大的 bitmap 中，一个一定

不存在的数据会被这个 bitmap 拦截掉，从而避免了对底层存储系统的查询压力。

(2) 如果缓存集中在一段时间内失效，发生大量的缓存穿透，所有的查询都落在数据库上，就会造成缓存雪崩。

解决方案：尽量让失效的时间点不分布在同一个时间点。

(3) 缓存击穿，是指一个 key 非常热点，在不停的扛着大并发，当这个 key 在失效的瞬间，持续的大并发就穿破缓存，直接请求数据库，就像在一个屏障上凿开了一个洞。

解决方案：可以设置 key 永不过期。

1.21.2 Redis 哨兵模式

(1) 主从复制中反客为主的自动版，如果主机 Down 掉，哨兵会从从机中选择一台作为主机，并将它设置为其他从机的主机，而且如果原来的主机再次启动的话也会成为从机。

(2) 哨兵模式是一种特殊的模式，首先 Redis 提供了哨兵的命令，哨兵是一个独立的进程，作为进程，它独立运行。其原理是哨兵通过发送命令，等待 Redis 服务器响应，从而监控运行的多个 Redis 实例。

(3) 当哨兵监测到 Redis 主机宕机，会自动将 Slave 切换成 Master，然后通过发布订阅模式通知其他服务器，修改配置文件，让他们换主机。

(4) 当一个哨兵进程对 Redis 服务器进行监控，可能会出现問題，为此可以使用哨兵进行监控，各个哨兵之间还会进行监控，这就形成了多哨兵模式。

1.21.3 Redis 数据类型

- String 字符串
- List 可以重复的集合
- Set 不可以重复的集合
- Hash 类似于 Map<String,String>
- Zset(sorted set) 带分数的 set

1.21.4 热数据通过什么样的方式导入 Redis

提供一种简单实现缓存失效的思路：LRU（最近少用的淘汰）。

即 Redis 的缓存每命中一次，就给命中的缓存增加一定 TTL（过期时间）（根据具体情况来设定，比如 10 分钟）。

一段时间后，热数据的 TTL 都会较大，不会自动失效，而冷数据基本上过了设定的 TTL

就马上失效了。

1.21.5 Redis 的存储模式 RDB, AOF

Redis 默认开启 RDB 持久化方式，在指定的时间间隔内，执行指定次数的写操作，则将内存中的数据写入到磁盘中。

RDB 持久化适合大规模的数据恢复但它的数据一致性和完整性较差。

Redis 需要手动开启 AOF 持久化方式，默认是每秒将写操作日志追加到 AOF 文件中。

AOF 的数据完整性比 RDB 高，但记录内容多了，会影响数据恢复的效率。

Redis 针对 AOF 文件大的问题，提供重写的瘦身机制。

若只打算用 Redis 做缓存，可以关闭持久化。

若打算使用 Redis 的持久化。建议 RDB 和 AOF 都开启。其实 RDB 更适合做数据的备份，留一后手。AOF 出问题了，还有 RDB。

1.21.6 Redis 存储的是 k-v 类型，为什么还会有 Hash?

Redis 的 hash 数据结构是一个键值对 (key-value) 集合，他是一个 String 类型的 field 和 value 的映射表，Redis 本身就是一个 key-value 类型的数据库，因此 Hash 数据结构等于在原来的 value 上又套了一层 key-value 型数据。所以 Redis 的 hash 数据类型特别适合存储关系型对象。

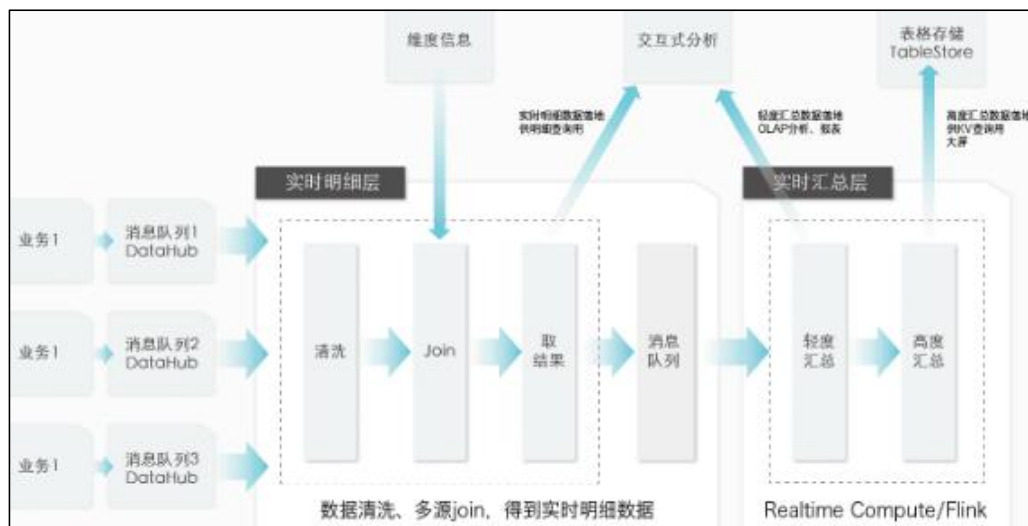
1.22 JVM

关注尚硅谷教育公众号，回复 java。

第 2 章 离线数仓项目

2.1 提高自信

云上数据仓库解决方案: <https://www.aliyun.com/solution/datavexpo/datawarehouse>



2.2 为什么做这个项目

随着公司的发展，老板需要详细的了解公司的运营情况。比如，日活、新增、留存、转化率等。所以公司决定招聘大数据人才来做这个项目，目的是为老板做决策提供数据支持。

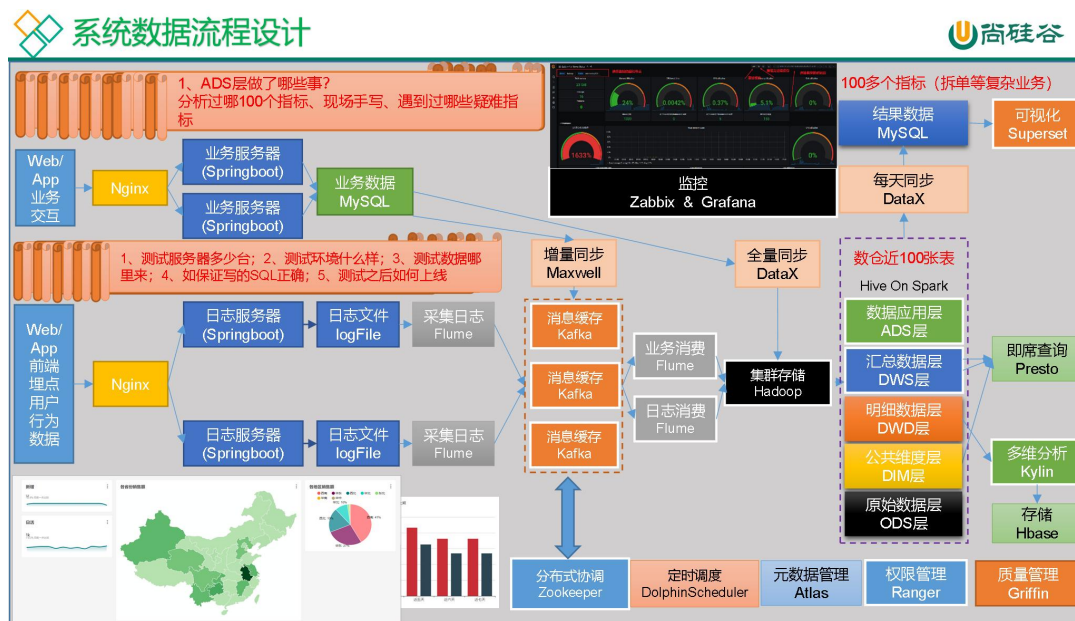
2.3 数仓概念

数据仓库的输入数据源和输出系统分别是什么？

(1) 输入系统：前端埋点产生的用户行为数据、JavaEE 后台产生的业务数据、个别公司有爬虫数据。

(2) 输出系统：报表系统、用户画像系统、推荐系统。

2.4 项目架构



2.5 框架版本选型

1) Apache: 运维麻烦，组件间兼容性需要自己调研。(一般大厂使用，技术实力雄厚，有专业的运维人员)。

2) CDH6.3.2: 国内使用最多的版本。CDH 和 HDP 合并后推出，CDP7.0。收费标准，10000 美金一个节点每年。(不建议使用)

3) HDP: 开源，可以进行二次开发，但是没有 CDH 稳定，国内使用较少。

4) 云服务选择

- (1) 阿里云的 EMR、MaxCompute、DataWorks
- (2) 腾讯云 EMR、流计算 Oceanus、数据开发治理平台 WeData
- (3) 华为云 EMR
- (4) 亚马逊云 EMR

星环国际、金蝶。。。神策、数梦、袋鼠云

Apache 框架各组件重要版本发版时间

| 框架 | 版本号 | 发版时间 | 框架 | 版本号 | 发版时间 |
|----|-----|------|----|-----|------|
|----|-----|------|----|-----|------|

| | | | | | |
|-----------|---------------|----------------|------------------|---------------|----------------|
| Hadoop | 2.7.2 | 2017-06 | Spark | 1.6.0 | 2016-01 |
| | 3.0.0 | 2018-03 | | 2.0.0 | 2016-07 |
| | 3.1.3 | 2020-07 | | 2.2.0 | 2018-05 |
| Zookeeper | 3.4.12 | 2018-05 | | 2.4.0 | 2018-11 |
| | 3.4.14 | 2019-04 | | 3.0.0 | 2020-06 |
| | 3.5.8 | 2020-05 | | 2.4.8 | 2022-06 |
| | 3.7.0 | 2021-03 | | 3.2.0 | 2021-10 |
| | 3.8.0 | 2022-03 | | 3.3.0 | 2022-06 |
| Flume | 1.9.0 | 2019-01 | Flink | 1.7.0 | 2018-11 |
| | 1.10.0 | 2022-03 | | 1.8.0 | 2019-04 |
| | 1.11.0 | 2022-10 | | 1.9.0 | 2019-08 |
| Kafka | 1.0.0 | 2017-11 | | 1.10.0 | 2020-02 |
| | 2.0.0 | 2018-07 | | 1.11.0 | 2020-07 |
| | 2.3.0 | 2019-03 | | 1.12.0 | 2020-12 |
| | 2.4.0 | 2019-12 | | 1.13.0 | 2021-04 |
| | 2.7.0 | 2020-12 | | 1.13.6 | 2022-02 |
| | 3.0.0 | 2021-09 | | 1.14.0 | 2021-09 |
| Hive | 1.2.1 | 2015-06 | | 1.15.0 | 2022-05 |
| | 2.0.0 | 2016-02 | | 1.16.0 | 2022-10 |
| | 2.2.0 | 2017-07 | DolphinScheduler | 1.2.0（最早） | 2020-01 |
| | 3.0.0 | 2018-05 | | 1.3.9 | 2021-10 |
| | 2.3.6 | 2019-08 | | 2.0.0 | 2021-11 |
| | 3.1.2 | 2019-08 | | 3.0.0 | 2022-08 |
| | 2.3.7 | 2020-04 | Doris | 0.13.0（最早） | 2020-10 |
| | 3.1.3 | 2022-04 | | 0.14.0 | 2021-05 |
| HBase | 1.2.0 | 2016-02 | | 0.15.0 | 2021-11 |
| | 1.4.0 | 2017-12 | | 1.1.0 | 2022-07 |
| | 1.5.0 | 2019-10 | Hudi | 0.10.0 | 2021-12 |
| | 1.6.0 | 2020-07 | | 0.11.0 | 2022-03 |
| | 2.0.0 | 2018-05 | | 0.12.0 | 2022-08 |
| | 2.2.0 | 2019-06 | Sqoop | 1.4.6 | 2017-10 |
| | 2.4.0 | 2020-12 | | 1.4.7 | 2020-07 |
| | 2.5.0 | 2022-08 | | | |

| | | | | | |
|---------|--------------------------------|----------------|--|--|--|
| Phoenix | 4.14.0
(1.4) | 2018-06 | | | |
| | 4.16.1
(1.3, 1.4, 1.5, 1.6) | 2021-05 | | | |
| | 5.1.2
(2.1, 2.2, 2.3, 2.4) | 2021-07 | | | |

*注：着重标出的为公司实际生产中的常用版本。

2.6 服务器选型

服务器使用物理机还是云主机？

1) 机器成本考虑：

(1) 物理机：以 128G 内存，20 核物理 CPU，40 线程，8THDD 和 2TSSD 硬盘，单台报价 4W 出头，惠普品牌。一般物理机寿命 5 年左右。

(2) 云主机，以阿里云为例，差不多相同配置，每年 5W。华为云、腾讯云、天翼云。

2) 运维成本考虑：

(1) 物理机：需要有专业的运维人员（1 万 * 13 个月）、电费（商业用户）、安装空调、场地。

(2) 云主机：很多运维工作都由阿里云已经完成，运维相对较轻松。

3) 企业选择

(1) 金融有钱公司选择云产品（上海）。

(2) 中小公司、为了融资上市，选择云产品，拉到融资后买物理机。

(3) 有长期打算，资金比较足，选择物理机。

2.7 集群规模

1) 硬盘方面考虑

集群数据量评估



1) 用户行为数据

- (1) 每天活跃用户100万，每人一天平均100条： $100万 * 100条 = 10000万条$
- (2) 每条日志1K左右，每天1亿条： $100000000 / 1024 / 1024 = 约100G$
- (3) 数仓ODS层采用gzip存储：100g压缩为30g左右
- (4) 数仓DWD层采用snappy+orc存储：30g左右
- (5) 数仓DWS层聚合存储（为了快速运算，不压缩）：不足10G，按照10G估算
- (6) 数仓ADS层数据量很小：忽略不计
- (7) 保存3副本： $70g * 3 = 210g$
- (8) 1年内不扩容服务器来算： $210g * 360天 = 约74T$
- (9) 预留20%-30%Buf= $74T / 0.7 = 105T$

2) Kafka中数据

- (1) 每天约100G数据*副本(2)=200g
- (2) 保存3天*200g=600g
- (3) 预留30%Buf=600g/0.7=857g=约1T

3) 业务数据

- (1) 每天活跃用户100万，每天下单的用户10万，每人每天产生的业务数据10条，每条日志1k左右： $10万 * 10条 * 1k = 1g$ 左右
- (2) 数仓五层存储：1g*3=3g
- (3) 保存3副本：3g*3=9g
- (4) 1年内不扩容服务器来算： $9g * 360天 = 约3.2T$
- (5) 预留20%-30%Buf= $3.2T / 0.7 = 5T$

4) 集群总数据量：105T+1T+5T=111T

让天下没有难学的技术

2) CPU 方面考虑

20 核物理 CPU 40 线程 * 8 = 320 线程 (指标 100-200)

3) 内存方面考虑

内存 128g * 8 台 = 1024g (计算任务内存 800g，其他安装框架需要内存)

128m=》512M 内存

=》

200g 数据 、800g 内存

4) 参考案例说明

根据数据规模搭建集群（在企业，干了三年 通常服务器集群 5-20 台之间）

(1) 参考腾讯云 EMR 官方推荐部署

| 节点类型 | 节点规格 | 节点数量 |
|------------|--|------|
| Master节点配置 | 标准型SA3: 8核16G
系统盘: SSD云盘50G*1
数据盘: SSD云盘200G*1 | 2 |
| Core节点配置 | 标准型SA3: 4核8G
系统盘: SSD云盘50G*1
数据盘: SSD云盘200G*1 | 3 |
| Task节点配置 | 标准型SA3: 4核8G
系统盘: SSD云盘50G*1
数据盘: SSD云盘200G*1 | 0 |
| Common节点配置 | 标准型SA3: 2核4G
系统盘: SSD云盘50G*1
数据盘: SSD云盘200G*1 | 3 |

配置费用 11.62元/小时

下一步

- **Master 节点：**管理节点，保证集群的调度正常进行；主要部署 NameNode、ResourceManager、HMaster 等进程；非 HA 模式下数量为 1，HA 模式下数量为 2。

- **Core 节点:** 为计算及存储节点, 我们在 HDFS 中的数据全部存储于 core 节点中, 因此为了保证数据安全, 扩容 core 节点后不允许缩容; 主要部署 DataNode、NodeManager、RegionServer 等进程。非 HA 模式下数量 ≥ 2 , HA 模式下数量 ≥ 3 。
- **Common 节点:** 为 HA 集群 Master 节点提供数据共享同步以及高可用容错服务; 主要部署分布式协调器组件, 如 ZooKeeper、JournalNode 等节点。非 HA 模式数量为 0, HA 模式下数量 ≥ 3 。

(2) 数据传输数据比较紧密的放在一起 (Kafka、clickhouse)

(3) 客户端尽量放在一到两台服务器上, 方便外部访问

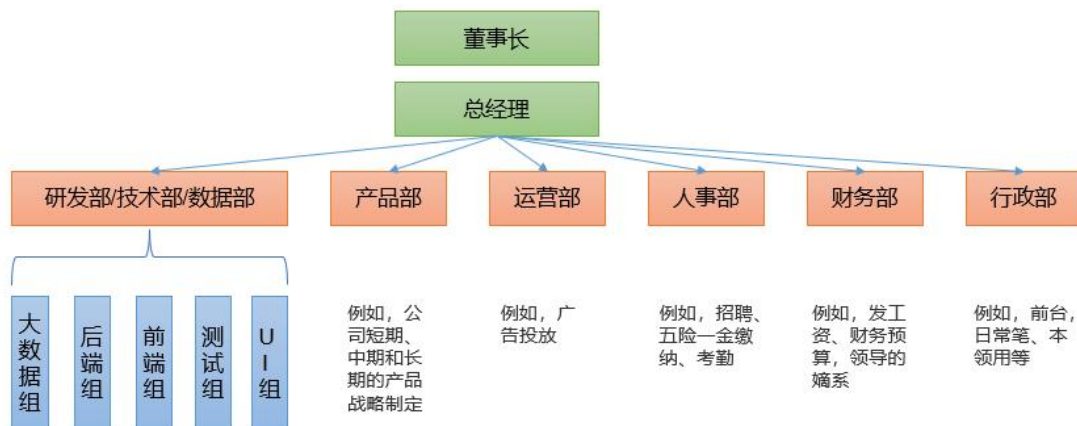
(4) 有依赖关系的尽量放到同一台服务器 (例如: Ds-worker 和 hive/spark, ClickHouse

必须单独部署)

| Master | Master | core | core | core | common | common | common |
|-----------|-----------|-----------|-----------|-----------|-------------|-------------|-------------|
| nn | nn | dn | dn | dn | JournalNode | JournalNode | JournalNode |
| rm | rm | nm | nm | nm | | | |
| | | | | | zk | zk | zk |
| hive | hive | hive | hive | hive | | | |
| | | kafka | kafka | kafka | | | |
| spark | spark | spark | spark | spark | | | |
| datax | datax | datax | datax | datax | | | |
| Ds-master | Ds-master | Ds-worker | Ds-worker | Ds-worker | | | |
| maxwell | | | | | | | |
| superset | | | | | | | |
| mysql | | | | | | | |
| flume | flume | | | | | | |
| flink | flink | | | | | | |
| | | | redis | | | | |
| hbase | | | | | | | |

2.8 人员配置参考

2.8.1 整体架构



大数据开发工程师 =》 大数据组组长=》项目经理=》部门经理=》技术总监 CTO
 =》 高级架构师 =》 资深架构师

2.8.2 你的职级等级及晋升规则

小公司：职级就分初级，中级，高级。晋升规则不一定，看公司效益和职位空缺。

大公司都有明确的职级：

TogoCareer
程序员求职招聘第一站

互联网大厂职级薪资一览表

| T职级 | 职衔 | 月薪区间 | M职级 | 职衔 | 月薪区间 |
|-----|-------|--------------|------|-------|--------------|
| T1 | 初级1 | 5-12K | | | |
| T2 | 初级2 | 12-20K | | | |
| T3 | 中级1 | 15-27K | | | |
| T4 | 中级2 | 20-38K | | | |
| T5 | 高级工程师 | 30-45K | M1 | 主管 | 30-45K |
| T6 | 资深工程师 | 37-60K | M2-1 | 副经理 | 37-60K |
| T7 | 架构师 | 50-75K | M2-2 | 经理 | 50-75K |
| T8 | 技术专家 | 60-105K | M3 | 高级经理 | 60-105K |
| T9 | 副总监 | 75-110K | M4-1 | 副总监 | 75-110K |
| T10 | 总监 | 90-130K | M4-2 | 总监 | 90-130K |
| T11 | 高级总监 | case by case | M4-3 | 高级总监 | |
| | | | M5-1 | VP | case by case |
| | | | M5-2 | CXO | |
| | | | M6 | 高级副总裁 | |

TogoCareer
程序员求职招聘第一站

互联网大厂职级薪资一览表

| 职级 | title | 薪资 |
|-----|---------|---------|
| D5 | 工程师 | |
| D6 | 高级工程师 | 25-45K |
| D7 | 资深工程师 | 35-60K |
| D8 | 专家工程师 | 60-80K |
| D9 | 高级专家工程师 | 75-100K |
| D10 | 首席工程师 | |
| D11 | 杰出工程师 | |
| D12 | D12 | |

互联网大厂职级薪资一览表

字节跳动

| T职级 | title |
|-----|--------------|
| 1-1 | 初级工程师（一般应届生） |
| 1-2 | 中级工程师 |
| 2-1 | 资深研发 |
| 2-2 | |
| 3-1 | |
| 3-2 | team领导层 |
| 4-1 | |
| 4-2 | 部门领导层 |
| 5-1 | |
| 5-2 | 公司领导层 |

不同序列间月薪base差异较大，技术base整体偏高。比如2-1月薪会在20k+，2-2的package会在90w-150w左右（算上期权，大概会占30%左右）。

T2-2 级别对标阿里P7，每年会有500股票，年薪总包大概是135-190w

互联网大厂职级薪资一览表

阿里巴巴

| 层级 | 层级名称 | 层级 | 层级名称 | 薪资（16薪） |
|-----|-------|-----|----------------------|--------------|
| | | M10 | 董事长 (chairman) | case by case |
| P14 | 资深科学家 | M9 | 副董事长 (Vice Chairman) | |
| P13 | 科学家 | M8 | 执行副总裁 (EVP) | |
| P12 | 资深研究员 | M7 | 资深副总裁 (Sr.VP) | |
| P11 | 高级研究员 | M6 | 副总裁 (VP) | |
| P10 | 研究员 | M5 | 资深总监 | 220W+ |
| P9 | 资深专家 | M4 | 总监 | 150-180W |
| P8 | 高级专家 | M3 | 资深经理 | 100-150W |
| P7 | 专家 | M2 | 经理 | 75-105W |
| P6 | 高级工程师 | M1 | 主管 | 60W |
| P5 | 中级工程师 | | | case by case |
| P4 | 初级工程师 | | | |

互联网大厂职级薪资一览表

腾讯

| 新职级体系 | 旧职级体系 | 薪酬中位数 |
|-------|-------|---------------|
| 4 | 1.1 | 18K |
| 5 | 1.2 | |
| 6 | 1.3 | |
| 7 | 2.1 | 22K |
| 8 | 2.2 | 27k |
| 9 | 2.3 | 30K |
| 10 | 3.1 | 38k(45-60W) |
| 11 | 3.2 | 50k(75-110W) |
| 12 | 3.3 | 65k(105-150W) |
| 13 | 4.1 | 180-240W |
| 14 | 4.2 | 300-450W |
| 15 | 4.3 | case by case |
| 16 | 5.1 | |
| 17 | 5.2 | |
| 18 | 5.3 | |
| 19 | 6.1 | |
| 20 | 6.2 | |
| 21 | 6.3 | |

互联网大厂职级薪资一览表

HUAWEI

| 级别 | 基本定义 | 工作年限 | 薪酬 |
|--------|--|------|-------------|
| 13 | 共13-22级，每一级（技术岗部分）分ABC三小级，大部分华为员工在18级以内，18级是个坎，正常每年升一小级，工资每年一涨，15级以上工资涨的较慢 | 1-3年 | 30-37W（无股票） |
| 14 | | 3-5年 | 50-54W（无股票） |
| 15 | | 5年以上 | 45-53W |
| 16 | | | 75-90W |
| 17 | | | 90-105W |
| 18 | 90-150W | | |
| 19 | 领导/专家 | -- | 225-300W |
| 20 | | | 450-600W |
| 21 | | | 750-950W |
| 22 | | | |
| 23级及以上 | 高级别boss，华为内网不显示 | | |

2.8.3 人员配置参考

小型公司（1-3 人左右）：组长 1 人，剩余组员无明确分工，并且可能兼顾 JavaEE 和前端。

中小型公司（3~6 人左右）：组长 1 人，离线 2 人左右，实时 1 人左右（离线一般多于实时），组长兼顾和 JavaEE、前端。

中型公司（5~10 人左右）：组长 1 人，离线 3~5 人左右（离线处理、数仓），实时 2 人左右，组长和技术大牛兼顾和 JavaEE、前端。

中大型公司（10~20 人左右）：组长 1 人，离线 5~10 人（离线处理、数仓），实时 5 人

左右，JavaEE 1 人左右（负责对接 JavaEE 业务），前端 1 人（有或者没有人单独负责前端）。

（发展比较好的中大型公司可能大数据部门已经细化拆分，分成多个大数据组，分别负责不同业务）

上面只是参考配置，因为公司之间差异很大，例如 ofo 大数据部门只有 5 个人左右，因此根据所选公司规模确定一个合理范围，在面试前必须将这个人员配置考虑清楚，回答时要非常确定。

咱们自己公司：大数据组组长：1 个人；离线 3-4 个人；实时 1-3 个人。

IOS 多少人？安卓多少人？前端多少人？JavaEE 多少人？测试多少人？

（IOS、安卓） 1-2 个人 前端 3 个人； JavaEE 一般是大数据的 1-1.5 倍，测试：有的有，1 个左右，有的没有。 产品经理 1 个、产品助理 1-2 个，运营 1-3 个。

公司划分：

0-50 小公司；50-500 中等；500-1000 大公司；1000 以上 大厂 领军的存在。

2.9 从 0-1 搭建项目，你需要做什么？

1) 需要问项目经理的问题

- (1) 数据量（历史数据、增量、全量）： 100g
- (2) 预算： 50 万
- (3) 数据存储多久： 1 年
- (4) 云主机、物理机： 云主机
- (5) 日活： 100 万
- (6) 数据源： 接口、用户行为数据(文件)、业务数据 (MySQL)
- (7) 项目周期： 1 个月-3 个月
- (8) 团队多少人： 3-5 个
- (9) 首批指标： 1-10 个
- (10) 未来的规划： 离线和实时 是否都要做

2) 项目周期（2 个月）

- (1) 数据调研（2 周） + 集群搭建
- (2) 明确数据域（2 天）
- (3) 构建业务矩阵（3 天）
- (4) 建模 至下而上 （2 周）
 - ①ODS 层 ②DWD 层 ③DIM 层
- (5) 指标体系建设 至上而下 （2 周）

(6) 处理 bug 1 周

2.10 数仓建模准备

1) 数据仓库建模的意义

如果把数据看作图书馆里的书，我们希望看到它们在书架上分门别类地放置：

- 减少重复计算。
- 快速查询所需要的数据。

2) ER 模型

如果对方向三范式问题。初步判断对方是一个 java 程序员，就不要和他深入聊，mysql 高级、redis、多线程、JVM、SSM 等框架了。

应该把话题转移到大数据技术。Spark、flink、海量数据如何处理、维度建模。

3) 维度建模

星型模型：事实表周围一级维度 减少 join => 大数据场景不适合频繁的 join

雪花模型：事实表周期多级维度

星座：多个事实表

4) 事实表

(1) 如何判断一张表是事实表？

具有度量值的 可以累加的 个数、件数、金额、次数

(2) 同步策略

数据量大 =》 通常增量 特殊的，加购（周期快照事实表）

(3) 分类

①事务型事实表

找原子操作。 例如：下单 加购 支付

①选择业务过程

②声明粒度

③确定维度

④确定事实

不足：

连续性指标，不好找原子操作。 例如，库存（周期快照事实表）

多事实表关联。例如，统计加购到支付的平均使用时长（累积型快照事实表）

②周期快照事实表

③累积型快照事实表

5) 维度表

(1) 如何判断一张表是维度表？

没有度量值，都是描述信息。 身高 体重、年龄、性别

(2) 同步策略

数据量小 =》 通常 全量 特殊的 用户表

(3) 维度整合 减少 Join 操作

①商品表、商品品类表、SPU、商品一级分类、二级分类、三级分类=》商品维度

表

②省份表、地区表 =》 地区维度表

③活动信息表、活动规则表 =》 活动维度表

(4) 拉链表

对用户表做了拉链。

缓慢变化维 场景



用户维度表每日装载思路



6) 建模工具是什么？

PowerDesigner、EZDML

2.11 数仓建模

1) 数据调研

(1) 先和 Java 人员要表，表中最好有字段的描述或者有表和字段的说明文档。(项目经理帮助协调) => 快速熟悉表中业务。梳理清楚业务线，找到事实表和维度表。

(2) 和业务人员聊 => 验证你猜测的是否正确

(3) 和产品经理聊

需求：派生指标、衍生指标

派生指标 = 原子指标 (业务过程 + 度量值 + 聚合逻辑) + 统计周期 + 统计粒度 + 业务限定

需求中的业务过程必须和实际的后台业务能对应上。

2) 明确数据域

(1) 用户域：登录、注册

(2) 流量域：启动、页面、动作、故障、曝光

(3) 交易域：加购、下单、支付、物流、取消下单、取消支付

(4) 工具域：领取优惠券、使用优惠券下单、使用优惠券支付

(5) 互动域：点赞、评论、收藏

3) 构建业务矩阵

用户、商品、活动、时间、地区、优惠券

(1) 用户域：

登录、注册

(2) 流量域： ✓

启动、页面、动作、故障、曝光

(3) 交易域：

加购、下单、支付、物流、取消下单、取消支付

(4) 工具域：

领取优惠券、使用优惠券下单、使用优惠券支付

(5) 互动域：

点赞、评论、收藏

4) 建模 至下而上

(1) ODS 层

- ①保持数据原貌不做任何修改 起到备份作用
- ②采用压缩 减少磁盘空间，采用 Gzip 压缩
- ③创建分区表 防止后续全表扫描

(2) DWD 层 事实表

①事务型事实表

找原子操作

a) 选择业务过程

选择感兴趣的业务过程。 产品经理提出的指标中需要的。

b) 声明粒度

粒度：一行信息代表什么含义。可以是一次下单、一周下单、一个月下单。

如果是一个月的下单,就没有办法统计一次下单情况。保持最小粒度。

只要你自己不做聚合操作就可以。

c) 确定维度

确定感兴趣的维度。 产品经理提出的指标中需要的。

例如：用户、商品、活动、时间、地区、优惠券

d) 确定事实

确定事实表的度量值。 可以累加的值，例如，个数、件数、次数、金额。

事务型事实表的不足：

- 连续性指标，不好找原子操作。 例如，库存（周期快照事实表）
- 多事实表关联。例如，统计加购到支付的平均使用时长（累积型快照事实表）

(2) 周期快照事实表

①选择业务过程

②声明粒度 =》 1 天

③确定维度

④确定事实

(3) 累积型快照事实表

①选择业务过程

②声明粒度

③确定维度

④确定事实 确定多个事实表度量值

(3) DIM 层 维度表

①维度整合 减少 join

a) 商品表、商品品类表、spu、商品一级分类、二级分类、三级分类=》商品
维度表

b) 省份表、地区表 =》 地区维度表

c) 活动信息表、活动规则表 =》 活动维度表

②拉链表

对用户表做了拉链。

缓慢变化维 场景。

5) 指标体系建设 至上而下

(1) ADS 层

需求、日活、新增、留存、转化率、GMV

(2) DWS 层 聚合层

需求：派生指标、衍生指标

派生指标 = 原子指标（业务过程 + 度量值 + 聚合逻辑） + 统计周期 + 统计
粒度 + 业务限定

例如，统计，每天各个省份手机品牌交易总额

交易总额 （下单 + 金额 + sum ） + 每天 + 省份 + 手机品牌

找公共的：业务过程 + 统计周期 + 统计粒度 建宽表

2.12 数仓每层做了哪些事

1) ODS 层做了哪些事？

(1) 保持数据原貌，不做任何修改

(2) 压缩采用 gzip，压缩比是 100g 数据压缩完 10g 左右。

(3) 创建分区表

2) DIM/DWD 层做了哪些事?

建模里面的操作，正常写。

(1) 数据清洗的手段

HQL、MR、SparkSQL、Kettle、Python（项目中采用 SQL 进行清除）

(2) 清洗规则

金额必须都是数字，[0-9]、手机号、身份证、匹配网址 URL

解析数据、核心字段不能为空、过期数据删除、重复数据过滤

json => 很多字段 => 一个一个判断 => 取数，根据规则匹配

(3) 清洗掉多少数据算合理

参考，1 万条数据清洗掉 1 条。

(4) 脱敏

对手机号、身份证号等敏感数据脱敏。

①加*

135****0013 互联网公司经常采用

②加密算法 md5 需要用数据统计分析，还想保证安全

美团 滴滴 md5 (12334354809) => 唯一值

③加权限 需要正常使用 军工、银行、政府

(5) 压缩 snappy

(6) orc 列式存储

3) DWS 层做了哪些事?

指标体系建设里面的内容再来一遍。

4) ADS 层做了哪些事?

一分钟至少说出 30 个指标。

日活、月活、周活、留存、留存率、新增（日、周、年）、转化率、流失、回流、七天内连续 3 天登录（点赞、收藏、评价、购买、加购、下单、活动）、连续 3 周（月）登录、GMV、复购率、复购率排行、点赞、评论、收藏、领优惠券人数、使用优惠券人数、沉默、值不值得买、退款人数、退款率 topn 热门商品

产品经理最关心的：留转 G 复活



2.13 数据量

数据量的描述都是压缩前的数据量。

1) ODS 层:

(1) 用户行为数据 (100g => 1 亿条; 1g => 100 万条)

曝光 (60g or 600 万条)、页面 (20g)、动作 (10g)、故障 + 启动 (10g)

(2) 业务数据 (1-2g => 100 万-200 万条)

登录 (20 万)、注册 (100-1000);

加购 (每天增量 20 万、全量 100 万)、下单 (10 万)、支付 (9 万)、物流 (9 万)、取消下单 (500)、退款 (500);

领取优惠券 (5 万)、使用优惠券下单 (4 万)、使用优惠券支付 (3 万);

点赞（1000）、评论（1000）、收藏（1000）；

用户（活跃用户 100 万、新增 1000、总用户 1 千万）、商品 SPU（1-2 万）、商品 SKU（10-20 万）、活动（1000）、时间（忽略）、地区（忽略）

2) DWD 层 + DIM 层：

和 ODS 层几乎一致；

3) DWS 层

轻度聚合后，20g-50g。

4) ADS 层

10-50m 之间，可以忽略不计。

2.14 项目中遇到哪些问题？（*****）

- 1) Flume 零点漂移
- 2) Flume 挂掉及优化
- 3) Datax 空值、调优
- 4) HDFS 小文件处理
- 5) Kafka 挂掉
- 6) Kafka 丢失
- 7) Kafka 数据重复
- 8) Kafka 消息数据积压
- 9) Kafka 乱序
- 10) Kafka 顺序
- 11) Kafka 优化（提高吞吐量）
- 12) Kafka 底层怎么保证高效读写
- 13) Kafka 单条日志传输大小
- 14) Hive 优化（Hive on Spark）
- 15) Hive 解决数据倾斜方法
- 19) 疑难指标编写（7 天内连续 3 次活跃、1730 指标、路径分析、用户留存率、最近 7/30 日各品牌复购率、最近 30 天发布的优惠券的补贴率、同时在线人数）
- 20) DS 任务挂了怎么办？

21) DS 故障报警

2.15 离线---业务

2.15.1 SKU 和 SPU

SKU：一台银色、128G 内存的、支持联通网络的 iPhoneX。

SPU：iPhoneX。

Tm_id：品牌 Id 苹果，包括 IPHONE，耳机，MAC 等。

2.15.2 订单表跟订单详情表区别？

订单表的订单状态会变化，订单详情表不会，因为没有订单状态。

订单表记录 user_id，订单 id 订单编号，订单的总金额 order_status，支付方式，订单状态等。

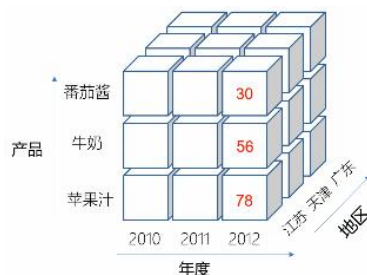
订单详情表记录 user_id，商品 sku_id，具体的商品信息（商品名称 sku_name，价格 order_price，数量 sku_num）

2.15.3 上卷和下钻

上卷：上卷是沿着维度的层次向上聚集汇总数据。

下探（钻）：下探是上卷的逆操作，它是沿着维度的层次向下，查看更详细的数据。

比如这个经典的数据立方体模型：



维度有产品、年度、地区等，统计销售额。实际上，维度还可以更细粒度，如时间维可由年、季、月、日构成，地区也可以由国家、省份、市、区县构成等。

下钻可以理解为由粗粒度到细粒度来观察数据，比如对产品销售情况分析时，可以沿着时间维从年到月到日更细粒度的观察数据。

| 产品名称 | 白米 | 白奶酪 | 饼干 | 糙米 | 大众奶酪 | 蛋糕 | 德国奶酪 | 番茄酱 | 干贝 | 光明奶酪 | 桂花糕 | 订 |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|------|
| 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订 |
| 2011 | 16660 | 3668 | 7621 | 2297 | 5153 | 971 | 2520 | 1400 | 1248 | 25091 | 11158 | 1810 |
| 2012 | 28348 | 5472 | 10421 | 7000 | 8421 | 3141 | 10336 | 1680 | 1768 | 51205 | 12393 | 4661 |
| 总计 | 45008 | 9140 | 18042 | 9297 | 13574 | 4112 | 12856 | 3080 | 3016 | 76296 | 23551 | 6 |

增加维度粒度“月”。

| 产品名称 | 白米 | 白奶酪 | 饼干 | 糙米 | 大众奶酪 | 蛋糕 | 德国奶酪 | 番茄酱 | 干贝 | 光明奶酪 | 桂花糕 | 订 |
|--------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|------|
| 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订单需求和 | 订 |
| 201102 | 60 | 150 | 1235 | 90 | 0 | 0 | 360 | 0 | 20 | 4400 | 2 | 2 |
| 201103 | 0 | 0 | 520 | 0 | 0 | 0 | 0 | 240 | 0 | 924 | 1 | 1 |
| 201104 | 720 | 1000 | 520 | 88 | 192 | 0 | 0 | 0 | 0 | 1760 | 0 | 0 |
| 201105 | 360 | 150 | 273 | 110 | 800 | 112 | 0 | 0 | 560 | 2552 | 0 | 0 |
| 201106 | 600 | 0 | 728 | 99 | 576 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 201107 | 1140 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 528 | 2 | 2 |
| 201108 | 3150 | 0 | 273 | 440 | 480 | 595 | 0 | 400 | 200 | 5060 | 0 | 0 |
| 201109 | 2490 | 0 | 1105 | 0 | 672 | 0 | 450 | 0 | 0 | 1584 | 0 | 0 |
| 201110 | 1500 | 0 | 689 | 550 | 480 | 210 | 0 | 160 | 0 | 7128 | 0 | 0 |
| 201111 | 5500 | 1600 | 850 | 920 | 0 | 54 | 0 | 0 | 0 | 220 | 0 | 0 |
| 201112 | 1140 | 768 | 1428 | 0 | 1953 | 0 | 1710 | 600 | 468 | 935 | 1 | 1 |
| 总计 | 16660 | 3668 | 7621 | 2297 | 5153 | 971 | 2520 | 1400 | 1248 | 25091 | 11158 | 1810 |
| 201201 | 3800 | 320 | 204 | 392 | 1050 | 225 | 0 | 0 | 0 | 2860 | 0 | 0 |
| 201202 | 1786 | 0 | 510 | 1400 | 294 | 135 | 0 | 140 | 0 | 4675 | 1 | 1 |

上卷和下钻是相逆的操作，所以上卷可以理解删掉维的某些粒度，由细粒度到粗粒度观察数据，向上聚合汇总数据。

2.15.4 TOB 和 TOC 解释

TOB (toBusiness): 表示面向的用户是企业。

TOC (toConsumer): 表示面向的用户是个人。

2.15.5 流转 G 复活指标

1) 活跃

日活: 100 万 ; 月活: 是日活的 2-3 倍 300 万

总注册的用户多少? 1000 万-3000 万之间。

渠道来源: app 公众号 抖音 百度 36 氪 头条 地推

2) GMV

GMV: 每天 10 万订单 (50-100 元) 500 万-1000 万

10%-20% 100 万-200 万 (人员: 程序员、人事、行政、财务、房租、收电费)

3) 复购率

某日常商品复购：（手纸、面膜、牙膏）10%-20%

电脑、显示器、手表 1%

4) 转化率

商品详情 =》 加购物车 =》 下单 =》 支付
1%-5% 50-60% 80%-95%

5) 留存率

1/2/3-60 日、周留存、月留存

搞活动： 10-20%

2.15.6 活动的话，数据量会增加多少？怎么解决？

日活增加 50%，GMV 增加多少 20%。（留转 G 复活）情人节，促销手纸。

集群资源都留有预量。11.11，6.18，数据量过大，提前动态增加服务器。

加多少机器：3-4 台

2.15.7 哪个商品卖的好？

面膜、手纸，每天销售 5000 个。下载 APP 根据自身业务

2.15.8 数据仓库每天跑多少张表，大概什么时候运行，运行多久？

基本一个项目建一个库，表格个数为初始的原始数据表格加上统计结果表格的总数。（一般 70-100 张表格）。

用户行为 5 张；业务数据 33 张表 =》 ods34 =》 dwd=>32 张=》 dws 22 张宽表=>ads=》
15 张 =》 103 张。

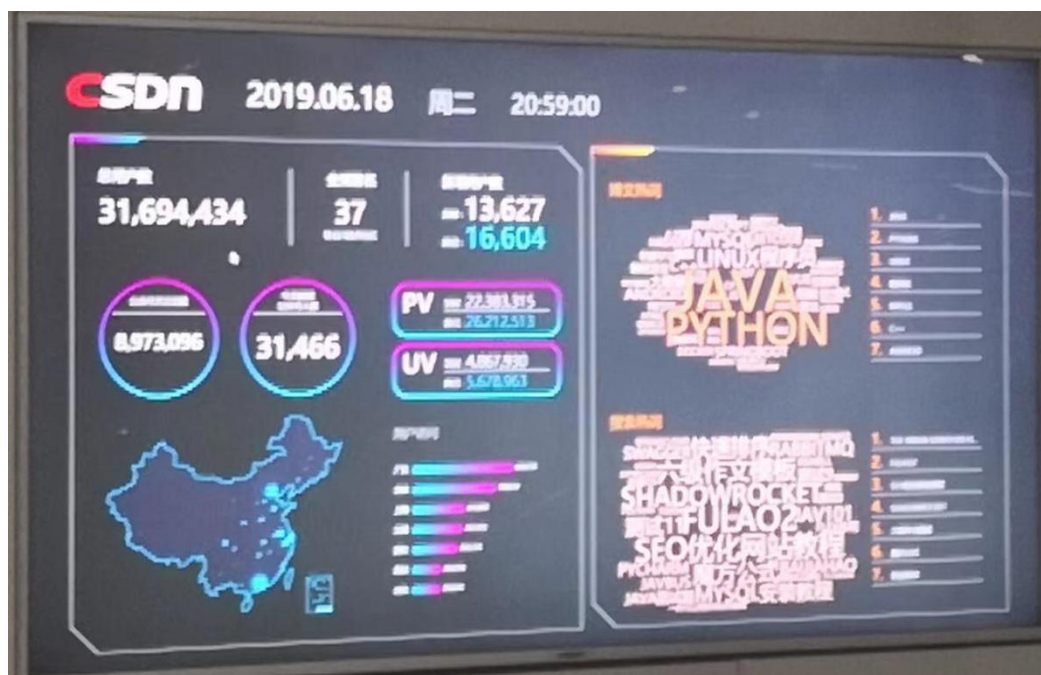
Datx: 00:10 => 10-20 分钟左右 第一次全量。

用户行为数据，每天 0: 30 开始运行。=》 ds =》 5-6 个小时运行完指标。

所有离线数据报表控制在 8 小时之内。

大数据实时处理部分控制在 5 分钟之内。（分钟级别、秒级别）

如果是实时推荐系统，需要秒级响应。



2.15.9 哪张表数据量最大

1) 用户行为数据

曝光 (60g or 6000 万条)、页面 (20g)

2) 业务数据 (1-2g => 100 万-200 万条)

登录（20 万）、注册（100-1000）；

加购（20 万）、下单（10 万）

用户（活跃用户 100 万、新增 1000、总用户 1 千万）

商品 SKU（10 万-20 万）

2.15.10 哪张表最费时间，有没有优化

最费时间，一般是发生数据倾斜时，会比较费时间。

1) Group By

（1）统计各个省份对应的交易额

第一个统计完的指标和最后一个统计完是时间相差 20 倍

我们从 Yarn 上看到的

一共执行了多长时间 4-5 小时

你想：发生了数据倾斜 任务停止掉

（2）解决办法：

①开启 map-side 预聚合

②skewindata

解决后的效果怎么样？

30-50 分钟内执行完了

2) Join

统计 事实表 和维度表 join => mapjoin

（1）小表 大表 join mapjoin

解决办法： mapjoin

（2）大表 =》 大表 join

项目中什么出现 统计 加购到支付的平均使用时长

执行时间 4-5 小时 yarn

①： skewjoin

②： smbjoin 分桶有序 join 使用的前提（分桶且有序）

③： 左表随机 右表扩容

④： 通过建模 规避 大表 join 大表

累积型快照事实表

2.15.11 并发峰值多少？大概哪个时间点？

高峰期晚上 7-12 点。Kafka 里面 20m/s 2 万/s 并发峰值在 1-2 万人

2.15.12 分析过最难的指标

- 路径分析
- 用户留存率
- 最近 7/30 日各品牌复购率
- 7 天内连续 3 天登录
- 每分钟同时在线人数
- 自己扩展

2.15.13 数仓中使用的哪种文件存储格式

常用的包括：textFile，ORC，Parquet，一般企业里使用 ORC 或者 Parquet，因为是列式存储，且压缩比非常高，所以相比于 textFile，**查询速度快，占用硬盘空间少。**

2.15.14 数仓当中数据多久删除一次

(1) 部分公司永久不删

(2) 有一年、两年“删除”一次的，这里面说的删除是，先将超时数据压缩下载到单独安装的磁盘上。然后删除集群上数据。很少有公司不备份数据，直接删除的。

2.15.15 Mysql 业务库中某张表发生变化，数仓中表需要做什么改变

修改表结构，将新增字段放置最后！

2.15.16 50 多张表关联，如何进行性能调优

详细看第一章 Hive 多表 join 优化手段。

大小表 join 大表 join 大表。

2.15.17 拉链表的退链如何实现

拉链表用于记录维度表中的历史变化。在拉链表中，当某个维度属性发生变化时，会插入一条新的记录，同时将原记录的有效期设置为截至。退链是指将一个已经生效的变更恢复到上一个状态。实现思路如下：

(1) 定位要退链的记录：例如找到用户最近一次信息更新

(2) 查询上一条记录：查询这条记录之前的一条记录（主键相同，不同版本记录，而不是单指上一条）

(3) 更新有效期：将当前记录的生效时间或者有效开始时间更新为无效，将上条记录截至日期改为最大值。

2.15.18 离线数仓如何补数

补数：指重新处理一段历史时间范围内的数据，以修复数据问题。

利用调度框架，海豚调度器的补数功能进行补数。

2.15.19 产品经理给新指标，该如何开发

(1) 确定需求：与产品经理和业务部门沟通

(2) 数据源分析：分析现有数据源确定是否可以满足新指标计算需求，如果不能支撑需要引入新的数据源或者扩展现有数据源

(3) 数据模型设计：根据新指标，设计模型

(4) 数据处理流程设计：利用 sql 进行数据的提取、清洗、转化和加载，实现指标

2.15.20 新出指标，原有建模无法实现，如何操作

引入新的数据源或者扩展现有数据源

2.15.21 和哪些部门沟通，以及沟通什么内容

产品经理：需求、统一业务口径

后端开发：数据源以及存储，数据格式，业务逻辑

前端开发：数据可视化要求，用户体验

测试人员：测试用例 bug

2.15.22 你们的需求和指标都是谁给的

产品经理

2.15.23 业务场景：时间跨度比较大，数据模型的数据怎么更新的，

例如：借款，使用一年，再还款，这个数据时间跨度大，在处理的时候怎么处理

累积型快照事实表。

2.15.24 map 任务从 0%到 1%很慢，但是从 1%到 100%很快，期间出了什么问题？

- 1) 申请资源
- 2) 加载数据
- 3) 如果存在于外部系统交互，获取连接慢

2.15.25 数据倾斜场景除了 group by 和 join 外，还有哪些场景

- (1) 使用 Snappy 压缩，原始文件大小不等，Map 阶段数据倾斜。
- (2) over 开窗，partition by 导致数据倾斜。

2.15.26 没有意义的用户是怎么判断的

在离线数据仓库中，没有意义的用户通常指那些对业务价值贡献极低或没有贡献的用户。这类用户可能是僵尸用户、垃圾用户或恶意用户。

(1) **低活跃度用户**：这类用户在平台上的活跃度极低，可能很长时间没有登录或者几乎没有操作。可以通过用户登录频率、活动次数等指标来定义低活跃度用户。

(2) **低价值用户**：这类用户对平台的业务价值贡献很低，例如在电商平台上从未进行过购买，或者在内容平台上几乎没有观看、点赞和评论等操作。可以通过消费金额、互动次数等指标来定义低价值用户。

(3) **僵尸用户**：这类用户可能是批量注册的账户或者长时间不活跃的用户。僵尸用户可能会导致平台数据泡沫，影响业务分析和决策。可以通过注册信息、活跃度和价值指标综合判断僵尸用户。

(4) **恶意用户**：这类用户可能会进行恶意操作，如发布垃圾信息、刷单、刷点击等。这些操作可能会对平台的数据准确性和用户体验产生负面影响。可以通过用户行为分析、异常检测等方法来识别恶意用户。

2.15.27 你的公司方向是电子商务，自营的还是提货平台？你们会有自己的商品吗？

我们的公司是自营的，有自己的商品，我们有自己经营的商品维度表，也有商品从采购到上架、发货、收货等一系列流程的业务数据和日志数据。

需要下载 APP 根据自身情况灵活回答！

2.15.28 ods 事实表中订单状态会发生变化，你们是通过什么方式去监测数据变化的

订单状态的改变会引起状态码的变更，这对应了不同的业务过程，可以被提取为不同的事务事实表，无论退款发生在哪个结点，支付成功、发货、收货这些业务过程都可以被记录在对应的事务事实表中。

2.15.29 当天订单没有闭环结束的数据量？

在离线数仓中，当天订单没有闭环结束的数据量通常指以下几种数据：

以当天 10 万订单为标准

(1) 未支付订单：指当天生成的订单，但客户还未完成支付行为的数据。

300-500

(2) 待发货订单：指当天已经收到付款的订单，但还未处理发货行为的数据。

6 万

(3) 退货订单：指当天客户发起的退货请求，但商家还未处理完退货流程的数据。

200-300

2.15.30 你们维度数据要做 ETL 吗？除了用户信息脱敏？没有做其他 ETL 吗

除了用户信息脱敏，我们还对用户埋点数据中的 IP、UA 等信息进行解析，同时我们对数据进行去重操作，还会对空值以及异常数据进行处理，对于部分维度数据我们将其合并。

2.15.31 怎么做加密，加密数据要用怎么办，我讲的 md5，他问我 md5 怎么做恢复

对于 MD5 加密算法，由于它是一种不可逆的散列算法，无法恢复原始数据。

2.15.32 表生命周期管理怎么做的？

表的生命周期管理是数据仓库和数据库管理的一个重要方面，需要关注表的创建、使用、优化和删除等阶段。管理表的生命周期有如下几个阶段：

1) 表设计

2) 数据导入和更新

- 3) 数据存储和备份
- 4) 数据安全和权限管理
- 5) 性能优化和监控
- 6) 数据归档和删除

2.15.33 当 ADS 计算完，如何判断指标是正确的

- (1) 样本数据验证：从计算结果抽取部分样本数据，与业务部门实际数据对比。
- (2) 逻辑验证：检查指标的计算 sql 是否正确
- (3) 指标间关系验证：比较不同指标间关系，检查它们是否符合预期。例如：某个指标是另外一个指标的累计值，那么这两个指标一定存在关系
- (4) 历史数据对比：将计算结果和过去数据进行对比，观察指标的变化趋势
- (5) 异常值检测：检查计算结果中是否存在异常值
- (6) 跨数据部门对比：可以将计算结果与其它部门或团队的数据进行对比，进一步验证

2.15.34 ADS 层指标计算错误，如何解决

- (1) 确定错误范围：找出指标计算错误的时间范围，指标及相关维度，缩小排查范围
- (2) 检查数据处理逻辑：从 ods 层开始排查，找出可能导致计算错误的数据清洗，转换和聚合等步骤，确认处理逻辑不出错误
- (3) 审查数据质量：确保每层数据完整性，一致性，准确性和时效性
- (4) 重新计算指标：修复数据质量问题和处理逻辑后，重新计算

2.15.35 任务跑起来之后，整个集群资源占用比例

数仓脚本是串行执行的，ods=》dim=》dwd=》dws=》ads

所以占用集群资源比例，仅为当前执行层的脚本转化成底层 Sparkjob 需要的资源

CPU 与内存比 1:4

离线: 128M 数据 512M 内存 (1 个 CPU 4G 内存 1g 数据)

实时: 并行度与 Kafka 分区一致，CPU 与 Slot 比 1:3

20M/s -> 3 个分区 -> CPU 与 Slot 比 1:3 -> 3 个 Slot -> Core 数 1 个 -> CPU 与内存比 1:4

-> TM 1 slot -> TM 4G 资源

JobManager 2G 内存 1CPU

平均 一个 Flink 作业 6G 内存，2Core

2.15.36 真实项目流程



01_尚硅谷在线教育数仓中台项目立项



02_尚硅谷在线教育数仓中台需求确认



03_尚硅谷在线教育数仓中台系统概要设计



尚硅谷大数据之数仓设计规范.docx

1) 数仓搭建完成之前：

- (1) 需求分析：与产品了解需求，明确数仓实现功能
- (2) 数据源分析：分析数据源是否充足，是否需要引进新数据源
- (3) 数据模型设计
- (4) 技术选型
- (5) 数据处理流程设计：Hivesql

2) 数仓搭建完成之后：

- (1) 开发与维护
- (2) 数据监控与报警
- (3) 数据质量管理
- (4) 性能优化
- (5) 新需求开发
- (6) 报表与可视化
- (7) 文档编写与知识分享

2.16 埋点

1) 埋点选择

免费的埋点：上课演示。前端程序员自己埋点。

收费的埋点：神策、百度统计、友盟统计。

2) 埋点方式主要有两种：

- (1) 按照页面埋点，有几个页面就创建几个表。
- (2) 按照用户行为：**页面数据、事件数据、曝光数据、启动数据和错误数据**。咱们项目中采用的这种方式。

3) 埋点数据日志格式

为了减少网络上数据的传输，日志格式设计时都会有公共信息。

```
{
  "common": {                                -- 公共信息
    "ar": "230000",                          -- 地区编码
    "ba": "iPhone",                          -- 手机品牌
    "ch": "Appstore",                        -- 渠道
    "md": "iPhone 8",                        -- 手机型号
    "mid": "YXfhjAYH6As2z9Iq",              -- 设备 id
    "os": "iOS 13.2.9",                      -- 操作系统
    "uid": "485",                            -- 会员 id
    "vc": "v2.1.134"                         -- app 版本号
  },
  "actions": [                               --动作(事件)
    {
      "action_id": "favor_add",              --动作 id
      "item": "3",                           --目标 id
      "item_type": "sku_id",                 --目标类型
      "ts": 1585744376605                    --动作时间戳
    }
  ],
  "displays": [
    {
      "displayType": "query",                -- 曝光类型
      "item": "3",                           -- 曝光对象 id
      "item_type": "sku_id",                 -- 曝光对象类型
      "order": 1                             --出现顺序
    },
    {
      "displayType": "promotion",
      "item": "6",
      "item_type": "sku_id",
      "order": 2
    },
    {
      "displayType": "promotion",
      "item": "9",
      "item_type": "sku_id",
      "order": 3
    }
  ],
  "page": {                                  --页面信息
    "during_time": 7648,                     -- 持续时间毫秒
    "item": "3",                             -- 目标 id
    "item_type": "sku_id",                   -- 目标类型
    "last_page_id": "login",                 -- 上页类型
    "page_id": "good_detail",               -- 页面 ID
    "sourceType": "promotion"                -- 来源类型
  },
  "err": {                                    --错误
    "error_code": "1234",                   --错误码
    "msg": "*****"                         --错误信息
  }
}
```

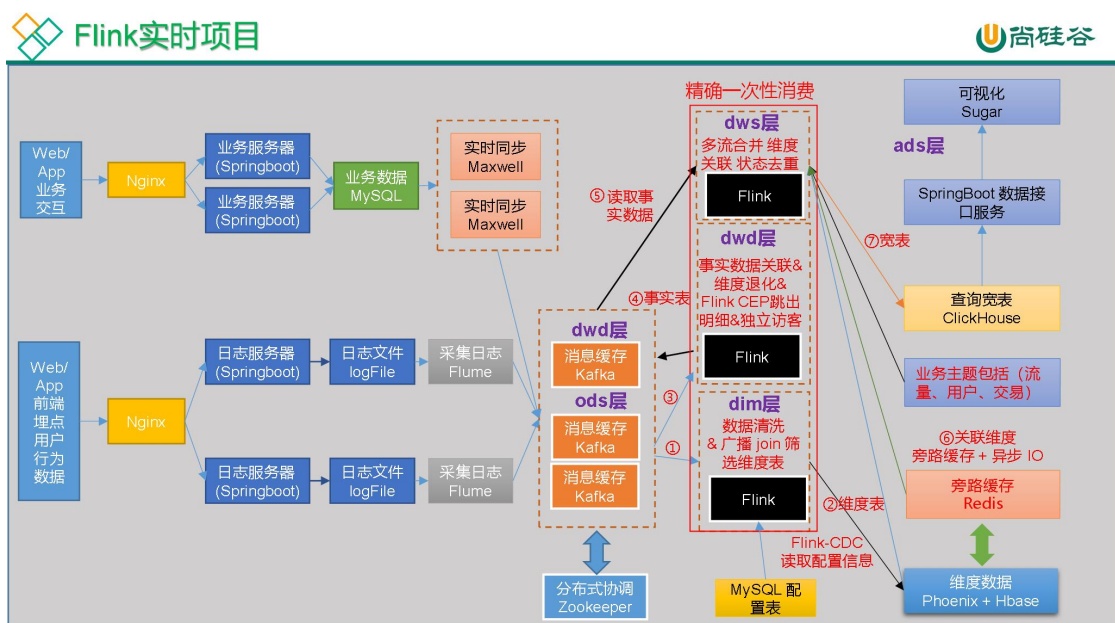
```
},
"ts": 1585744374423 --跳入时间戳
}
```

第 3 章 实时数仓项目

3.1 为什么做这个项目

随着公司不断业务不断发展，产品需求和内部决策对于数据实时性要求越来越迫切，传统离线数仓 T+1 模式已经不能满足，所以需要实时数仓的能力来赋能。

3.2 项目架构



3.3 框架版本选型

和离线保持一致。

3.4 服务器选型

和离线保持一致。

3.5 集群规模

1) 生产集群规模、Flink 集群规模（10 台为例）

项目中方便作业提交，Flink 作为客户端，部署在所有的 Worker 节点

举例：Job 数量在 20 左右，需要 10 台服务器

Clickhouse 单独部署，服务器使用 128G，64C

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|-------|-------|-------|-------|---------|-------|-------|-------|----|----|
| nn | nn | dn | dn | dn | dn | dn | dn | | |
| rm | rm | nm | nm | nm | nm | nm | nm | | |
| | | | | | zk | zk | zk | | |
| | | | | | Kafka | Kafka | Kafka | | |
| | | | | | Flume | Flume | Flume | | |
| Hive | Hive | | | | | | | | |
| MySQL | | | | | | | | | |
| Spark | Spark | | | | | | | | |
| DS | DS | | Datax | maxwell | | | | | |
| | | Hbase | Hbase | Hbase | | | | | |
| | | Flink | Flink | Flink | Flink | Flink | Flink | | |
| | | | | | | | | CK | CK |

3.6 项目建模

1) 数据调研

(1) 先和 Java 人员要表，表中最好有字段的描述或者有表和字段的说明文档。(项目经理帮助协调) => 快速熟悉表中业务。梳理清楚业务线，找到事实表和维度表。

(2) 和业务人员聊 => 验证你猜测的是否正确

(3) 和产品经理聊

需求：派生指标、衍生指标

派生指标 = 原子指标 (业务过程 + 度量值 + 聚合逻辑) + 统计周期 + 统计粒度 + 业务限定

需求中的业务过程必须和实际的后台业务能对应上。

2) 明确数据域

(1) 用户域：登录、注册

(2) 流量域：启动、页面、动作、故障、曝光

(3) 交易域：加购、下单、支付、物流

(4) 工具域：领取优惠券、使用优惠券下单、使用优惠券支付

(5) 互动域：点赞、评论、收藏

3) 构建业务矩阵

用户、商品、活动、时间、地区、优惠券

(1) 用户域:

登录、注册

(2) 流量域:

✓

启动、页面、动作、故障、曝光

(3) 交易域:

加购、下单、支付、物流

(4) 工具域:

领取优惠券、使用优惠券下单、使用优惠券支付

(5) 互动域:

点赞、评论、收藏

4) 建模 至下而上

(1) ODS 层

①存 Kafka: topic_log\topic_db ,保持数据原貌不做处理

(2) DWD 层 事实表

①事务型事实表

找原子操作

a) 选择业务过程

选择感兴趣的业务过程。 产品经理提出的指标中需要的。

b) 声明粒度

粒度: 一行信息代表什么含义。可以是一次下单、一周下单、一个月下单。

如果是一个月的下单,就没有办法统计一次下单情况。保持最小粒度。

只要你自己不做聚合操作就可以。

c) 确定维度

确定感兴趣的维度。 产品经理提出的指标中需要的。

例如: 用户、商品、活动、时间、地区、优惠券

d) 确定事实

确定事实表的度量值。 可以累加的值, 例如, 个数、件数、次数、金额。

e) 维度退化

通过 Lookupjoin 将字典表中字段退化到明细表中

(3) DIM 层 维度表

①维度数据存储 Hbase，同时不做维度整合

5) 指标体系建设 至上而下

(1) ADS 层

需求、日活、新增、留存、转化率、GMV

(2) DWS 层 聚合层

需求：派生指标、衍生指标

派生指标 = 原子指标（业务过程 + 度量值 + 聚合逻辑） + 统计周期 + 统计粒度 + 业务限定

例如，统计，每天各个省份手机品牌交易总额

交易总额 （下单 + 金额 + sum） + 每天 + 省份 + 手机品牌

找公共的：业务过程 + 统计周期 + 统计粒度 建宽表

3.7 数据量

3.7.1 数据分层数据量

1) ODS 层

(1) 用户行为数据（100g => 1 亿条；1g => 100 万条）

曝光（60g or 600 万条）、页面（20g）、动作（10g）、故障 + 启动（10g）

(2) 业务数据（1-2g => 100 万-200 万条）

登录（20 万）、注册（100-1000）；

加购（每天增量 20 万、全量 100 万）、下单（10 万）、支付（9 万）、物流（9 万）、取消下单（500）、退款（500）；

领取优惠券（5 万）、使用优惠券下单（4 万）、使用优惠券支付（3 万）；

点赞（1000）、评论（1000）、收藏（1000）；

用户（活跃用户 100 万、新增 1000、总用户 1 千万）、商品 SPU（1-2 万）、商品 SKU（10-20 万）、活动（1000）、时间（忽略）、地区（忽略）

2) DWD 层 + DIM 层

和 ODS 层几乎一致；

3) DWS 层

轻度聚合后，20g-50g。

4) ADS 层

10-50m 之间，可以忽略不计。

3.7.2 实时组件存储数据量

1) Kafka:

ods 层和 dwd 层数据

ods 和 dwd 数据一致，每天约 200G 数据

考虑 kafka 副本 2 个，保存三天，kafka 存储 400G 数据

2) Hbase:

存储 dim 层数据，与离线一致

3) Clickhouse:

存储 dws 层数据，每天约 20~30G 数据

考虑 dws 层数据保存一年，Clickhouse 三个副本

数据约 15~20T

3.7.3 实时 QPS 峰值数据量

QPS 峰值：20000 条/s 或者 2M/s

3.8 项目中遇到哪些问题及如何解决？

3.8.1 业务数据采集框架选型问题

详见第一章（FlinkCDC，Maxwell，Canal）对比

3.8.2 项目中哪里用到状态编程，状态是如何存储的，怎么解决大状态问题

1) Dim 动态分流使用广播状态，新老访客修复使用键控状态

状态中数据少使用 HashMap，状态中数据多的使用 RocksDB

2) 大状态优化手段

(1) 使用 rocksdb

(2) 开启增量检查点、本地恢复、设置多目录

(3) 设置预定义选项为 磁盘+内存 的策略，自动设定 writerbuffer、blockcache 等

3.8.3 项目中哪里遇到了反压，造成的危害，定位解决 (*重点*)

1) 项目中反压造成的原因

流量洪峰：不需要解决

频繁 GC：比如代码中大量创建临时对象

大状态：新老访客修复

关联外部数据库：从 Hbase 读取维度数据或将数据写入 Clickhouse

数据倾斜：keyby 之后不同分组数据量不一致

2) 反压的危害

问题：Checkpoint 超时失败导致 job 挂掉

内存压力变大导致的 OOM 导致 job 挂掉

时效性降低

3) 定位反压

(1) 利用 Web UI 定位

定位到造成反压的节点，排查的时候，先把 operator chain 禁用，方便定位到具体算子。

Flink 现在在 UI 上通过颜色和数值来展示繁忙和反压的程度。

上游都是 high，找到第一个为 ok 的节点就是瓶颈节点。

(2) 利用 Metrics 定位

可以根据指标分析反压： buffer.inPoolUsage、buffer.outPoolUsage

可以分析数据传输

4) 处理反压

反压可能是暂时的，可能是由于负载高峰、CheckPoint 或作业重启引起的数据积压而导致反压。如果反压是暂时的，应该忽略它。

(1) 查看是否数据倾斜

(2) 使用火焰图分析看顶层的哪个函数占据的宽度最大。只要有"平顶" (plateaus)，就表示该函数可能存在性能问题。

(3) 分析 GC 日志，调整代码

(4) 资源不合理造成的：调整资源

(5) 与外部系统交互:

写 MySQL、Clickhouse: 攒批写入

读 HBase: 异步 IO、旁路缓存

3.8.4 数据倾斜问题如何解决 (****重点****)

1) 数据倾斜现象:

相同 Task 的多个 Subtask 中, 个别 Subtask 接收到的数据量明显大于其他 Subtask 接收到的数据量, 通过 Flink Web UI 可以精确地看到每个 Subtask 处理了多少数据, 即可判断出 Flink 任务是否存在数据倾斜。通常, 数据倾斜也会引起反压。

2) 数据倾斜解决

(1) 数据源倾斜

比如消费 Kafka, 但是 Kafka 的 Topic 的分区之间数据不均衡

读进来之后调用重分区算子: rescale、rebalance、shuffle 等

(2) 单表分组聚合 (纯流式) 倾斜

API: 利用 flatmap 攒批、预聚合

SQL: 开启 MiniBatch+LocalGlobal

(3) 单表分组开窗聚合倾斜

第一阶段聚合: key 拼接随机数前缀或后缀, 进行 keyby、开窗、聚合

注意: 聚合完不再是 WindowedStream, 要获取 WindowEnd 作为窗口标记作为第二阶段分组依据, 避免不同窗口的结果聚合到一起)

第二阶段聚合: 按照原来的 key 及 windowEnd 作 keyby、聚合

在我们项目中, 用到了 Clickhouse, 我们可以第一阶段打散聚合后, 直接写入 Click house, 查 clickhouse 再处理第二阶段

3.8.5 数据如何保证一致性问题

上游: kafka 保证 offset 可重发, kafka 默认实现

Flink: Checkpoint 设置执行模式为 Exactly_once

下游: 使用事务写入 Kafka, 使用幂等写入 Clickhouse 且查询使用 final 查询

3.8.6 FlinkSQL 性能比较慢如何优化

(1) 设置空闲状态保留时间

- (2) 开启 MiniBatch
- (3) 开启 LocalGlobal
- (4) 开启 Split Distinct
- (5) 多维 Distinct 使用 Filter

3.8.7 Kafka 分区动态增加，Flink 监控不到新分区数据导致数据丢失

设置 Flink 动态监控 kafka 分区的参数

3.8.9 Kafka 某个分区没有数据，导致下游水位线无法抬升，窗口无法关闭计算

注入水位线时，设置最小等待时间

3.8.10 Hbase 的 rowkey 设计不合理导致的数据热点问题

详见 Hbase 的 rowkey 设计原则

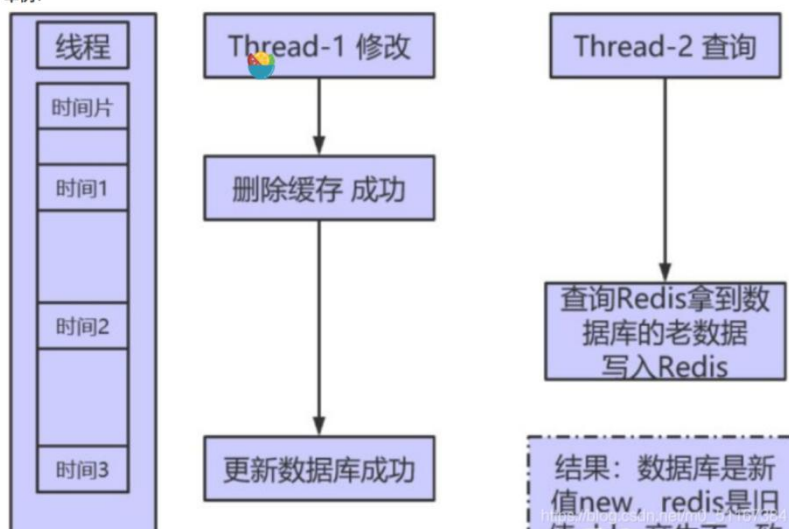
3.8.11 Redis 和 HBase 的数据不一致问题

对 Redis 和数据库的操作有 2 种方案：

- (1) 先操作（删除）Redis，再操作数据库

并发下可能产生数据一致性问题。

举例：



上面的图表示，Thread-1 是个更新流程，Thread-2 是个查询流程，CPU 执行顺序是：

Thread-1 删除缓存成功，此时 Thread-2 获取到 CPU 执行查询缓存没有数据，然后查询数据库把数据库的值写入缓存，因为此时 Thread-1 更新数据库还没有执行，所以缓存里的值

是一个旧值 (old)，最后 CPU 执行 Thread-1 更新数据库成功的代码，那么此时数据库的值是新增 (new)，这样就产生了数据不一致行的问题。

解决上述问题的两种方案：

①加锁，使线程顺序执行：如果一个服务部署到了多个机器，就变成了分布式锁，或者是分布式队列按顺序去操作数据库或者 Redis，带来的副作用就是：数据库本来是并发的，现在变成串行的了，加锁或者排队执行的方案降低了系统性能，所以这个方案看起来不太可行。

②采用双删：先删除缓存，再更新数据库，当更新数据后休眠一段时间再删除一次缓存。

(2) 先操作数据库，再操作 (删除) Redis

我们如果更新数据库成功，删除 Redis 失败，那么 Redis 里存放的就是一个旧值，也就是删除缓存失败导致缓存和数据库的数据不一致了

上述二种方案，都希望数据操作要么都成功，要么都失败，也就是最好是一个原子操作，我们不喜欢看到一个失败，一个成功的结果，因为这样就产生了数据不一致的问题。

3.8.12 双流 join 关联不上如何解决

(1) 使用 interval join 调整上下限时间，但是依然会有迟到数据关联不上

(2) 使用 left join，带回撤关联

(3) 可以使用 Cogroup+connect 关联两条流

3.9 生产经验

3.9.1 Flink 任务提交使用那种模式，为何选用这种模式

项目中提交使用的 per-job 模式，因为每个 job 资源隔离、故障隔离、独立调优

3.9.2 Flink 任务提交参数，JobManager 和 TaskManager 分别给多少

JobManager: 内存默认 1G，cpu 默认 1 核

TaskManager: 数据量多的 job，例如：Topic_log 分流的 job 可以给 8G

数据量少的 job，例如：Topic_db 分流的 job 可以给 4G

实时:并行度与 Kafka 分区一致，CPU 与 Slot 比 1: 3

20M/s -> 3 个分区 -> CPU 与 Slot 比 1:3 -> 3 个 Slot -> Core 数 1 个 -> CPU 与内存比 1:4
-> TM 1 slot -> TM 4G 资源

JobManager 2G 内存 1CPU

平均 一个 Flink 作业 6G 内存, 2Core

3.9.3 Flink 任务并行度如何设置

全局并行度设置和 kafka 分区数保持一致为 5, Keyby 后计算偏大的算子, 单独指定。

3.9.4 项目中 Flink 作业 Checkpoint 参数如何设置

Checkpoint 间隔: 作业多久触发一次 Checkpoint, 由 job 状态大小和恢复调整, 一般建议 **3~5 分钟**, 时效性要求高的可以设置 s 级别。

Checkpoint 超时: 限制 Checkpoint 的执行时间, 超过此时间, Checkpoint 被丢弃, 建议 10 分钟。

Checkpoint 最小间隔: 避免 Checkpoint 过于频繁, 可以设置分钟级别。

Checkpoint 的执行模式: Exactly_once 或 At_least_once, 选择 Exactly_once。

Checkpoint 的存储后端: 一般存储 HDFS。

3.9.5 迟到数据如何解决

- (1) 设置乱序时间
- (2) 窗口允许迟到时间
- (3) 侧输出流

生产中侧输出流, 需要 Flink 单独处理, 在写入 Clickhouse, 通过接口再次计算

3.9.6 实时数仓延迟多少

反压, 状态大小, 资源偏少, 机器性能, checkpoint 时间都会影响数仓延迟。

一般影响最大就是窗口大小, 一般是 5s。

如果启用两阶段提交写入 Kafka, 下游设置读已提交, 那么需要加上 CheckPoint 间隔时间。

3.9.7 项目开发多久, 维护了多久

开发周期半年, 维护半年多

3.9.8 如何处理缓存冷启动问题

初次启动, Redis 没有缓存数据, 大量读请求访问 Habse, 类似于缓存雪崩

从离线统计热门维度数据, 最近三天用户购买, 活跃的 sku, 手动插入 Redis。

3.9.9 如何处理动态分流冷启动问题（主流数据先到，丢失数据怎么处理）

在 Open 方法中预加载配置信息到 HashMap 以防止配置信息后到。

3.9.10 代码升级，修改代码，如何上线

Savepoint 停止程序，通过 Savepoint 恢复程序。

代码改动较大，savepoint 恢复不了怎么办，看历史数据要不要，要从头跑，不要就不适用 savepoint 恢复直接提交运行。

3.9.11 如果现在做了 5 个 Checkpoint，Flink Job 挂掉之后想恢复到第三次 Checkpoint 保存的状态上，如何操作

在 Flink 中，我们可以通过设置 `externalized-checkpoint` 来启用外部化检查点，要从特定的检查点（例如第三个检查点）恢复作业，我们需要手动指定要从哪个检查点（需要指定到 `chk-xx` 目录）恢复。

3.9.12 需要使用 flink 记录一群人，从北京出发到上海，记录出发时间和到达时间，同时要显示每个人用时多久，需要实时显示，如果让你来做，你怎么设计？

按照每个人 KeyBy，将出发时间存入状态，当到达时使用到达时间减去出发时间。

3.9.13 flink 内部的数据质量和数据的时效怎么把控的

内部的数据质量：内部一致性检查点

数据的时效：结合 3.9.5 回答

3.9.14 实时任务问题（延迟）怎么排查

实时任务出现延迟时，可以从以下几个方面进行排查：

- （1）**监控指标**：看是否反压
- （2）**日志信息**：查看任务运行时的日志信息，定位潜在的问题和异常情况。例如，网络波动、硬件故障、不当的配置等等。
- （3）**外部事件**：如果延迟出现在大量的外部事件后，则可能需要考虑其他因素（如外部系统故障、网络波动等）。**框架混部，资源争抢！**

3.9.15 维度数据查询并发量

未做优化之前，有几千 QPS，做完 Redis 的缓存优化，下降到几十

3.9.16 Prometheus+Grafana 是自己搭的吗，监控哪些指标

是我们自己搭建的，用来监控 Flink 任务和集群的相关指标

- 1) TaskManager Metrics: 这些指标提供有关 TaskManager 的信息，例如 CPU 使用率、内存使用率、网络 IO 等。
- 2) Task Metrics: 这些指标提供有关任务的信息，例如任务的延迟时间、记录丢失数、输入输出速率等。
- 3) Checkpoint Metrics: 这些指标提供有关检查点的信息，例如检查点的持续时间、成功/失败的检查点数量、检查点大小等。
- 4) Operator Metrics: 这些指标提供有关 Flink 操作符的信息，例如操作符的输入/输出记录数、处理时间、缓存大小等。

3.9.17 怎样在不停止任务的情况下改 flink 参数

动态分流，其他没有做过！

3.9.18 hbase 中有表，里面的 1 月份到 3 月份的数据我不要了，我需要删除它（彻底删除），要怎么做

在 HBase 中彻底删除表中的数据，需要执行以下步骤：

- (1) 禁用表
- (2) 创建一个新表
- (3) 复制需要保留的数据，将需要保留的数据从旧表复制到新表。
- (4) 删除旧表
- (5) 重命名新表

在执行这些步骤之前，建议先进行数据备份以防止意外数据丢失。此外，如果旧表中的数据量非常大，复制数据到新表中的过程可能会需要很长时间。

3.9.19 如果 flink 程序的数据倾斜是偶然出现的，可能白天可能晚上突然出现，然后几个月都没有出现，没办法复现，怎么解决？

Flink 本身存在反压机制，短时间数据倾斜问题可以自身消化掉，所以针对于这种偶然性数据倾斜，不做处理。

3.9.20 维度数据改变之后，如何保证新 join 的维度数据是正确的数据

(1) 我们采用的是低延迟增量更新，本身就有延迟，没办法保证完全的正确数据。

(2) 如果必须要正确结果，只能直接读取 MySQL 数据，但是需要考虑并发，MySQL 机器性能。

3.10 实时---业务

3.10.1 数据采集到 ODS 层

1) 前端埋点的行为数据为什么又采集一份？

时效性

Kafka 保存 3 天，磁盘够：原来 1T，现在 2T，没压力

2) 为什么选择 Kafka？

实时写、实时读

=> 消息队列适合，其他数据库受不了

3) 为什么用 Maxwell？历史数据同步怎么保证一致性？

FlinkCDC 在 20 年 7 月才发布

Canal 与 Maxwell 区别：

Maxwell 支持同步历史数据

Maxwell 支持断点还原（存在元数据库）

数据格式更轻量

保证至少一次，不丢

4) Kafka 保存多久？如果需要以前的数据怎么办？

跟离线项目保持一致：3 天

我们的项目不需要，如果需要的话可以去数据库或 Hive 现查，ClickHouse 也有历史的

宽表数据。

3.10.2 ODS 层

1) 存储原始数据

2 个 topic: 埋点的行为数据 ods_base_log、业务数据 ods_base_db

2) 业务数据的有序性:

maxwell 配置, 指定生产者分区的 key 为 table。

3.10.3 DWD+DIM 层

1) 存储位置, 为什么维度表存 HBase?

事实表存 Kafka、维度表存 HBase

基于热存储加载维表的 Join 方案:

随机查

长远考虑

适合实时读写

2) 埋点行为数据分流

(1) 修复新老访客 (选择性): 以前是前端试别新老访客, 不够准确

(2) 分流: 侧输出流

分了 3 个 topic: 启动、页面、曝光

(3) 用户跳出、独立访客统计

3) 业务数据处理

(1) 动态分流: FlinkSQL 读取 topic_base_db 数据, 过滤出每张明细表写回 kafka

(2) 订单预处理表设计: 双流 join, 使用 leftjoin

(3) 字典表维度退化

4) 维度数据写入 Hbase

(1) 为了避免维度数据发生变化而重启任务, 在 mysql 存一张配置表来动态配置。

动态实现: 通过广播状态

==》 读取一张配置表 ==》 维护这张配置表

source 来源 sink 写到哪 操作类型 字段 主键 扩展

==》 实时获取配置表的变化 ==》 CDC 工具

=> FlinkCDC

=> 使用了 sql 的方式，去同步这张配置表

=> sql 的数据格式比较方便

(2) 怎么写 HBase: 借助 phoenix

没有做维度退化

维表数据量小、变化频率慢

(3) Hbase 的 rowkey 怎么设计的? 有没有数据热点问题?

最大的维表: 用户维表

=> 百万日活, 2000 万注册用户为例, 1 条平均 1k: $2000 \text{ 万} * 1\text{k} = \text{约 } 20\text{G}$

使用 Phoenix 创建的盐表, 避免数据热点问题

<https://developer.aliyun.com/article/532313>

3.10.4 DWS 层

1) 为什么选择 ClickHouse

(1) 适合大宽表、数据量多、聚合统计分析 => 快

(2) 宽表已经不再需要 Join, 很合适

2) 关联维度数据

(1) 维度关联方案: 预加载、读取外部数据库、双流 Join、LookupJoin

(2) 项目中读取 Hbase 中维度数据

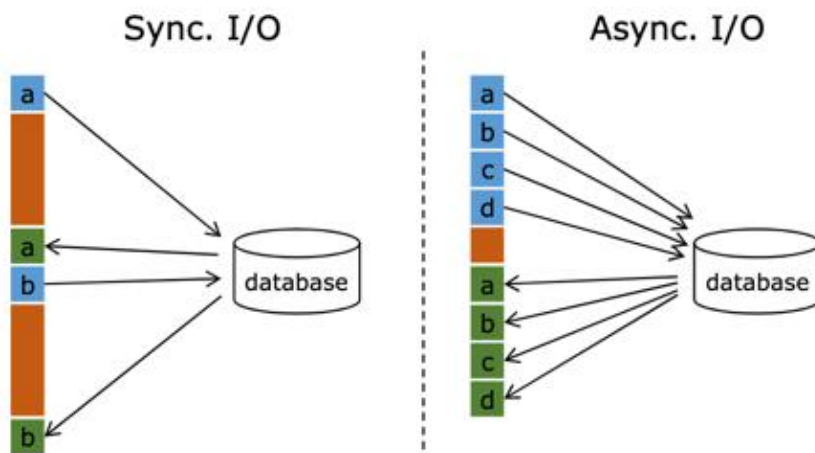
(3) 优化 1: 异步 IO

异步查询实际上是把维表的查询操作托管给单独的线程池完成, 这样不会因为某一个查询造成阻塞, 单个并行可以连续发送多个请求, 提高并发效率。

这种方式特别针对涉及网络 IO 的操作, 减少因为请求等待带来的消耗。

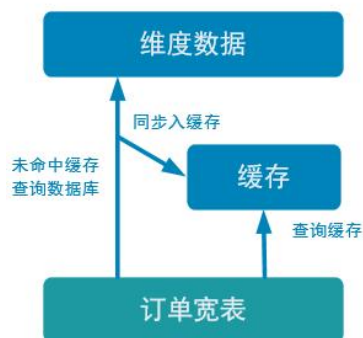
Flink 在 1.2 中引入了 Async I/O, 在异步模式下, 将 IO 操作异步化, 单个并行可以连续发送多个请求, 哪个请求先返回就先处理, 从而在连续的请求间不需要阻塞式等待, 大大提高了流处理效率。

Async I/O 是阿里巴巴贡献给社区的一个呼声非常高的特性, 解决与外部系统交互时网络延迟成为了系统瓶颈的问题。



(4) 优化 2: 旁路缓存

旁路缓存模式是一种非常常见的按需分配缓存的模式。如图，任何请求优先访问缓存，缓存命中，直接获得数据返回请求。如果未命中则，查询数据库，同时把结果写入缓存以备后续请求使用。



(5) 怎么保证缓存一致性

方案 1: 当我们获取到维表更新的数据，也就是拿到维度表操作类型为 `update` 时:

更新 Hbase 的同时，删除 redis 里对应的之前缓存的数据

Redis 设置了过期时间: 24 小时

方案 2: 双写

3) 轻度聚合

(1) DWS 层要应对很多实时查询，如果是完全的明细那么查询的压力是非常大的。将更多的实时数据以主题的方式组合起来便于管理，同时也能减少维度查询的次数。

(2) 开一个小窗口，5s 的滚动窗口

(3) 同时减轻了写 ClickHouse 的压力，减少后续聚合的时间

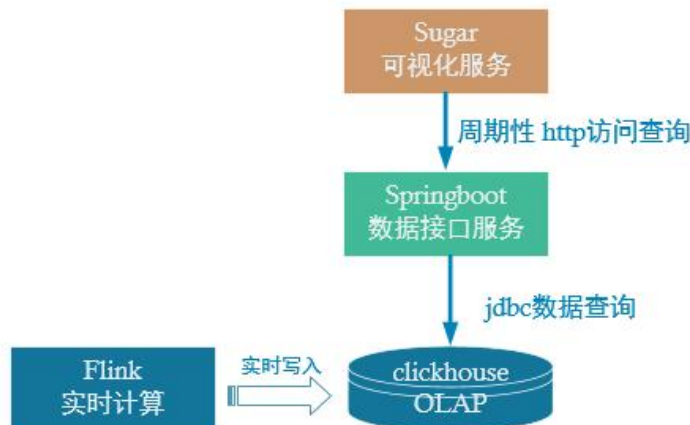
(4) 几张表? 表名、字段

访客、商品、地区、关键词

3.10.5 ADS 层

1) 实现方案

为可视化大屏服务，提供一个数据接口用来查询 ClickHouse 中的数据。

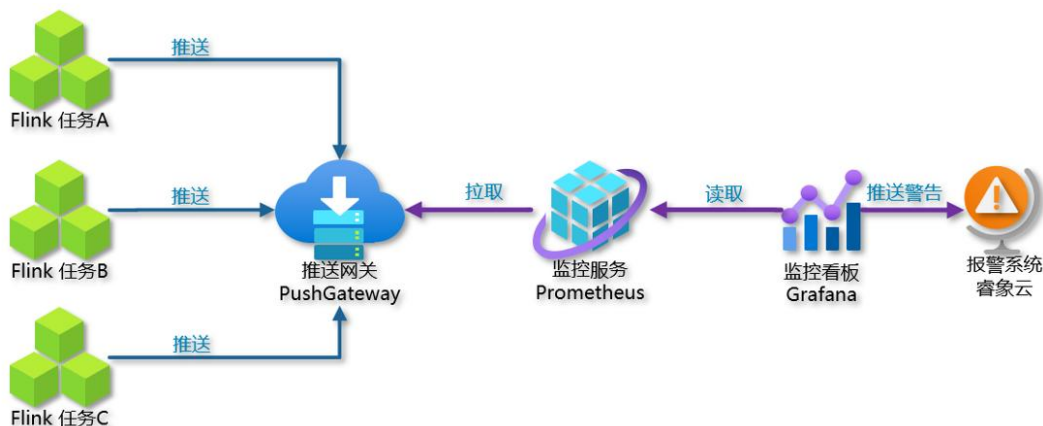


2) 怎么保证 ClickHouse 的一致性?

ReplacingMergeTree 只能保证最终一致性，查询时的 sql 语法加上去重逻辑。

3) Flink 任务如何监控

Flink 和 ClickHouse 都使用了 Prometheus + Grafana。



第 4 章 用户画像项目

4.1 画像系统主要做了哪些事

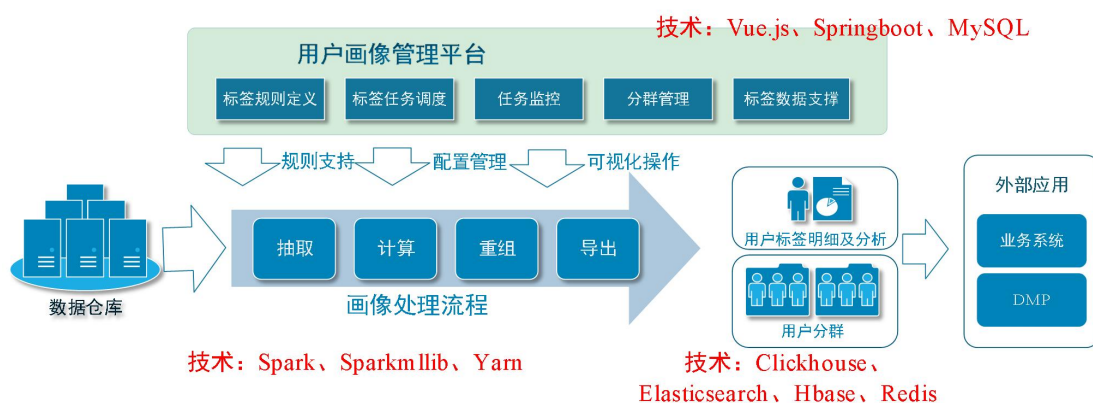
- 1) 用户信息标签化
- 2) 对标签化的数据的应用（分群、洞察分析）
- 3) 标签如何建模的，有哪些标签

根据用户需求，协调产品经理一起规划了四级标签。前两级是分类，第三级是标签，第四级是标签值。

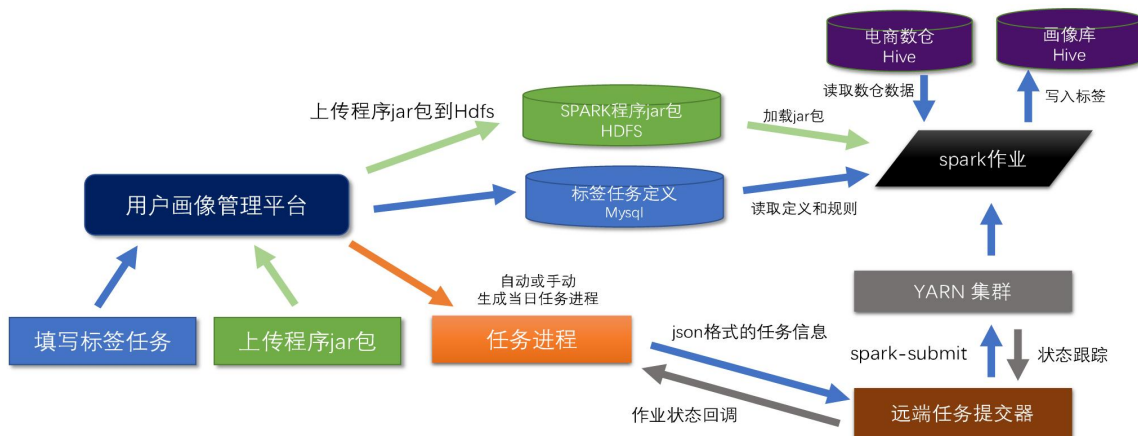


电商画像标签2022
.xlsx

4.2 项目整体架构



4.3 讲一下标签计算的调度过程



4.4 整个标签的批处理过程

四个任务：

- (1) 通过根据每个标签的业务逻辑编写 SQL，生产标签单表。
- (2) 把标签单表合并为标签宽表。

(3) 把标签宽表导出到 Clickhouse 中的标签宽表。

(4) 把 Clickhouse 中的标签表转储为 Bitmap 表。

四个任务通过编写 Spark 程序完成。并通过画像平台调度，以后新增标签只需要在平台填写标签定义、SQL 及相关参数即可。

4.5 你们的画像平台有哪些功能？

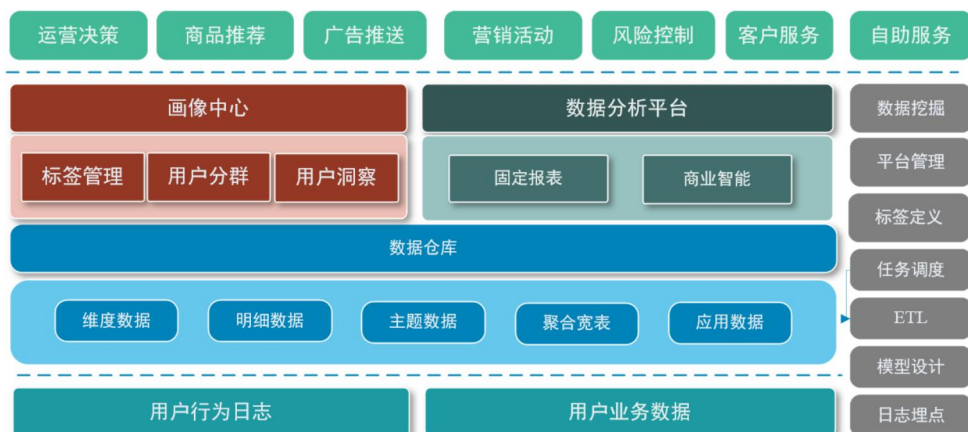
- (1) 标签定义
- (2) 标签任务设定
- (3) 任务调度
- (4) 任务监控
- (5) 分群创建维护
- (6) 人群洞察

4.6 是否做过 Web 应用开发，实现了什么功能

- (1) 画像平台 分群
- (2) 画像平台 其他功能（可选）
- (3) 实时数仓 数据接口

4.7 画像平台的上下游

- (1) 上游：数仓系统
- (2) 下游：写入到 Redis 中，由广告、运营系统访问。



4.8 BitMap 原理，及为什么可以提高性能

Bitmap 是一个二进制集合，用 0 或 1 标识某个值是否存在。

在求两个集合的交集运算时，不需要遍历两个集合，只要对位进行与运算即可。无论是比较次数的降低（从 $O(N^2)$ 到 $O(N)$ ），还是比较方式的改善（位运算），都给性能带来巨大的提升。

业务场景：把每个标签的用户 id 集合放在一个 Bitmap 中，那多个标签求交集（比如：女性 + 90 后）这种分群筛选时，就可以通过两个标签的 Bitmap 求交集运算即可。

第 5 章 数据湖项目

5.1 数据湖与数据仓库对比

数据湖（Data Lake）是一个存储企业的各种各样原始数据的大型仓库，其中的数据可供存取、处理、分析及传输。

Hudi、Iceberg、Data Lake

5.2 为什么做这个项目？解决了什么痛点？

（1）离线数仓痛点

时效性：T+1 模式，时效性差

数据更新只能 overwrite，耗费资源

（2）实时数仓痛点

数据一致性问题：维护麻烦

历史数据修正：没有持久化明细数据，需要重跑，流程繁琐

（3）传统的数仓发展方向

流批一体：一套架构，一套代码，可以跑批也可以跑流

==》节省 资源 人力

（4）Hudi 数据湖的优势

将离线的时效性降低到了分钟级别（5~10 分钟）

本身支持增量处理

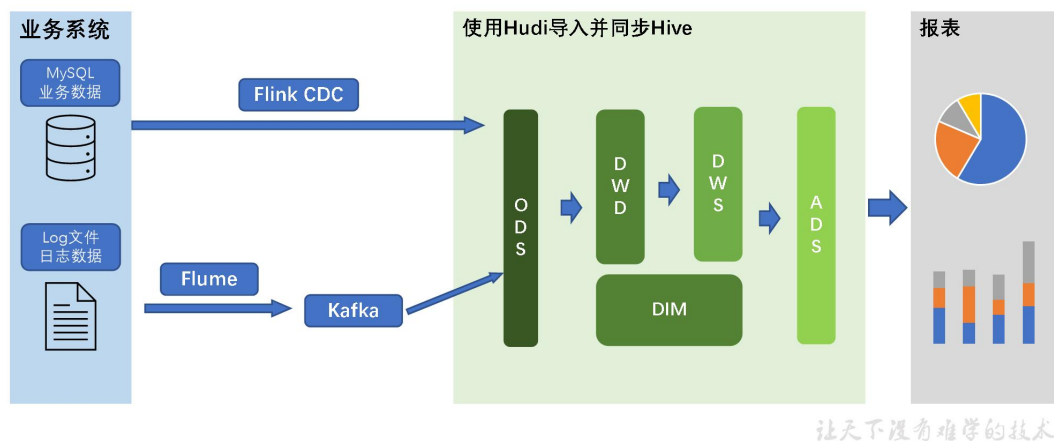
数据更新支持 upsert

随着大数据技术发展趋势，公司对单一的数据湖和数据架构并不满意，想要去融合数据湖和数据仓库，构建在数据湖低成本的数据存储架构之上，又继承数据仓库的数据处理和管理功能。

5.3 项目架构



湖仓一体核心架构



5.4 业务

业务与实时数仓一致。

5.5 优化 or 遇到的问题怎么解决

1) 断点续传采集如何处理

FlinkCDC 分为全量和 binlog，他们都是基于 Flink state 的能力，同步过程会将进度存储在 state 中，如果失败了，下一次会从 state 中恢复即可。

2) 写 Hudi 表数据倾斜问题

FlinkCDC 在全量阶段，读取完一张表后在读取下一张表，如果下游接了多个 Sink，则只有一个 Sink 有数据写入。

使用多表混合读取方式解决。

3) 大状态: regular join + 无 TTL

rocksdb + 增量

4) Hudi 优化

(1) MOR 表，离线 compaction（不跟写入过程绑定在一起）

(2) 相关并发、内存

Compaction、write 并发 =》4

内存: compaction =》1G

(3) 大状态: rocksdb + 增量

5) Hudi 二期规划

(1) 解决大状态问题=》不使用多留 join, 使用“部分列”更新方案

=》hudi<=0.12,官方没有提供, 需要自定义 payload 实现类(大厂实现)

=》0.13.0, 官方加入了 部分列更新的 payload 类

(2) Dws 层换成 olap=》clickhouse, 存储明细

希望 dws 是一种明细, 支持灵活的自助分析=》未来连实时项目也可以干掉

第 6 章 测试&上线流程

6.1 测试相关

6.1.1 公司有多少台测试服务器?

测试服务器一般三台。

6.1.2 测试服务器配置?

有钱的公司和生产环境电脑配置一样。

一般公司测试环境的配置是生产的一半。

6.1.3 测试数据哪来的?

一部分自己写 Java 程序自己造(更灵活), 一部分从生产环境上取一部分(更真实)。

6.1.4 如何保证写的 SQL 正确性(重点)

先在 MySQL 的业务库里面把结果计算出来; 在给你在 ads 层计算的结果进行比较;

需要造一些特定的测试数据, 测试。

从生产环境抓取一部分数据, 数据有多少你是知道的, 运算完毕应该符合你的预期。

离线数据和实时数据分析的结果比较。(日活 1 万 实时 10100), 倾向取离线。

算法异构

实时数据质量监控(脚本、调度器、可视化、故障报警)

(1) 样本数据验证: 从计算结果抽取部分样本数据, 与业务部门实际数据对比。

- (2) 逻辑验证：检查指标的计算 sql 是否正确
- (3) 指标间关系验证：比较不同指标间关系，检查它们是否符合预期。例如：某个指标是另外一个指标的累计值，那么这两个指标一定存在关系
- (4) 历史数据对比：将计算结果和过去数据进行对比，观察指标的变化趋势
- (5) 异常值检测：检查计算结果中是否存在异常值
- (6) 跨数据部门对比：可以将计算结果与其它部门或团队的数据进行对比，进一步验证

6.1.5 测试之后如何上线？

大公司：上线的时候，将脚本打包，提交 git。先发邮件抄送经理和总监，运维。运维负责上线。

小公司：跟项目经理说一下，项目经理技术把关，项目经理通过了就可以上线了。风险意识。

所谓的上线就是编写脚本，并在 DolphinScheduler 中进行作业调度。

6.1.6 A/B 测试了解

1) 什么是 A/B 测试？

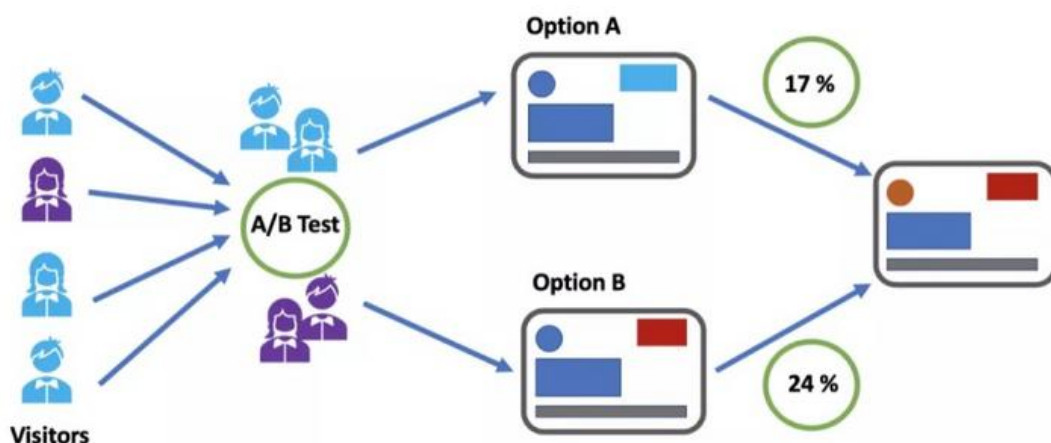
A/B 测试本质上是一种实验，即随机向用户显示变量的两个或多个版本，并使用统计分析来确定哪个变量更适合给定的转化目标。

2) 为什么要做 A/B 测试？

举例：字节跳动有一款中视频产品叫西瓜视频，最早它叫做头条视频。为了提升产品的品牌辨识度，团队想给它起个更好的名字。经过一些内部调研和头脑风暴，征集到了西瓜视频、奇妙视频、筷子视频、阳光视频 4 个名字，于是团队就针对一共 5 个 APP 名称进行了 A/B 实验。

这个实验中唯一改变的是应用市场里该产品的名称和对应的 logo，实验目的是为了验证哪一个应用名称能更好地提升“头条视频”APP 在应用商店的点击率。最后西瓜视频和奇妙视频的点击率位列前二，但差距不显著，结合用户调性等因素的综合考量，最终决定头条视频正式更名为**西瓜视频**。

通过这个案例可以看到，A/B 测试可以帮助业务做最终决策。结合案例的直观感受，我们可以这样来定义 A/B 测试：在同一时间对目标受众做科学抽样、分组测试以评估效果。



以上图图示为例，假设我们有 100 万用户要进行 A/B 测试：

先选定目标受众，比如一线城市的用户。A/B 测试不可能对所有用户都进行实验，所以要科学抽样，选择小部分流量进行实验。

- 抽样之后需要对样本进行分组，比如 A 组保持现状，B 组的某一个因素有所改变。
- 分组之后在同一时间进行实验，就可以看到改变变量后用户行为的变化。
- 再根据对应实验目标的指标，比如点击率的高低，来评估实验的结果。

3) 哪个首页新 UI 版本更受欢迎

今日头条 UI 整体风格偏大龄被诟病已久，不利于年轻和女性用户泛化，历史上几次红头改灰头实验都对大盘数据显著负向。因此团队设计了 A/B 实验，目标是在可接受的负向范围内，改一版用户评价更好的 UI。通过控制变量法，对以下变量分别开展数次 A/B 实验：

头部色值饱和度、字号、字重、上下间距、左右间距、底部 tab icon。

结合用户调研（结果显示：年轻用户和女性用户对新 UI 更偏好）。

综合来看，效果最好的 UI 版本如下图所示，全量上线。



图1: 原 UI 方案

图2: 新 UI 方案

新 UI 上线后, Stay duration 显著负向从-0.38% 降至 -0.24%, 图文类时长显著 +1.66%, 搜索渗透显著 +1.47%, 高频用户 (占 71%) 已逐渐适应新 UI。

6.2 项目实际工作流程

以下是活跃用户需求的整体开发流程。

产品经理负责收集需求: 需求来源与**客户反馈**、**老板的意见**。

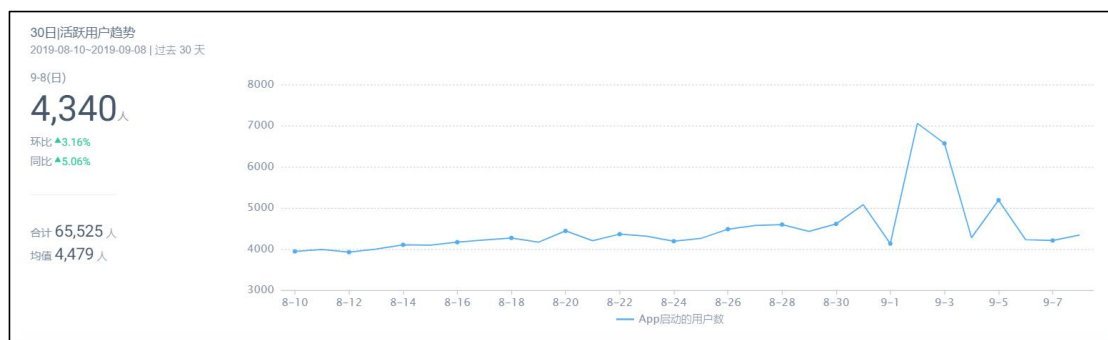
第 1 步: 确定指标的业务口径

由产品经理主导, 找到提出该指标的运营负责人沟通。首先要问清楚**指标是怎么定义的**, 比如活跃用户是指启动过 APP 的用户。设备 id 还是用户 id。

产品经理先编写需求文档并画原型图。=》需求不要口头说。

第 2 步: 需求评审

由产品经理主导设计原型, 对于活跃主题, 我们最终要展示的是**最近 n 天的活跃用户数变化趋势**, 效果如下图所示。此处**大数据开发工程师**、**后端开发工程师**、**前端开发工程师**一同参与, 一起说明整个功能的价值和详细的操作流程, 确保大家理解的一致。



工期：

接口：数据格式、字段类型、责任人。

第 3 步：大数据开发

大数据开发工程师，通过数据同步的工具如 **Flume**、**Datax**、**Maxwell** 等将数据同步到 **ODS** 层，然后就是一层一层的通过 **SQL** 计算到 **DWD**、**DWS** 层，最后形成可为应用直接服务的数据填充到 **ADS** 层。

第 4 步：后端开发

后端工程师负责，为大数据工程师提供业务数据接口。

同时还负责读取 **ADS** 层分析后，写入 **MySQL** 中的数据。

第 5 步：前端开发

前端工程师负责，前端埋点。

对分析后的结果数据进行可视化展示。

第 6 步：联调

此时**大数据开发工程师**、**前端开发工程师**、**后端开发工程师**都要参与进来。此时会要求大数据开发工程师基于历史的数据执行计算任务，大数据开发工程师承担数据准确性的校验。前后端解决用户操作的相关 **BUG** 保证不出现低级的问题完成自测。

第 7 步：测试

测试工程师对整个大数据系统进行测试。测试的手段包括，边界值、等价类等。

提交测试异常的软件有：**禅道**（测试人员记录测试问题 1.0，输入是什么，结果是什么，跟预期不一样->需要开发人员解释，是一个 **bug**，下一个版本解决 1.1->测试人员再测试。测试 1.1ok->测试经理关闭 **bug**）

1 周开发写代码 =》 2 周测试时间

第 8 步：上线

运维工程师会配合我们的前后端开发工程师更新最新的版本到服务器。此时产品经理要找到该指标的负责人长期跟进指标的准确性。重要的指标还要每过一个周期内部再次验证，从而保证数据的准确性。

6.3 项目当前版本号是多少？多久升级一次版本

敏捷开发（少量需求=>代码编写=>测试=>少量需求=>代码编写=>测试...），又叫小步快跑。

差不多一个月会迭代一次。每月都有节日（元旦、春节、情人节、3.8 妇女节、端午节、618、国庆、中秋、1111/6.1/5.1、生日、周末）新产品、新区域。

就产品或我们提出优化需求，然后评估时间。每周我们都会开会做下周计划和本周总结。（日报、周报、月报、季度报、年报）需求 1 周的时间，周三一定完成。周四周五（帮同事写代码、自己学习工作额外的技术）。

5.1.2

5 是大版本号：必须是重大升级

1：一般是核心模块变动

2：一般版本变化

6.4 项目中实现一个需求大概多长时间

（1）刚入职第一个需求大概需要 7 天左右。对业务熟悉后，平均一天一个需求。

（2）影响时间的因素：对业务熟悉、开会讨论需求、表的权限申请、测试等。新员工培训（公司规章制度、代码规范）

6.5 项目开发中每天做什么事

（1）新需求（活动、优化、新产品、新市场）。 60%

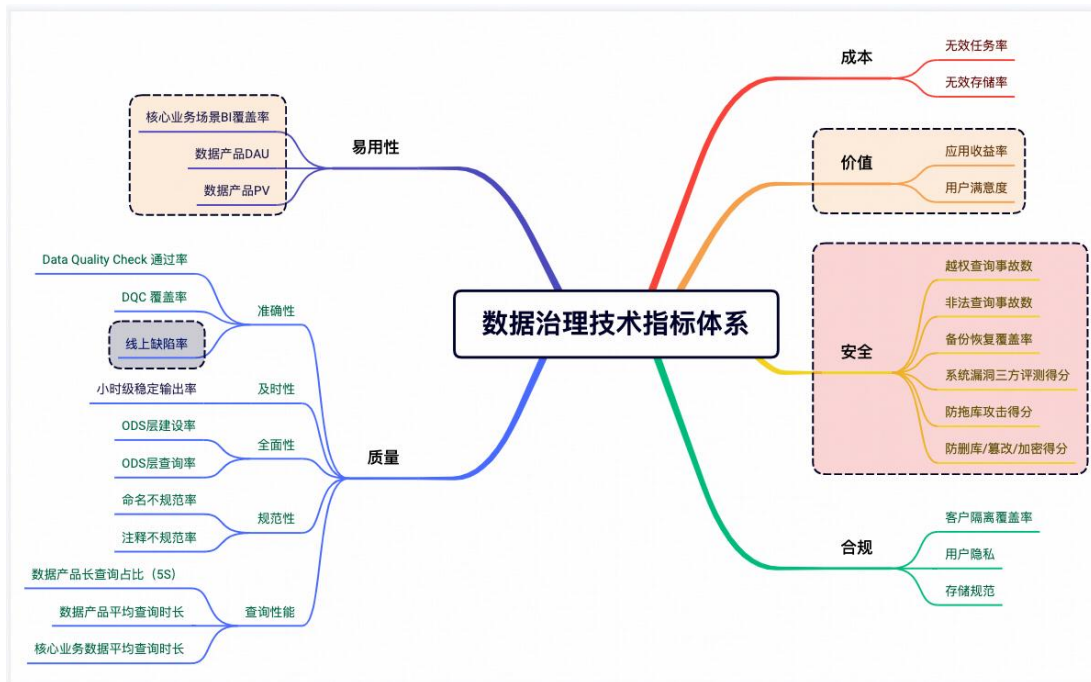
（2）故障分析：数仓的任何步骤出现问题，需要查看问题，比如日活，月活下降或快速上升等。20%

（3）新技术的预言（比如湖仓一体 数据湖 Doris 实时数据质量监控）10%

（4）其临时任务 10%

（5）晨会-》10 做操-》讨论中午吃什么-》12 点出去吃 1 点-》睡到 2 点-》3 点茶歇水果-》晚上吃啥-》吃加班餐-》开会-》晚上 6 点吃饭-》7 点开始干活-10 点-》11 点

第7章 数据治理



7.1 元数据管理

元数据管理目前开源的框架中，Atlas 框架使用的较多。再就是采用自研的系统。

1) 元数据管理底层实现原理

解析如下 HQL，获取对应的原数据表和目标表直接的依赖关系。

```
insert into table ads_user
```

```
select id, name from dws_user
```

依赖关系能够做到：表级别和字段级别 neo4j

2) 用处：作业执行失败，评估他的影响范围。主要用于表比较多的公司

atlas 版本问题：

0.84 版本：2019-06-21

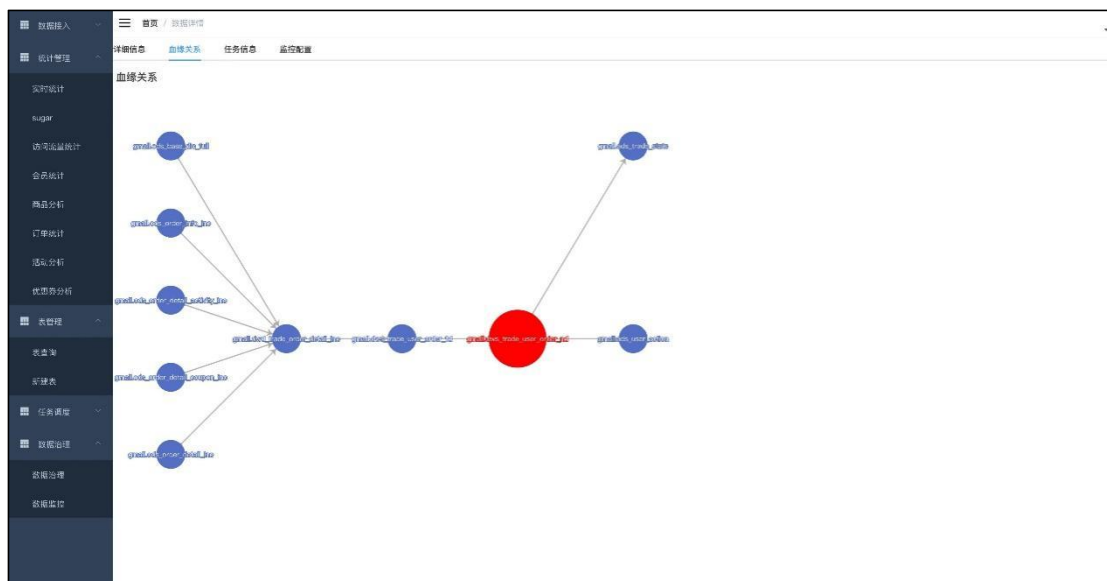
2.0 版本：2019-05-13

框架版本：

Apache 0.84 2.0 2.1

CDH 2.0

3) 尚大自研的元数据管理



7.2 数据质量监控

7.2.1 实现数据质量监控，你具体怎么做，详细说？

实现数据质量检测的功能，我们需要首先明确数据质量的维度，例如准确性、完整性、唯一性、及时性和一致性。

1) 确定数据源

确定需要进行数据质量检测的数据源。这可能是数据库表、文件、API 等。

2) 定义质量规则

为每个数据质量维度定义具体的规则。例如：

- 准确性：检查数据是否符合预期的范围或分布。
- 完整性：检查数据是否存在缺失值或空值。
- 唯一性：检查数据中是否存在重复项。
- 及时性：检查数据是否在预期的时间范围内更新。
- 一致性：检查数据是否符合预定义的格式或标准。

3) 实现检测功能

使用编程语言（如 SQL）编写检测数据质量的函数。这些函数可以包括：

- 数据导入：从数据源导入数据。
- 数据清理：对数据进行预处理，如去除空格、转换数据类型等。
- 应用质量规则：根据定义的质量规则，实现相应的检测函数。例如，检查缺失值、重

复项或数据范围等。

- 输出报告：生成数据质量报告，如将检测结果汇总成表格或可视化图表。

4) 自动化和监控

将数据质量检测功能集成到数据管道或 ETL 过程中，以实现自动化检测。此外，可以设置监控和警报机制，以便在检测到数据质量问题时及时通知相关人员。

7.2.2 监控原则

1) 单表数据量监控

一张表的记录数在一个已知的范围内，或者上下浮动不会超过某个阈值

- SQL 结果：var 数据量 = select count (*) from 表 where 时间等过滤条件
- 报警触发条件设置：如果数据量不在[数值下限, 数值上限], 则触发报警
- 同比增加：如果((本周的数据量 - 上周的数据量)/上周的数据量*100)不在 [比例下线, 比例上限], 则触发报警
- 环比增加：如果((今天的数据量 - 昨天的数据量)/昨天的数据量*100)不在 [比例下线, 比例上限], 则触发报警
- 报警触发条件设置一定要有。如果没有配置的阈值，不能做监控

日活、周活、月活、留存（日周月）、转化率（日、周、月）GMV（日、周、月）

复购率（日周月） 30%

2) 单表空值检测

某个字段为空的记录数在一个范围内，或者占总量的百分比在某个阈值范围内

- 目标字段：选择要监控的字段，不能选“无”
- SQL 结果：var 异常数据量 = select count(*) from 表 where 目标字段 is null
- 单次检测：如果（异常数据量）不在[数值下限, 数值上限], 则触发报警

3) 单表重复值检测

一个或多个字段是否满足某些规则

- 目标字段：第一步先正常统计条数；select count(*) from 表；
- 第二步，去重统计；select count(*) from 表 group by 某个字段
- 第一步的值和第二步不的值做减法，看是否在上下线阈值之内
- 单次检测：如果（异常数据量）不在[数值下限, 数值上限], 则触发报警

4) 单表值域检测

一个或多个字段没有重复记录

- 目标字段：选择要监控的字段，支持多选
- 检测规则：填写“目标字段”要满足的条件。其中\$1 表示第一个目标字段，\$2 表示第二个目标字段，以此类推。上图中的“检测规则”经过渲染后变为“delivery_fee = delivery_fee_base+delivery_fee_extra”
- 阈值配置与“空值检测”相同

5) 跨表数据量对比

主要针对同步流程，监控两张表的数据量是否一致

- SQL 结果：count（本表） - count（关联表）
- 阈值配置与“空值检测”相同

7.2.3 数据质量实现



尚硅谷大数据项目
之电商数仓（9质量



尚硅谷大数据项目
之尚品汇（11数据

Ds3.x

7.3 权限管理（Ranger）



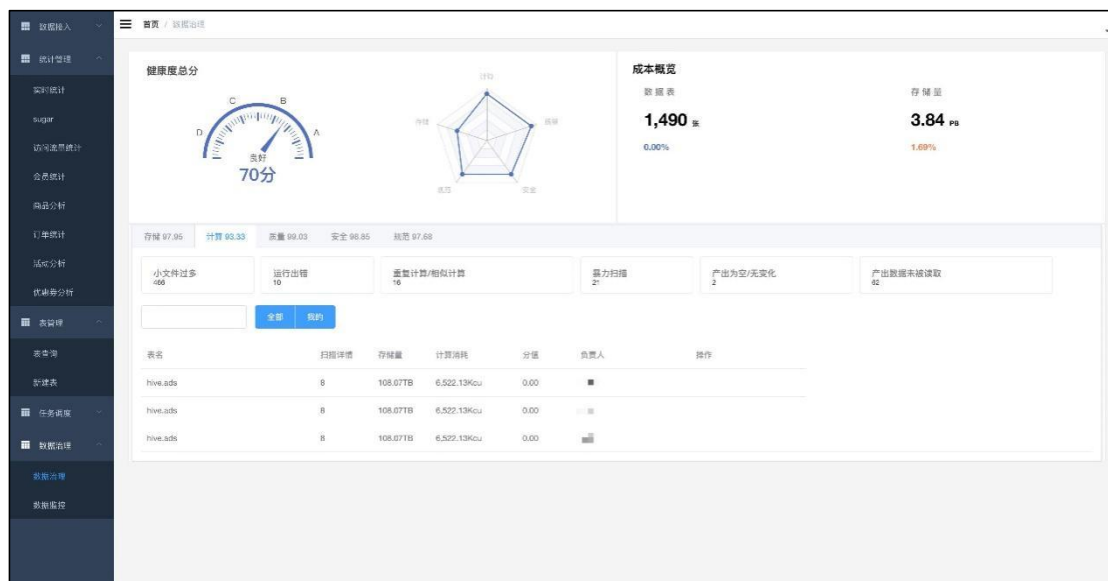
尚硅谷大数据项目
之电商数仓（8权限

7.4 用户认证（Kerberos）



尚硅谷大数据技术
之Kerberos.docx

7.5 数据治理



资产健康度量化模型。

根据数据资产健康管理的关键因素，明确量化积分规则。根据数据基础信息完整度、数据存储和数据计算健康度、数据质量监控规则合理性等，完整计算数据资产健康分。

1) 资产健康分基础逻辑

(1) 健康分基本设定原则：

- 健康分采用百分制，100 最高，0 分最低；
- 健康度以表为最细粒度，每个表都有一个健康分；
- 个人、业务版块、团队、一级部门、以及集团的健康分以所属表的健康分加权平均；
- 数据表权重=（表字节数 + 1）再开立方根；空表的权重为 1；

(2) 数据表资产健康分：

数据表资产健康分 $score = (规范合规健康分 * 10\% + 存储健康分 * 30\% + 计算健康分 * 30\% + 数据质量健康分 * 15\% + 数据安全健康分 * 15\%)$ ；

2) 数据资产特征列表：

| 资产健康类型 | 特征 | 特征分计算逻辑 |
|--------|-----------|---------|
| | 有技术 owner | 0/1 |

| | | |
|---|------------|--|
| 规范 Specification

规范健康分=
$100 * \text{sum}(\text{特征分}) / \text{count}(\text{特征})$ | 有业务 owner | 0/1 |
| | 有分区信息 | 0/1 |
| | 有归属部门 | 0/1 |
| | 表命名合规 | 0/1 |
| | 数仓分层合规 | 0/1 |
| | 表有备注信息 | 0/1 |
| | 字段有备注信息 | 有备注字段数 / 总字段数 |
| 存储 Storage

存储健康分=
$100 * \text{完成度}$ | 生命周期合理性 | <ul style="list-style-type: none"> • 永久保留表: 不可再生源头表、白名单表、冷备表、最近 93 天有访问非分区表, 按 100%完成度; • 未管理表: 分区表但未配置生命周期, 按 0%完成度; • 无访问表: 在 93 天前创建, 但最近 93 天无访问, 按 0%完成度; • 新建表: 创建时间小于 93 天, 尚未积累访问数据和未配置合理生命周期, 默认按 80%完成度; • 普通表: 以上之外普通表, 系统计算建议保留天数与当前生命周期比值作为完成度; |
| 计算 Calc

计算健康分=
$100 * \text{sum}(\text{特征分}) / \text{count}(\text{特征})$ | hdfs 路径被删除 | 被删除, 0 分; 否则为 1 |
| | 产出为空 | 连续 15 天无数据产出, 0 分; 否则为 1; |
| | 产出表未被读取 | 最近 30 天产出数据表无读取, 0 分; 否则为 1; |
| | 运行出错 | 最近 3 天任务运行有出错, 0 分; 否则为 1; |
| | 重复表/相似表 | 与其他数据表 50%相似, 则为 0 分, 否则为 1; |
| | 责任人不合理 | 对应的调度节点责任人已经离职, 或调度节点责任人在职但与数据表责任人不一致, 0 分; 否则为 1; |
| | 简单加工 | 生产 sql 只简单 select 字段出来, 没有 join、group、where 条件只有分区字段; 0 分, 否则为 1; |
| | 暴力扫描 | 表中被查询分区大于 90 天, 同时被查询分区的总存储量大于 100G; 0 分, 否则为 1; |

| | | |
|--|----------|--|
| | 两侧类型不一致 | 类似这个例子:
<pre>select ... from table1 t1 join table2 t2 on t1.a_bigint=t2.a_string;</pre> 这种情况在 on 条件中两边都被 double 了, 这个其实不合理; 是个大坑, 会导致行为和用户期待不一致 |
| | 数据倾斜 | 长尾运行实例耗费时长高于平均值 20%, 分数记为 0, 否则记为 1; |
| | 需要列剪裁 | 判断 select 语句及后续使用逻辑, 是否 select 出来的列都被使用, 用被使用的列数/总 select 列数计算使用率, 低于 50% 就需要列剪裁, 0 分, 否则为 1; |
| 质量 Quality
质量健康分=
$100 * \text{sum}(\text{特征分}) / \text{count}(\text{特征})$ | 表产出时效监控 | qdc 有定义产出时间预警或已经归属于某个生产基线; 0/1; |
| | 表内容监控 | 有配置表级规则; 0/1; |
| | 字段内容监控 | 有配置字段级规则; 0/1; |
| | 表产出 SLA | X 点及时性 SLA 测量函数:
<ul style="list-style-type: none"> • realTime: 数据实际产出时间 • expectTime: 期望数据产出的时间点 • n: 数据产出周期 (针对多次调度) |
| | 表内容 SLA | 1-触发监控规则/总监控规则数 |
| | 字段内容 SLA | 1-触发监控规则/总监控规则数 |
| 安全 Security
安全健康分=
$100 * \text{sum}(\text{特征分}) / \text{count}(\text{特征})$ | 数据分类 | 有明确设置归属的“资产目录”; 0/1; |
| | 资产分级 | 有指定资产等级; 0/1; |
| | 字段级安全等级 | 有字段设置了安全等级; 0/1; |

第 8 章 中台

<https://mp.weixin.qq.com/s/nXI0nSSOneteIClA7dming>

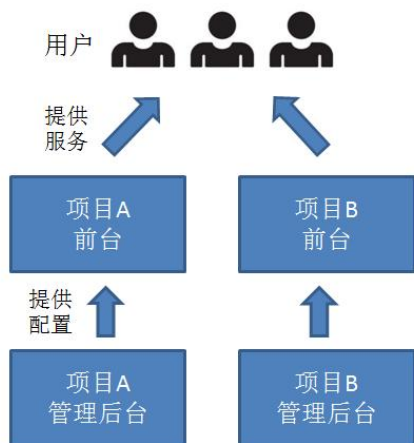
8.1 什么是中台？

1) 什么是前台？

首先，这里所说的“前台”和“前端”并不是一回事。所谓前台即包括各种和用户直接交互的界面，比如 web 页面，手机 app；也包括服务端各种实时响应用户请求的业务逻辑，比如商品查询、订单系统等等。

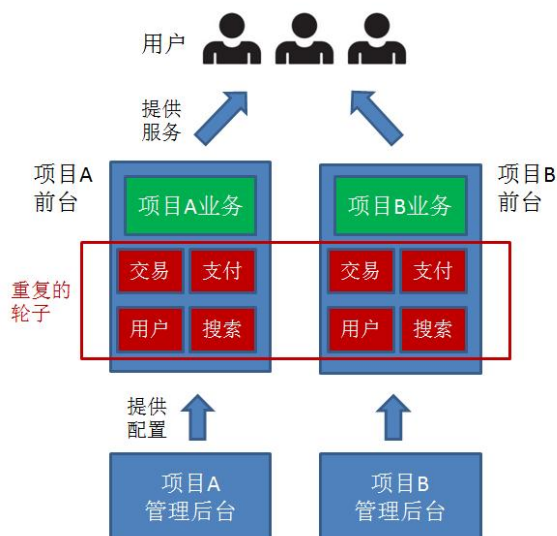
2) 什么是后台？

后台并不直接面向用户，而是面向运营人员的配置管理系统，比如商品管理、物流管理、结算管理。后台为前台提供了一些简单的配置。



3) 为什么要做中台

传统项目痛点：重复造轮子。



8.2 各家中台

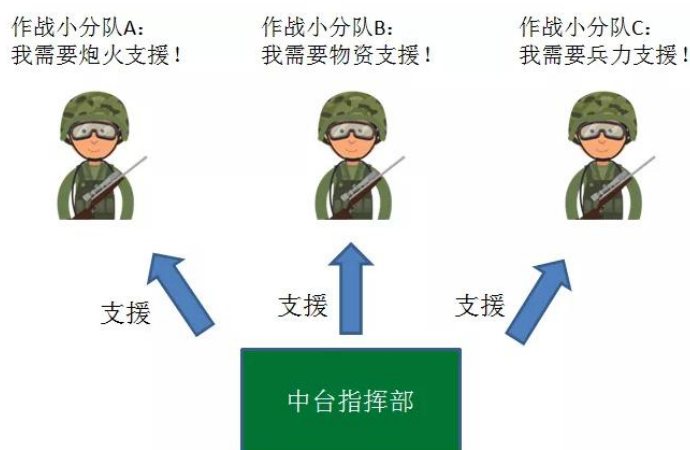
1) SuperCell 公司



2) 阿里巴巴提出了“大中台，小前台”的战略



3) 华为提出了“平台炮火支撑精兵作战”的战略



8.3 中台具体划分

1) 业务中台 & 技术中台



图 业务中台



图 技术中台

2) 数据中台 & 算法中台



图 数据中台



图 算法中台

8.4 中台使用场景

1) 从 0 到 1 的阶段，没有必要搭建中台。

从 0 到 1 的创业型公司，首要目的是生存下去，以最快的速度打造出产品，证明自身的市场价值。

这个时候，让项目野蛮生长才是最好的选择。如果不慌不忙地先去搭建中台，恐怕中台还没搭建好，公司早就饿死了。

2) 从 1 到 N 的阶段，适合搭建中台。

当企业有了一定规模，产品得到了市场的认可，这时候公司的首要目的不再是活下去，而是活的更好。

这个时候，趁着项目复杂度还不是特别高，可以考虑把各项目的通用部分下沉，组建中台，以方便后续新项目的尝试和旧项目的迭代。

3) 从 N 到 N+1 的阶段，搭建中台势在必行。

当企业已经有了很大的规模，各种产品、服务、部门错综复杂，这时候做架构调整会比较痛苦。

但是长痛不如短痛，为了项目的长期发展，还是需要尽早调整架构，实现平台化，

以免日后越来越难以维护。

8.5 中台的痛点

牵一发动全身，中台细小的改动，都需要严格测试。周期比较长。

大厂一般有总的中台，也有部门级别的中台，保证效率。

第9章 算法题（LeetCode）

9.1 时间复杂度、空间复杂度理解

在计算机算法理论中，用时间复杂度和空间复杂度来分别从这两方面衡量算法的性能。

1) 时间复杂度（Time Complexity）

算法的时间复杂度，是指执行算法所需要的计算工作量。

一般来说，计算机算法是问题规模 n 的函数 $f(n)$ ，算法的时间复杂度也因此记做： $T(n) = O(f(n))$ 。

问题的规模 n 越大，算法执行的时间的增长率与 $f(n)$ 的增长率正相关，称作渐进时间复杂度（Asymptotic Time Complexity）。

2) 空间复杂度

算法的空间复杂度，是指算法需要消耗的内存空间。有时候做递归调用，还需要考虑调用栈所占用的空间。

其计算和表示方法与时间复杂度类似，一般都用复杂度的渐近性来表示。同时间复杂度相比，空间复杂度的分析要简单得多。

所以，我们一般对程序复杂度的分析，重点都会放在时间复杂度上。

9.2 常见算法求解思想

1) 暴力求解

不推荐。

2) 动态规划

动态规划（Dynamic Programming, DP）是运筹学的一个分支，是求解决策过程最优化的过程。

动态规划过程是：把原问题划分成多个“阶段”，依次来做“决策”，得到当前的局部解；

每次决策，会依赖于当前“状态”，而且会随即引起状态的转移。

这样，一个决策序列就是在变化的状态中，“动态”产生出来的，这种多阶段的、最优化解，解决问题的过程就称为动态规划（Dynamic Programming，DP）。

3) 分支

对于复杂的最优化问题，往往需要遍历搜索解空间树。最直观的策略，就是依次搜索当前节点的所有分支，进而搜索整个问题的解。为了加快搜索进程，我们可以加入一些限制条件计算优先值，得到优先搜索的分支，从而更快地找到最优解：这种策略被称为“分支限界法”。

分支限界法常以广度优先（BFS）、或以最小耗费（最大效益）优先的方式，搜索问题的解空间树。

9.3 基本算法

9.3.1 冒泡排序

冒泡排序是一种简单的排序算法。

它的基本原理是：重复地扫描要排序的数列，一次比较两个元素，如果它们的大小顺序错误，就把它交换过来。这样，一次扫描结束，我们可以确保最大（小）的值被移动到序列末尾。这个算法的名字由来，就是因为越小的元素会经由交换，慢慢“浮”到数列的顶端。

冒泡排序的时间复杂度为 $O(n^2)$ 。

```
public void bubbleSort(int nums[]) {
    int n = nums.length;
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(nums[j + 1] < nums[j])
                swap(nums, j, j + 1);
        }
    }
}
```

9.3.2 快速排序

快速排序的基本思想：通过一趟排序，将待排记录分隔成独立的两部分，其中一部分记录的关键字均比另一部分的关键字小，则可分别对这两部分记录继续进行排序，以达到整个序列有序。

快排应用了分治思想，一般会用递归来实现。

快速排序的时间复杂度可以做到 $O(n \log n)$ ，在很多框架和数据结构设计中都有广泛的

应用。

```
public void qSort(int[] nums, int start, int end){

    if (start >= end) return;

    int mid = partition(nums, start, end);
    qSort(nums, start, mid - 1);
    qSort(nums, mid + 1, end);
}

// 定义分区方法，把数组按一个基准划分两部分，左侧元素一定小于基准，右侧大于基准
private static int partition( int[] nums, int start, int end ){
    // 以当前数组起始元素为 pivot
    int pivot = nums[start];
    int left = start;
    int right = end;

    while ( left < right ){
        while ( left < right && nums[right] >= pivot )
            right--;
        nums[left] = nums[right];

        while ( left < right && nums[left] <= pivot )
            left++;
        nums[right] = nums[left];
    }
    nums[left] = pivot;
    return left;
}
```

9.3.3 归并排序

归并排序是建立在归并操作上的一种有效的排序算法。该算法是采用分治法（Divide and Conquer）的一个非常典型的应用。

将已有序的子序列合并，得到完全有序的序列；即先使每个子序列有序，再使子序列段间有序。若将两个有序表合并成一个有序表，称为 2-路归并。

归并排序的时间复杂度是 $O(n\log n)$ 。代价是需要额外的内存空间。

```
public void mergeSort(int[] nums, int start, int end){

    if (start >= end ) return;
    int mid = (start + end) / 2;

    mergeSort(nums, start, mid);
    mergeSort(nums, mid + 1, end);

    merge(nums, start, mid, mid + 1, end);
}

private static void merge(int[] nums, int lstart, int lend, int rstart, int rend){
    int[] result = new int[rend - lstart + 1];
```

```

int left = lstart;
int right = rstart;
int i = 0;

while (left <= lend && right <= rend){
    if (nums[left] <= nums[right])
        result[i++] = nums[left++];
    else
        result[i++] = nums[right++];
}

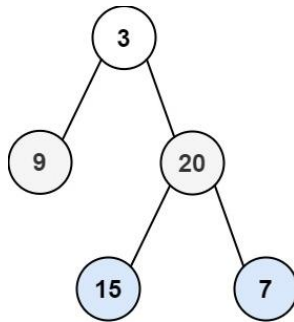
while (left <= lend)
    result[i++] = nums[left++];
while (right <= rend)
    result[i++] = nums[right++];

System.arraycopy(result, 0, nums, lstart, result.length);
}

```

9.3.4 遍历二叉树

题目：求下面二叉树的各种遍历（前序、中序、后序、层次）



- 中序遍历：即左-根-右遍历，对于给定的二叉树根，寻找其左子树；对于其左子树的根，再去寻找其左子树；递归遍历，直到寻找最左边的节点 i，其必然为叶子，然后遍历 i 的父节点，再遍历 i 的兄弟节点。随着递归的逐渐出栈，最终完成遍历
- 先序遍历：即根-左-右遍历
- 后序遍历：即左-右-根遍历
- 层序遍历：按照从上到下、从左到右的顺序，逐层遍历所有节点。

9.3.5 二分查找

给定一个 n 个元素有序的（升序）整型数组 **nums** 和一个目标值 **target**，写一个函数搜索 **nums** 中的 **target**，如果目标值存在返回下标，否则返回-1。

二分查找也称折半查找（Binary Search），它是一种效率较高的查找方法，前提是数据结构必须先排好序，可以在对数时间复杂度内完成查找。

二分查找事实上采用的就是一种分治策略，它充分利用了元素间的次序关系，可在最坏的情况下用 $O(\log n)$ 完成搜索任务。

```
/**
 * @param a 要查找的有序 int 数组
 * @param key 要查找的数值元素
 * @return 返回找到的元素下标；如果没有找到，返回-1
 */
public int binarySearch(int[] a, int key){
    int low = 0;
    int high = a.length - 1;

    if ( key < a[low] || key > a[high] )
        return -1;

    while ( low <= high ){
        int mid = ( low + high ) / 2;

        if( a[mid] < key)
            low = mid + 1;
        else if( a[mid] > key )
            high = mid - 1;
        else
            return mid;
    }

    return -1;
}
```

9.4 小青蛙跳台阶

题目：一只青蛙一次可以跳上 1 级台阶，也可以跳上 2 级台阶。求该青蛙上一个 n 级台阶总共有多少种跳法？

9.5 最长回文子串

题目：给你一个字符串 s ，找到 s 中最长的回文子串。

实例：

输入： $s = \text{"babad"}$

输出： "bab"

解释： "aba" 也是符合题意答案

9.6 数字字符转化成 IP

题目：现在有一个只包含数字的字符串，将该字符串转化成 IP 地址的形式，返回所有可能的情况。

例如：

给出的字符串为 “25525511135”

返回["255.255.11.135", "255.255.111.35"]（顺序没有关系）

第 10 章 场景题

10.1 手写 Flink 的 UV

10.2 Flink 的分组 TopN

10.3 Spark 的分组 TopN

1) 方法 1:

(1) 按照 key 对数据进行聚合 (groupByKey)

(2) 将 value 转换为数组, 利用 scala 的 sortBy 或者 sortWith 进行排序 (mapValues)

数据量太大, 会 OOM。

2) 方法 2:

(1) 取出所有的 key

(2) 对 key 进行迭代, 每次取出一个 key 利用 spark 的排序算子进行排序

方法 3:

(1) 自定义分区器, 按照 key 进行分区, 使不同的 key 进到不同的分区

(2) 对每个分区运用 spark 的排序算子进行排序

10.4 如何快速从 40 亿条数据中快速判断, 数据 123 是否存在

10.5 给你 100G 数据, 1G 内存, 如何排序?

10.6 公平调度器容器集中在同一个服务器上?

10.7 匹马赛跑，1 个赛道，每次 5 匹进行比赛，无法对每次比赛计时，但知道每次比赛结果的先后顺序，最少赛多少次可以找出前三名？

10.8 给定一个点、一条线、一个三角形、一个有向无环图，请用 java 面向对象的思想进行建模

10.9 现场出了一道 sql 题，让说出 sql 的优化，优化后效率提升了多少

```
select 2d from t_order where 2d in (SELECT 2d from t_order_f)
```

对于这条 SQL 语句，可以使用内连接（INNER JOIN）来代替子查询（IN）。这通常可以提高查询性能，因为内连接在大多数数据库系统中的性能优化更为成熟。以下是优化后的 SQL 语句：

```
SELECT t1.2d  
FROM t_order t1  
INNER JOIN t_order_f t2 ON t1.2d = t2.2d
```

优化后，如何判断效率提升了多少？

查看执行时间：执行优化前后的 SQL 语句，比较它们的执行时间。执行时间的减少表示性能得到了提升。

第 11 章 HQL 场景题

尚大自研刷题网站的网址：<http://forum.atguigu.cn/interview.html>

HQL 刷题模块，刷分到 1000 分以上。

第 12 章 面试说明

12.1 面试过程最关键的是什么？

- （1）大大方方的聊，放松
- （2）体现优势，避免劣势

12.2 面试时该怎么说？

- 1) 语言表达清楚

- (1) 思维逻辑清晰，表达流畅
 - (2) 一二三层次表达
- 2) 所述内容不犯错
- (1) 不说前东家或者自己的坏话
 - (2) 往自己擅长的方面说
 - (3) 实质，对考官来说，内容听过，就是自我肯定；没听过，那就是个学习的过程。

12.3 面试技巧

12.3.1 六个常见问题

1) 你的优点是什么？

大胆的说出自己各个方面的优势和特长

2) 你的缺点是什么？

不要谈自己真实问题；用“缺点”衬托自己的优点

3) 你的离职原因是什么？

- 不说前东家坏话，哪怕被伤过
- 合情合理合法
- 不要说超过 1 个以上的原因

4) 您对薪资的期望是多少？

- 非终面不深谈薪资
- 只说区间，不说具体数字
- 底线是不低于当前薪资
- 非要具体数字，区间取中间值，或者当前薪资的+20%

5) 您还有什么想问的问题？

- 这是体现个人眼界和层次的问题
- 问题本身不在于面试官想得到什么样的答案，而在于你跟别的应聘者的对比
- 标准答案：

公司希望我入职后的 3-6 个月内，给公司解决什么样的问题

公司（或者对这个部门）未来的战略规划是什么样子的？

以你现在对我的了解，您觉得我需要多长时间融入公司？

6) 您最快多长时间能入职?

一周左右, 如果公司需要, 可以适当提前。

12.3.2 两个注意事项

(1) 职业化的语言

(2) 职业化的形象

12.3.3 自我介绍

1) 个人基本信息

2) 工作履历

时间、公司名称、任职岗位、主要工作内容、工作业绩、离职原因。